СБЕР

Руководство Whitepaper

AI-Disrupt PDLC

Инженерия намерения. Новая архитектура генеративной разработки ПО

Кирилл Меньшов

CIO Сбер

Релиз 2.0, июнь 2026

Оглавление

Резюме для руководства **1**
Формула-1: четыре контура 1
Четыре архитектурных принципа 1
Среда важнее модели 2
Восемь частей методологии 3
Ключевые термины 3
Часть 1. Основания для трансформации **6**
1.1 Экономический императив трансформации 6
1.2 Принцип «Среда важнее модели» 9
1.3 Рынок агентного ИИ: реальность против ажиотажа 11
1.4 ИИ как усилитель команды 13
1.5 Подходы к интеграции ИИ в разработку 14
1.6 Стратегические прогнозы 15
1.7 Адаптация или перепроектирование как драйвер ИИ-эффекта 16
1.8 Рамка принятия решения на стадии Discovery 18
1.9 Ориентированность на измеримый результат 21
1.10 Императив контура управления (Governance Imperative) 21
Часть 2. Архитектурное ядро PDLC **24**
2.1 Двухпетлевая модель с ИИ-валидацией 24
2.2 Петля намерения 26
2.3 Среда работы ИИ-агента важнее модели 29
2.4 Обязательный и рекомендуемый контекст 41
2.5 Агентное кодирование: уровни автономии и task horizon 41
2.6 Децентрализация IDE 42
2.7 Архитектурные паттерны агентных систем 43
2.8 Контекст-инжиниринг и сжатие контекста 44
2.9 Мультиагентные системы: паттерны проектирования 45
2.10 Долгосрочные агенты: внутренний и внешний цикл, передача сессии 46
2.11 Eval-driven development и evidence bundle как контракт завершения 48
2.12 Слой сквозной ИИ-валидации 49
2.13 Контур управления (Governance Mesh): третья ось PDLC 51
Часть 3. Роли, команды, компетенции **53**
3.1 От исполнителя к оркестратору 53
3.2 Новые роли в ИИ-нативной команде 54
3.3 Компетенции инженера в AI-Disrupt PDLC 56
3.4 Tiny teams: модель компактных команд 61

3.5 Влияние на рынок труда и навыки 64
3.6 Уровни зрелости команды L0–L5 68
Часть 4. Инфраструктурная платформа разработки (IDP) 72
4.1 Integrated Development Platform (IDP) 72
4.2 Среда важнее модели 76
4.3 Управление и автономия 92
4.4 Контур разработки агентного продукта 96
4.5 Guardian Agents: архитектура и уровни зрелости 99
Часть 5. Безопасность, governance, эксплуатация 103
5.1 Контур управления 103
5.2 Токеномика, cost-to-outcome и динамическое распределение вычислений 116
Часть 6. Метрики и измерения 121
6.1 Категории метрик по AWS 122
6.2 Сводная система метрик PDLC 123
6.3 Причины искажения метрик 129
6.4 Управление риском через метрики 131
Часть 7. Стратегия внедрения 136
7.1 Принципы внедрения 137
7.2 Горизонты внедрения 139
7.3 Событийно-управляемые агенты в PDLC 142
7.4 Антипаттерны трансформации 144
7.5 Трансформация людей и инструментов 148
Часть 8. Российский корпоративный контекст 152
8.1 Регуляторные контуры 152
8.2 Ландшафт поставщиков и кадров 154
8.3 Policy-as-Code Library «RU Compliance Core» 156
8.4 Governance Mesh для российского контекста 157
8.5 Governance metrics — российская специфика 158
8.6 Guardian Agents в российском контексте 160
8.7 Референс-архитектура 161
8.8 Дорожная карта локализации 164
8.9 Выводы 165
Заключение 166
Глоссарий 167

AI-Disrupt PDLC

ИИ-нативная трансформация разработки ПО для зрелого корпоративного контура

Практическое руководство

Версия 2.0, июнь 2026

Сбер

Резюме для руководства

Формула-1: четыре контура

Формула-1 — отличная метафора для AI-Disrupt PDLC. Гонка демонстрирует, как намерение, реализация, валидация и управление работают параллельно. Команда в боксе перед гонкой и в каждом пит-стопе формирует план: резина, дозаправка, атака соперника — это петля намерения. Пилот ведёт болид по трассе и принимает решения в каждом повороте — это петля реализации.

Телеметрия в реальном времени передаёт давление в шинах, температуру тормозов, нагрузку на мотор и скорость в каждой точке — это контур валидации (Validation Spine): правильно ли едем. Стены трассы, штраф за пересечение белой линии, ограничения скорости в пит-лейне, регламент FIA и обязательный логгер на болиде встроены в саму гонку и срабатывают автоматически — это сквозной контур управления (Governance Mesh): по правилам ли едем. Пилот не теряет скорость от того, что телеметрия и регламент работают параллельно. Без них он либо разобьётся, либо получит дисквалификацию.

AI-Disrupt PDLC — это архитектура, в которой жизненный цикл разработки ПО переосмыслен вокруг трёх сдвигов:

- ИИ-агент — субъект петли реализации, а человек — субъект петли намерения.
- Качество результата определяет среда работы вокруг модели, а не сама модель.
- Контур управления встраивается в каждый шаг, а не накладывается в конце.

Эти три сдвига формулируют центральный тезис:

Модель задаёт минимально доступное качество. Среда работы определяет возможности того, что можно достичь.

Четыре архитектурных принципа

Принцип	**Что означает**
Среда работы ИИ-агентов важнее модели	Инвестиции в среду исполнения накапливают ценность с каждым поколением моделей; инвестиции в подписки — нет
Исследование перед генерацией	Спецификация без исследования воспроизводит старый процесс на новой технологии
Валидация встроена, не добавлена	Отдельная стадия ревью разрушает принцип бесшовной разработки без каких-либо препятствий и задержек (Zero Friction); контур валидации пронизывает обе петли намерения и реализации
Контур управления — третья ось PDLC	Контур управления работает в реальном времени, а не постфактум — кодифицированные правила, риск-адаптивная автономия, аудируемый след

Методологическую основу архитектуры задает ряд компонентов агентной системы: агент, оркестратор, контекст, инструменты, политики, аудит, выходы, оценки, события, контур

управления (governance). Эта таксономия пронизывает все восемь Частей концепции — от двухпетлевой модели до платформенного слоя IDP.

Эффект встроенной само-коррекции измерим. В отраслевом бенчмарке DeepSense.ai на 225 задачах агентный контур с циклами ревью и автотестов поднял долю корректных решений с 53,8% (одиночный запрос модели) до 81,8% — ценой роста стоимости с $0,04 до $0,61 на задачу. Это не теоретический аргумент, а просчитанный результат архитектурного выбора: за автономию платят токенами.

Ключевой вывод: сложные схемы оркестрации экономически нецелесообразны вне бигтеха — стоимость координации агентов превышает выигрыш от параллелизма. Приоритет — фундаментальные паттерны и паттерны управления; мультиагентную оркестрацию вводят лишь после устойчивого результата на одном агенте.

Среда важнее модели

По данным крупнейших технологических консультантов, один разработчик с одной моделью в разных средах показывает разброс результата в 22 п. п. Разрыв между лучшими и худшими моделями на одних и тех же задачах — лишь 1–3 п. п. Инвестиции в среду дают в разы больше, чем смена модели.

По прогнозам мировых исследовательских компаний, к 2028 году доля ручного кода сократится с 50% до 25%. Это произойдёт не за счёт замены разработчика, а за счёт сдвига его функции от написания к проектированию, валидации и оркестрации. Согласно данным ключевых исследовательских компаний, организации со зрелой средой работы ИИ-агентов сократят количество дефектов после развёртывания на 60% к 2028 году; организации без зрелой среды — останутся в проигрыше.

Прирост производительности при зрелом подходе измеряется по объёму кода производственного качества:

Подход	**Строк кода/месяц**
Разработчик без ИИ	~ 2 000
Базовый ИИ-ассистент	~ 3 000
ИИ-нативная разработка через спецификацию (SDD)	5 000–8 000
Скоординированные команды агентов (L5)	15 000–25 000

Речь идёт не о повышении эффективности, а о качественно ином масштабе возможностей. Задачи, прежде требовавшие 20 человеко-лет, выполняются командой из 4–6 разработчиков за 3 месяца.

Восемь частей методологии

Документ охватывает восемь взаимосвязанных частей:

- Часть 1 — экономический императив, рыночная реальность, три подхода к интеграции ИИ, стратегические прогнозы, контур управления.
- Часть 2 — двухпетлевая модель, среда работы ИИ-агентов, разработка через спецификацию (Specification-Driven Development, SDD), агентное кодирование, мультиагентные системы, разработка на основе оценок (eval-driven development), контур управления (governance). Session Handoff Protocol обеспечивает ключевой операционный механизм: при смене контекста между сессиями агент передаёт состояние без потери когерентности задачи.
- Часть 3 — трансформация ролей, шесть ключевых компетенций, модель компактных команд (Tiny Teams), уровни зрелости L0–L5.
- Часть 4 — интегрированная платформа разработки (Integrated Development Platform, IDP): архитектура, 15 ключевых возможностей, риск-адаптивная лестница R0–R5, золотой путь.
- Часть 5 — безопасность, контур управления, эксплуатация: модель угроз, токеномика, комплаенс, ИИ SRE как замыкающее звено PDLC. Среди критических рисков ИИ-нативной разработки — усталость от подтверждений (Approval Fatigue): когда частота запросов на подтверждение превышает порог когнитивной нагрузки команды, контроль деградирует в формальность.
- Часть 6 — метрики и измерения: DORA 2025, объём вывода ИИ, парные метрики, ADLC-фреймворк, метрики эффективности контура управления.
- Часть 7 — стратегия внедрения: принципы, три горизонта, 90-дневный MVP, событийные триггеры, антипаттерны. Наиболее распространённые антипаттерны первого года — от имитации ИИ-нативного процесса поверх старых процедур до преждевременного масштабирования автономии без готовой среды.
- Часть 8 — российский корпоративный контекст: регуляторный контур, рынок поставщиков и специалистов, локализация мировой референс-архитектуры.

Методология не является хронологией версий. Это живая архитектурная позиция — единая система взглядов на то, как строить ИИ-нативную разработку сейчас и в ближайшие годы.

Ключевые термины

Данное руководство опирается на набор терминов, которые встречаются в резюме и раскрываются подробно в основных частях.

Термин	**Что означает**
PDLC (Product Development Life Cycle)	Жизненный цикл разработки продукта: от проблемы пользователя до работающей фичи в проде и её эксплуатации. Шире SDLC (Software Development Life Cycle) — включает этап исследования (Discovery), продуктовую валидацию и операционный контур.
AI-Disrupt PDLC	Название самой концепции; переосмысление PDLC вокруг трёх сдвигов: ИИ-агент как субъект реализации, среда выше модели, контур управления встроен в каждый шаг.

Термин	**Что означает**
PDLC (Product Development Life Cycle)	Жизненный цикл разработки продукта: от проблемы пользователя до работающей фичи в проде и её эксплуатации. Шире SDLC (Software Development Life Cycle) — включает этап исследования (Discovery), продуктовую валидацию и операционный контур.
Среда работы ИИ-агентов (harness)	Окружение вокруг модели: промпты, инструменты, контекст, память, правила-ограничители (guardrails), протоколы, телеметрия. Сильная среда со слабой моделью даёт больше, чем сильная модель без среды.
Zero Friction	Операционное свойство среды, при котором путь от намерения до выпуска не содержит организационных швов: нет отдельной стадии ревью, передач между функциями, ожидания согласований. Контроль встроен в каждый шаг, а не вынесен в конец.
Discovery	Исследование до генерации: PR/FAQ, измеримая гипотеза результата, решение «адаптация/перепроектирование», выбор класса применимости агента. Шире PRD: фиксирует риски и обратимость действий агента до того, как он начнёт их выполнять.
SDD (Spec-Driven Development)	Разработка через спецификацию: источник истины — спецификация, а не код. Агент генерирует все артефакты (код, тесты, документацию) из спецификации; изменения всегда идут через обновление спецификации.
IDP (Integrated Development Platform)	Платформа-операционная система для агентов и разработчиков: 15 ключевых возможностей от каталога инструментов и протоколов (MCP, A2A) до агентов-стражей (Guardian Agents), ModelOps и золотого пути внедрения.
Контур управления (governance)	Третья ось PDLC наряду с намерением и реализацией: контур управления, работающий в реальном времени — кодифицированные правила, риск-адаптивная автономия, аудируемый след. Разлит по всей среде, а не вынесен отдельной стадией.
R0–R5	Риск-адаптивная лестница разрешений агента: от «только подсказывает» (R0) до «работает автономно в очерченном домене» (R5). Класс определяет правила-ограничители, политику подтверждений и требования к агентам-стражам.
L0–L5	Шкала зрелости инженерной организации от ручного процесса (L0) до скоординированных команд агентов (L5). Не путать с R0–R5 и с L task horizon (длиной задачи агента).
ADLC (Agent Development Life Cycle)	Жизненный цикл разработки самого агента: выбор модели, разработка на основе оценок (eval-driven development), выкатка, мониторинг, деградация. Именно вокруг ADLC построен фреймворк метрик Outcome / Quality / Trust / Lifecycle Health.
Tiny Teams	Модель компактных команд (4–6 человек) с оркестрацией агентов, выполняющих работу, которая раньше требовала 20+ разработчиков.

Термин	Что означает
PDLC (Product Development Life Cycle)	Жизненный цикл разработки продукта: от проблемы пользователя до работающей фичи в проде и её эксплуатации. Шире SDLC (Software Development Life Cycle) — включает этап исследования (Discovery), продуктовую валидацию и операционный контур.
Guardian Agents	Специализированные агенты-стражи: проверяют выходы основных агентов, сравнивают с политиками, блокируют опасные действия, фиксируют аудиторский след.
MCP, A2A, AG-UI	Протоколы взаимодействия: MCP (Model Context Protocol) — инструменты и данные для агента; A2A — агент с агентом; AG-UI — агент с интерфейсом и человеком.
DORA	Набор инженерных метрик (deployment frequency, lead time, change failure rate, MTTR), дополненных ИИ-специфичными показателями в версии DORA 2025.
MTTR (Mean Time To Recovery)	Среднее время восстановления после инцидента в проде.
ИИ SRE (Site Reliability Engineering)	Роль и функция, отвечающая за работоспособность ИИ-систем в проде: мониторинг агентов, детекция деградации, реагирование на инциденты. Замыкает петлю PDLC через эксплуатацию.
Session Handoff Protocol	Механизм передачи состояния между сессиями агента без потери когерентности задачи. Критичен для long-running агентов.
Approval Fatigue	Антипаттерн: частота запросов на подтверждение превышает порог когнитивной нагрузки команды, и контроль деградирует в формальность.

Часть 1. Основания для трансформации

Стратегическое преимущество получают те, кто перестраивает весь жизненный цикл разработки, а не ограничивается встраиванием ИИ-ассистентов поверх существующих процессов. Часть 1 отвечает на вопрос, почему точечные решения упираются в потолок и где проходит граница между локальными улучшениями и изменением класса решаемых задач. На кону — конкурентная позиция на горизонте 2028–2030 годов.

1.1 Экономический императив трансформации

Экономический эффект ИИ в разработке нелинейный. Он возникает из комплексной интеграции по всему жизненному циклу. Точечное ускорение отдельных этапов даёт только локальный прирост; кратный эффект даёт перестройка процессов вокруг собственной среды. На этом строится экономический императив трансформации.

По данным McKinsey (Developer Velocity, 2020), организации с высокой зрелостью разработки (Developer Velocity Index, DVI) демонстрируют рост выручки в 4–5 раз выше, чем аутсайдеры; доходность акций у лидеров DVI на 60% выше, инновационность — на 55%. ИИ-генерируемый код создаётся на 35% быстрее, но без надлежащего ревью вводит на 25% больше уязвимостей. Это не повод отказываться от ИИ — это повод строить среду, в которой скорость не жертвует качеством.

Критически важный вывод о нелинейности отдачи: организации, применяющие ИИ в десяти и более сценариях процесса разработки (PDLC), в 66% случаев сообщают о росте скорости выхода на рынок — против 35% при менее чем пяти сценариях. Применение в одном-двух сценариях даёт локальный эффект; сквозная интеграция по всему PDLC даёт синергию.

Наиболее результативные сотрудники с ИИ-навыками в 2,8 раза чаще остальных переосмысливают рабочий процесс при внедрении ИИ (55% против 20%). Внедрения, ориентированные на сокращение затрат, дают локальный прирост; ориентированные на рост и инновации — кратный.

Это выражается формулой:

ИИ-эффект = среда ИИ-агентов × редизайн процессов.

Без среды работы ИИ-агентов (harness) инструменты не масштабируются за пределы пилотов; без переосмысления процессов среда лишь воспроизводит старый процесс на новой технологии.

Три потока создания ценности

Экономический эффект складывается из трёх потоков ценности:

- сокращение прямых затрат — экономия часов;
- ускорение времени выхода на рынок (time-to-market);
- расширение объёма работ — задачи, которые раньше не выполнялись.

Без перенаправления сэкономленных часов в новый объём работ метрики DORA остаются «зелёными», а финансовый эффект для бизнеса оказывается нулевым. Центральный тезис обзора Bain 2025: сэкономленное время не направляется на более ценную работу, и именно поэтому ИИ-инвестиции не окупаются.

Здесь стоит сделать оговорку: нарратив Bain — это интерпретация результатов, а не факт.

Сэкономленное время разработчик мог потратить на задачи, которые сам считает более ценными — исследовательские, рефакторинг технического долга, обучение, — но которые не отражаются в продуктовых метриках; или просто перестать перерабатывать. С точки зрения P&L организации эффект тот же: видимый результат на единицу инвестиций не растёт линейно. Без явного механизма перенаправления экономии в новый объём работ экономический императив не материализуется.

По данным ведущих исследовательских компаний, инженеры тратят около 50% времени на задачи, связанные с кодом в широком смысле, и лишь около 25% — на написание нового кода. По оценке Bain (From Pilots to Payoff, 2025), написание и тестирование кода занимает 25–35% времени; остальные 65–75% приходятся на этапы до написания кода (аналитика, проработка архитектуры) и после него (ревью, интеграционное тестирование, отладка производительности, внедрение).

Вместе эти оценки дают диапазон: на прямое написание кода приходится от 25% до 50%, остальное — работа вокруг него. Сигнал ясен: ИИ-инвестиции, направленные только на генерацию кода, не покрывают большую часть задачи. Антипаттерн «ИИ только в кодировании» напрямую следует из этой статистики.

Эта пропорция отражает усреднённую картину; в реальной практике структура трудозатрат сильно зависит от класса задачи. Простые изменения с быстрым выходом на рынок имеют более высокую долю кодогенерации; крупные тиражируемые изменения — заметно более низкую, поскольку доля исследования, планирования, ревью и интеграции в них выше. Точечная ИИ-помощь при кодировании не меняет структуру трудозатрат кардинально.

Юнит-экономика разработки с ИИ

Юнит-экономика разработки с ИИ меняет структуру затрат: один и тот же старший разработчик показывает принципиально разный объём кода в зависимости от подхода.

Подход	**Производительность**	**Примечание**
Без ИИ	~2 000 строк/мес.	Промышленное качество, полная ответственность
ИИ как дополнение (Copilot, ChatGPT для ревью)	~3 000 строк/мес.	Новая динамика времени выхода на рынок, переосмысление процессов
ИИ-нативный подход + SDD	~5 000–8 000 строк/мес.	Дальнейшее сокращение времени выхода на рынок, запуск ИИ-механизмов валидации
Координированные агентные команды (L5)	~15 000–25 000 строк/мес.	Эквивалентное качество через оркестрацию агентов, малые команды

Прирост в 5–10 раз меняет класс достижимых задач:

задачи, прежде требовавшие 20 человеко-лет, выполняет команда из 4–6 разработчиков за 3 месяца. Это не повышение эффективности, а качественно иной масштаб возможностей. Оценка опирается на внутренние наблюдения Сбера и публичные вендорские бенчмарки (GitHub Copilot: до 55% ускорения в контролируемом тесте и 10–15% в типовой работе;

Accenture: +84% успешных сборок). Количество строк здесь — грубая оценка масштаба сдвига, а не целевая метрика: она иллюстрирует порядок величины, но не измеряет ценность кода.

Среда работы ИИ-агентов — капитальный актив, а вендорские подписки — операционный расход без накопительного эффекта.

Построенная среда окупается на тысячах часов агентного времени и сотнях задач: чем больше команда её использует, тем ниже маржинальная стоимость одной агентной операции. Экономика та же, что у платформы данных (data platform): дорогая в построении, дешёвая в эксплуатации, плохо переносимая.

Затраты на ИИ распределяются по трём уровням:

Уровень	**Состав**	**Тип затрат**	**Горизонт окупаемости**	**ROI на 2+ года**
1 — доступ	Cursor, Copilot, Claude Code и подобные	Операционные	Месяцы	1,1–1,3×
2 — контекст	Корпоративный ИИ-шлюз, стандартизованный доступ агентов к данным, аудит-логирование	Капитальные	~1,5 года	1,5–2×
3 — память и автономия	Библиотека архитектурных паттернов, среда для длинных агентных сессий, объяснимость в реальном времени	Капитальные	~3 года	3–5×

На 1 уровне у каждого инженера есть ИИ-помощник, но он не знает корпоративного контекста. На 2 уровне ИИ работает с нашей кодовой базой и политиками, и скорость масштабируется на команды. На 3 уровне система накапливает успешные решения, а агенты работают над задачами часами без надзора.

Антипаттерн «полный цикл от вендора — под ключ»

Многие коммерческие ИИ-вендоры предлагают решения «под ключ», обещая закрыть весь цикл одной платформой. Это разумный путь для небольших команд без внутренней возможности построить собственную интегрированную платформу разработки, но ловушка для крупных организаций (enterprise) с регуляторными обязательствами и большим масштабом. **Привязка к вендору (vendor lock-in) через консолидированную платформу создаёт три риска:**

- зависимость от вендора в части дальнейшего развития;
- регуляторную уязвимость при компрометации вендора;
- потерю собственных компетенций команды.

Модульная IDP (внутренняя платформа разработки, internal developer platform) с вендоронезависимыми протоколами (MCP, A2A) и заменяемыми компонентами – способ обойти антипаттерн.

Российский контекст и ИИ-суверенитет

Российский контекст добавляет аргументы в пользу собственной среды: ограничения доступа к публичным вендорам, требования локализации данных и регуляторные требования к аудиту ИИ-систем. Зарубежные инструменты доступны в режиме шлюза (gateway), но не в режиме сквозной оркестрации. Собственная среда — инфраструктурная необходимость, а не опция.

ИИ-суверенитет — возможность сменить базовую модель (в том числе перейти с зарубежной на отечественную) **без** переписывания среды работы ИИ-агентов. Приоритет среды над моделью делает эту смену операционно дешёвой, поскольку детерминированный слой не зависит от конкретного провайдера. Это самостоятельный экономический аргумент в пользу инвестиций в среду работы ИИ-агентов, а не только в подписки.

Стратегия и горизонты трансформации

Важно не строить всё сразу. Трансформация через попытку построить все три уровня одновременно — известный антипаттерн, описанный как «big bang transformation» (трансформация одним скачком): команда пытается перейти на новый стек без проверки гипотез, копит все процессы в новую форму, сталкивается с осознанием через полгода. Последовательный подход по горизонтам позволяет проверять отдачу на каждом этапе и уточнять курс до того, как он зафиксирован на следующем этапе.

Стратегия для российской крупной организации (50+ инженерных команд):

- Год 1: формирование платформенных команд (5–7 человек); MVP платформы разработки с интеграцией отечественной модели; пилот в 3 командах с обновлённым подходом к исследованию (Discovery), SDD и базовой валидации.
- Год 2: масштабирование на 15 команд; библиотека шаблонов; стабильные процессы поддержки; минимальный цикл разработки ИИ-агентов (Agent Development Life Cycle, ADLC).
- Год 3: масштабирование на 30+ команд; ADLC на полном жизненном цикле; хуки кросс-доменного управления; доля многоагентного внедрения ≥30% в ведущих командах.
- Год 4: внедрение по всей организации; закреплённое конкурентное преимущество; агентные продукты в промышленной эксплуатации.

1.2 Принцип «Среда важнее модели»

Модель можно сравнить с мотором, а среду — с болидом. Моторы меняют каждый сезон, болид строят годами, и команда с болидом переживает любую смену мотора, тогда как команда с одним мотором в следующем сезоне начинает с нуля.

Модель задаёт минимально доступное качество. Среда работы определяет возможности того, что можно достичь. Это не метафора, а измеримое соотношение: по данным крупнейших технологических консультантов, разброс качества модели, применяемой в разных средах, достигает 22 процентных пунктов — та же модель показывает успешность (success rate) от 23% до 45% в зависимости от качества среды вокруг неё. При этом разрыв между топовыми моделями на одних и тех же задачах составляет лишь 1–3 процентных пункта. Инвестиция в среду работы ИИ-агентов даёт прирост в 22 п.п.; инвестиция в смену модели — не более 3 п.п. Это измеримая экономика.

Различие между двумя подходами к построению среды принципиально для стратегического выбора:

Характеристика	Среда-как-продукт	Среда-как-слой
Ценность	Остаётся у вендора	Накапливается в организации
Смена модели	Требует пересмотра интеграции	Операционно прозрачна
Корпоративный контекст	Ограничен возможностями платформы	Проектируется под конкретные нужды
Регуляторная прозрачность	Зависит от вендора	Полная аудируемость
Горизонт окупаемости	Быстрый, операционный расход	Капитальный, 1,5–3 года

Среда-как-продукт — обоснованный выбор для старта; среда-как-слой — стратегический выбор для масштаба. Крупная организация, строящаяся на идеологии среда-как-продукт, воспроизводит антипаттерн «полный цикл от вендора» с дополнительным риском: среда, определяющая максимально возможный результат, не принадлежит ей.

Среда как капитальный актив

Среда работы ИИ-агентов обладает свойствами капитального актива: высокая стоимость первоначального построения, низкая стоимость последующего использования, накопительный эффект с течением времени. Каждая успешная агентная задача обогащает библиотеку паттернов, уточняет контекст-инжиниринг, верифицирует правила-ограничители. Через год активной эксплуатации среда знает специфику организации лучше, чем любой индивидуальный инструмент.

Этот накопительный характер объясняет, почему инвестиция в среду — стратегическая, а не тактическая. Организация, начавшая строить среду на год раньше конкурента, через три года будет иметь не просто более быструю разработку, а возможность агентно решать принципиально другой класс задач.

Экономика ИИ в разработке делится на две категории:

- ИИ в производственном процессе: как ИИ трансформирует разработку.
- PDLC для ИИ: как строить надёжные ИИ-приложения как продукт.

Разработка агентных систем как продукта относится ко второй категории и раскрывается далее в документе.

Архитектурная и инженерная реализация

Концепция среды состоит из двух составляющих:

- **Архитектурная теория**: как среда устроена, какие циклы её образуют и как она соотносится со всем PDLC.
- **Инженерная реализация**: как среда строится на платформе, какие компоненты обязательны и как обеспечивается надёжность.

В части 1 принцип среды выступает как стратегическое основание, куда направить инвестицию.

Принцип среды не противоречит принципу суверенитета моделей. Организация с сильной средой и доступом к нескольким моделям находится в наилучшей позиции: она может выбирать интеллектуальный актив под задачу, не перестраивая среду. **Именно порядок «сначала среда, затем модель» отличает стратегический подход от тактического.**

Важно отметить, что принцип «среда важнее модели» не отрицает ценность моделей. Мощная модель в сильной среде лучше, чем слабая в той же среде, при этом предельный прирост от улучшения среды кратно превышает предельный прирост от улучшения модели при прочих равных. Это означает, что организация, инвестирующая только в модели, получает 1–3 п. п. вместо 22 п. п. **Эффективная последовательность: сначала среда, затем модель.**

1.3 Рынок агентного ИИ: реальность против ажиотажа

Коммерческое освоение агентного ИИ на рынке стремительно растет: по данным аналитических агентств, 17% организаций уже развернули агентный ИИ в промышленной среде, 42% планируют развёртывание в течение 12 месяцев, ещё 22% — в течение двух лет. Совокупная доля организаций с активным агентным ИИ превысит 80% к концу 2027 года.

Однако значительная часть «развёртываний» — это пилоты, **реальный уровень зрелого агентного ИИ в производственной среде существенно ниже.** Согласно анализу отраслевых аналитиков, к 2026 году 49% организаций перешли от пилотов к промышленной эксплуатации, при этом 27% топ-менеджмента считают этот переход ключевым барьером развития агентного ИИ.

Это проблема в том числе **«имитации» агентов (agent-washing).** Существующие продукты роботизированной автоматизации процессов (Robotic Process Automation, RPA) и автоматизации на основе правил (Rule-Based Automation, RBA) переименовывают в «агенты», не меняя сути. По прогнозам мировых исследовательских компаний, к 2027 году 40% ИИ-проектов будет отменено или закрыто — из-за ранней концептуальной стадии, ажиотажа, репозиционирования и использования не по назначению.

Подход к оценке реального агентного ИИ

Есть пять критериев, отличающих подлинную агентность от «имитации» (систем, основанных на больших языковых моделях, но без глубокой агентной архитектуры).

Критерий	**Реальная агентность**	**Имитация агентности**
Автономное планирование (целеориентированность и рассуждения)	Сам декомпозирует цель на подзадачи и адаптирует план с учётом ограничений и оптимального сценария	Фиксированный сценарий или пошаговая инструкция без адаптации под контекст
Самостоятельное использование внешних инструментов	Агент сам определяет, какие инструменты и с какими параметрами нужно использовать – реальный вызов API, файловой системы, браузеров и др. с последующей интерпретацией результатов	Только структурированные потоки данных, инструкция по инструментам без обращения к ним

Критерий	Реальная агентность	Имитация агентности
Память между сессиями	Контекст сохраняется и накапливается для будущих задач, включая опыт, знания, предпочтения	Только кратковременный контекст или контекст обучающих данных
Динамическая адаптация к новым вводным	Агент реагирует на новые вводные и гибко пересматривает план при изменении условий	Нет реакции на изменение контекста и адаптации стратегии
Многошаговое контролируемое исполнение	Цепочка действий без подтверждения каждого шага, мониторинг выполнения и обработка ошибок	Один шаг — один запрос

Эти критерии работают и для оценки поставщиков ИИ-агентов, и как внутренний инструмент команды для развития агента-продукта. На этапе оценки применимости агента они помогают отобрать задачи, которым нужен агентный подход, а не алгоритмическая автоматизация или базовый ИИ-инструмент.

Три технологии под одним названием

Разработка данного подхода оценки подлинности ИИ-агентов стала необходимой, поскольку в современных реалиях под названием «агент» объединены три принципиально разные технологии:

- Алгоритмическая автоматизация / RPA — инструменту ИИ задается фиксированный сценарий, отсутствует адаптация под контекст и сохранение состояния между сессиями. Подходит для стабильных предсказуемых процессов.
- Чат-бот или инструмент с функциями ИИ — реактивно отвечает на запрос, не планирует и не действует автономно; подходит для запрос-ответных и Copilot-сценариев.
- Агент — сам декомпозирует задачу, адаптирует план, использует инструменты и сохраняет контекст между сессиями; подходит для задач с переменной структурой и многошаговым исполнением.

Применение агентного подхода к задаче, для решения которой достаточно автоматизированного процесса, приведет к избыточному и ресурсоемкому процессу (over-engineering) с высоким расходом токенов и операционными рисками. Применение алгоритмической автоматизации к задаче, требующей агентности, — слишком упрощенное решение (under-engineering), ведущее к сбоям и отказам. Исследование должно определить категорию задачи до входа в агентный цикл.

Ажиотаж и стратегические окна

Склонность применять агентов даже там, где их применение избыточно и задача может быть решена проще, может быть наглядно представлена через цикл развития агентного ИИ. Все отслеживаемые технологии находятся в фазах «инновационного триггера» или «пика завышенных ожиданий». Ни одна не достигла «впадины разочарования» — фазы, где ожидания систематически превышают реальный эффект. Инвестиции в ИИ-технологии, планируемые в 2026 году, будут выполнены и получат переоценку в 2027–2029 годах.

Часть технологий цикла со временем отпадёт из-за завышенных ожиданий. При этом стратегический прогресс в агентном ИИ требует своевременных инвестиций в отдельные технологии цикла.

Особого внимания требуют ИИ-агенты для кодинга (AI Coding Agents) и протокол Model Context Protocol (MCP) на «пике завышенных ожиданий». Инвестиции в эти технологии в 2026 году потенциально выводят на «плато продуктивности» к 2027–2029 годам.

Такого же пристального внимания требуют технологии в фазе «инновационного триггера» («плато продуктивности» прогнозируется к 2028–2031 годам):

- цикл разработки ИИ-агентов (Agent Development Life Cycle, ADLC);
- применение агентов в разработке ПО (AI Agents in Software Engineering);
- платформы разработки ИИ-агентов (AI Agent Development Platform);
- протоколы межагентного взаимодействия (AI Agent Communication Protocols);
- агенты-супервизоры для контроля действий (Guardian Agents).

Анализ цикла обосновывает двухуровневую стратегию инвестиций по зрелости инструментов. Технологии «инновационного триггера» проходят апробацию через пилоты; технологии «пика завышенных ожиданий» — технологический базис зрелых агентных решений. Они требуют быстрых инвестиций для попадания в стратегические окна и выхода на «плато продуктивности».

1.4 ИИ как усилитель команды

ИИ не просто ускоряет существующие задачи — он открывает класс работ, на которые раньше не хватало времени команды:

По данным Anthropic, инженеры привлекают ИИ примерно к 60% своих задач — но полностью делегируют агенту лишь 0–20% из них. Большинство работают в режиме тесного соавторства, а не делегирования. При этом около 27% задач, выполненных с участием ИИ, — это работа, которую вообще бы не стали делать без агентов: исследование альтернатив, документация, тесты для legacy-кода.

Человек в контуре решений (human-in-the-loop) — не параметр конфигурации, а архитектурное решение: кто, когда и по каким критериям валидирует результат агента для конкретной задачи.

McKinsey (2025) фиксирует это как ключевой признак зрелых ИИ-организаций: 65% из них имеют формализованные процессы, определяющие, как и когда результат модели требует валидации человеком — против 23% у остальных. В нашей методологии карта решений (human-in-the-loop decision map) — обязательный раздел SDD.

ИИ в зрелой команде расширяет возможности человека, а не замещает его. Отдача возникает там, где место человека в контуре решений закреплено как процесс, а не как ручная настройка.

Метрики зрелости ИИ-разработки (DORA AI Capabilities)

Зрелость ИИ-разработки измеряется семью индикаторами DORA AI Capabilities:

- **Continuous Delivery** — плотность внедрений для ИИ-генерируемого кода vs. написанного вручную (целевой ориентир: паритет к концу Горизонта 2)
- **Psychological safety** — доля инженеров, которые комфортно сообщают о проблемах с ИИ-артефактами (прокси-метрика: доля ИИ в повестке командного ретро)
- **Technical quality** — regression rate ИИ-кода относительно baseline (цель: <1,5×)
- **Experimentation culture** — пропускная способность opportunistic backlog: задач, которые агенты выполняют в свободные циклы и которые раньше не получали приоритета
- **Code review** — agreement rate: доля случаев, когда оценка review-агента совпала с оценкой человека-ревьюера (цель: >85%)
- **MTTR** — время agent-driven восстановления плюс время эскалации на человека
- **Deployment frequency** — разделяется на «типовой» и «генеративный ИИ»

Шкала автономии агента: пять уровней взаимодействия (0–4)

Уровень	Режим взаимодействия
0	Агенту нужно явное подтверждение на каждое действие
1	Агент группирует действия, проверяет несколько шагов за раз
2	Агент выполняет задачу, периодически запрашивая подтверждение
3	Агент завершает задачу, отчитываясь только об ошибках
4	Полностью автономная работа без явного надзора

По данным Anthropic (2026), типичная сессия Claude Code соответствует уровню 2 — агент выполняет задачи с периодическими запросами подтверждения. Самые длительные сессии (хвост распределения, от 45 минут) достигают уровня 3. При этом опытные пользователи (от 750 сессий) прерывают агента чаще, чем новички — не потому, что меньше доверяют, а потому что перешли от пошагового подтверждения к мониторингу: они дают агенту больше свободы, но точнее замечают отклонения.

1.5 Подходы к интеграции ИИ в разработку

Три подхода к интеграции ИИ различаются не инструментами, а тем, кто является субъектом PDLC — человек, контролируемый ИИ, или автономный агент:

Подход	Описание	Характеристика	Прирост
ИИ как дополнение	ИИ встраивается в существующий PDLC как ассистент	Human-in-the-loop, роли неизменны	11–25%

Подход	Описание	Характеристика	Прирост
ИИ-нативный подход (AI-DLC)	Артефакты и ритуалы переосмыслены, основаны на SDD	Specification-Driven	25–50%
Агентный режим	Агенты выполняют автономные задачи часами/днями	Human-on-the-loop	30–50%+

Подходы не исключают друг друга. Они вызревают по-разному:

- «ИИ как дополнение» даёт быстрый прирост в первые 90 дней, но к концу года команда упирается в потолок, поскольку сам процесс не изменился;
- ИИ-нативный подход (AI-DLC) требует 6–12 месяцев на одновременное изменение процесса (ритуалы исследования, SDD) и команды (работа со спецификациями, грамотность в валидации ИИ-результатов);
- Агентная разработка — самостоятельный режим для классов задач, прошедших оценку применимости агента, а не следующая ступень.

Зрелая команда применяет три подхода параллельно для разных задач, а не проходит их последовательно как ступени.

Выбор подхода — решение петли исследования (Discovery Loop) для каждой задачи. Для legacy-системы достаточно ИИ как дополнения, для нового продукта — ИИ-нативного подхода, для исследовательских задач — агентного режима. Выбор должен быть явным, а не побочным продуктом настроек инструмента. Он формализуется через оценку применимости агента.

1.6 Стратегические прогнозы

Сводка стратегических прогнозов формирует единую картину трансформации до 2030 года. Приведённые прогнозы, по данным мировых исследовательских компаний, — это сценарии, а не гарантии.

Прогноз:

- 90% инженеров (в enterprise-среде) используют ИИ-ассистентов (против менее 14% в 2024)
- 70%+ корпоративных инженеров используют агентные инструменты
- Асинхронные SWE-агенты повышают продуктивность на 30–50%
- ИИ-агенты устраняют 70% уязвимостей кода
- Организации с инфраструктурой среды работы ИИ-агентов сокращают дефекты после развёртывания на 60%
- 40% платформенных команд используют ИИ в каждой фазе PDLC
- Организации без SDD: −25% скорости разработки
- Роль product engineer заменяет 50%+ позиций разработчиков
- 40% портфелей приложений — кастомная разработка через ИИ (против 2%)

Прогноз сокращения дефектов после развёртывания на 60% к 2028 году адресован тем, кто инвестирует в полноценную инфраструктуру среды сегодня. Это следствие системной

работы со средой: работы с контекстом, правил-ограничителей и встроенных контрольных точек проверки.

Прогноз потери 25% скорости к 2027 году у организаций без разработки через спецификацию (SDD) — про отставание от конкурентов, а не про абсолютную деградацию. Преимущество SDD реализуется только при наличии исследования: без него SDD превращается в «длинный тикет» — формальную обёртку без переосмысления задачи.

Прогноз 90% охвата инженеров ИИ-ассистентами к 2028 году означает гомогенизацию Уровня 1: ИИ-помощник станет такой же нормой, как редактор кода или система контроля версий. Дифференциация сместится на Уровни 2 и 3 — качество среды вокруг модели. Именно поэтому архитектурный выбор в пользу инфраструктуры среды, сделанный в 2026 году, определяет конкурентную позицию в 2028–2030 годах.

1.7 Адаптация или перепроектирование как драйвер ИИ-эффекта

Предельно достижимый эффект от ИИ определяется решением на входе: воспроизвести прежний процесс или перепроектировать его под новые возможности. Это решение принимается на стадии исследования до того, как написана спецификация.

Исследование (Discovery) — этап петли намерения, на котором задача проходит путь от исходной причины для изменения (заказчик, инцидент, сигнал в метриках, изменение регулятора или плановый бэклог) до сформулированного намерения, готового к разработке через спецификацию (Specification-Driven Development, SDD).

Цель исследования — переосмыслить, как задача должна выполняться в ИИ-нативной среде, до написания спецификации. Product Engineer ведёт задачу, разработчик отвечает за техническую сторону, заказчик подключается на ключевых решениях.

Этап даёт результаты:

- постановку задачи в виде текста;
- решение о подходе — адаптация или перепроектирование;
- оценку, нужен ли задаче ИИ-агент и в каком объёме. Это определяет глубину дальнейшего исследования и детализацию спецификации;
- выбранный из библиотеки сценарий вовлечения агента, чтобы не проектировать его с нуля.

Задача проходит цикл намерения по одной из двух стратегий: прежний процесс с новыми инструментами или переосмысленный процесс, использующий эти инструменты. Разница между стратегиями напрямую определяет эффект от внедрения ИИ. Мы называем её пробелом исследования (Discovery Gap).

Принципиальные различия сценариев

Характер и масштаб изменений в процессах определяют достижимый эффект:

- Адаптация — ИИ-инструменты автоматизируют существующий рабочий процесс. Процесс остаётся прежним, агент берёт на себя один или несколько шагов. Прирост линейный, 11–25%, без существенных организационных изменений.
- Перепроектирование — рабочий процесс перестраивается вокруг возможностей агентных инструментов. Пересматриваются шаги, роли человека и агента. Прирост нелинейный, от 25–50% и выше при преодолении порога, требует организационных изменений.

Мы считаем, что решение между адаптацией и перепроектированием должно быть осознанным и задокументированным. Без этого наиболее вероятный выбор — адаптация как путь наименьшего сопротивления. Главное последствие этого выбора — с помощью новой технологии воспроизводится старый процесс.

Примеры адаптации и перепроектирования

Сценарий адаптации (домен страховых выплат): добавление ИИ-агента для первичной классификации обращений при сохранении существующего процесса рассмотрения — агент ускоряет один шаг, остальное остаётся прежним. Прирост: 15–20% пропускной способности, минимальные организационные изменения.

Сценарий перепроектирования (тот же домен): полное перепроектирование обработки обращений о выплатах вокруг возможностей agentic-рабочего процесса — агент классифицирует, оценивает, готовит ответ, собирает дополнительные данные через MCP-интеграции с внешними системами; человек вступает только в спорных и высокорисковых случаях. Прирост: 60–80% пропускной способности, существенные организационные изменения.

Точка сдвига индустрии

Точка сдвига индустрии в 2026 году — переход от спецификации функциональности (что агент должен делать) к спецификации результата (что считать результатом и как понять, что он достигнут). Это первый структурный сдвиг, который формализует PDLC; следующие пока вне горизонта концепции.

Ведущие компании формируют агентные команды без традиционных разработчиков — только Product Engineers и агенты. В этой модели измеримый результат — единственный способ управлять работой агентов: без чёткой метрики агент не знает, когда считать задачу выполненной.

Ориентированность на измеримый результат перестаёт быть рекомендацией и становится обязательным условием зрелости. Операционно меняется суть спецификаций (SDD) и требований: команда переключает внимание с разработки на то, что происходит до неё и после — на решение проблемы.

Три сдвига для команды:

1. Спецификация (SDD) перестаёт быть списком требований к поведению системы: в неё входит обязательный блок проверки гипотезы — какая метрика изменится, как и за какой срок.
2. Отношение к процессу смещается от «мы пишем код» к «мы создаём инструмент проверки гипотезы».
3. Команда формулирует проблему, а не просто берёт тикеты в работу. Без доступа к аналитике и без навыка постановки гипотез команда скатывается в «конвейер фич» (feature-factory) как более простой и привычный режим.

1.8 Рамка принятия решения на стадии Discovery

Команда принимает решение на стадии этапа исследования. Отвечаем на вопросы:

1. Текущий процесс создан до ИИ или с учётом ИИ?
2. Каков потенциал нелинейного прироста при переосмыслении процесса?
3. Какие барьеры для изменений?
4. Каков горизонт возврата инвестиций (адаптация — 3–6 месяцев; перепроектирование — 12–24 месяца)?

Если ответы указывают на адаптацию — фиксируем это в SDD; если перепроектирование — нужен отдельный воркшоп с заказчиками и владельцами решения и более детальное PR/FAQ.

Типичный антипаттерн — «промежуточный вариант». Команда выбирает псевдо-перепроектирование: частично переосмысливает рабочий процесс, но без полной поддержки заказчиков и владельцев решения или с компромиссами в ключевых точках. Такой выбор не даёт ни линейного прироста адаптации, ни нелинейного прироста перепроектирования. Стоимость трансформации оплачена, а эффект слабый.

Защита: явная ответственность за решение (один владелец решения, обычно product engineer); явная фиксация в SDD; явные критерии успеха для каждого сценария.

Операционные детали этапа Discovery

Субцикл исследования имеет четыре ритма: ad-hoc приём (идея от заказчика, инцидента, сигналов метрик), плановый приём (квартальная дорожная карта), реактивный приём (регуляторный или конкурентный триггер) и груминг бэклога. Временная шкала: ad-hoc — часы, плановый — дни, реактивный — в зависимости от срочности.

Каждый Discovery-сеанс включает несколько ключевых артефактов. PR/FAQ — метод Amazon для ИИ-контекста: написать пресс-релиз о желаемом результате, затем список вопросов скептиков. Outcome Hypothesis — конкретное измеримое утверждение: «мы ожидаем, что изменение X приведёт к результату Y, измеримому через метрику Z, в течение периода W». Рамка «что бы мы сделали, если бы результат был опровергнут?» — прямой тест на фальсифицируемость гипотезы.

Матрица применимости агента включает пять критериев:

- Критерий 1: степень неопределённости — задача с предсказуемым результатом (deterministic / AI-augmented); задача со средней неопределённостью (AI-Native); задача с высокой неопределённостью (Agentic).
- Критерий 2: требуемая адаптивность — фиксированные сценарии → deterministic; средняя гибкость → AI-augmented; высокая адаптивность → agentic.
- Критерий 3: цена ошибки — низкая (любой подход); средняя → нужны проверки оценок (AI-Native); высокая → человек в контуре решений (осторожный agentic с human-in-the-loop).
- Критерий 4: обратимость действий — полностью обратимые → высокая автономия; частично обратимые → средняя; необратимые → низкая автономия + human-in-the-loop.
- Критерий 5: наличие подходящей записи в библиотеке шаблонов — да, паттерн хорошо подходит → AI-Native (или Agentic, если паттерн мультиагентный); нет явного паттерна → вероятно, нужно больше исследований.

Антипаттерны и операционные проверки Discovery

Антипаттерны этапа Исследования в зрелой команде:

- **Антипаттерн 1**: на все вопросы дают шаблонные ответы, не содержательные. Признак: 100% SDD заявляют результат «улучшение на 25%»; реальные результаты широко расходятся. Способ митигации: взаимная проверка артефактов исследования; культура честности.
- **Антипаттерн 2**: мелкие баг-фиксы и тривиальные фичи проходят полное исследование, тратя время впустую. Способ митигации: оценка применимости агента помогает определить нужную глубину исследования; простые задачи могут пропускать существенную часть.
- **Антипаттерн 3**: почти всегда выбирается адаптация без явного рассмотрения перепроектирования. Способ митигации: явный чекбокс «рассмотрено ли перепроектирование» в шаблоне исследования.

Операционные проверки качества исследования (самопроверка команды):

1. Если написание SDD началось без полного PR/FAQ — этап исследования пропущен.
2. Если Гипотеза результата (Outcome Hypothesis) звучит как «улучшить производительность» — она неизмерима.
3. Если решение адаптация/перепроектирование не зафиксировано в документе — по умолчанию выбрана адаптация (вероятно, неверно).
4. Если класс применимости агента не задан явно — агент будет применён неправильно.
5. Если ссылки на библиотеку шаблонов нет — возможность переиспользовать паттерн упущена.

Для каждой проверки есть автоматизированный аналог в IDP CI/CD — контроли CI/CD, специфичные для ИИ. С опытом время цикла исследования снижается: первый раз — 5–7 рабочих дней; через 6 месяцев — 2–3 дня; через 12 месяцев — 1–2 дня. Это накопительный эффект: рост библиотеки шаблонов ускоряет этап исследования; опыт команды ускоряет формулирование гипотезы результата (Outcome Hypothesis); чемпионы эффективнее ведут фасилитацию.

Типичное сопротивление стейкхолдеров

Сопротивление ритуалам исследования принимает три типичные формы. На каждую есть ответ:

- «Нам нужно выпускать быстрее, а не открывать больше». Ответ: исследование экономит время на последующих этапах: стоимость переделок превышает стоимость исследования в 5–10 раз.
- «Гипотеза результата (Outcome Hypothesis) — очередное обязательное поле». Ответ: без измеримого результата команда гонится за метриками выпуска, не ценностью.
- «Мы пишем документы с продуктовыми требованиями (PRD, Product Requirements Document) двадцать лет — Discovery это просто новое название». Ответ: Discovery — не переименование, а другой документ.

Классический PRD описывает бизнес-цели и пользовательские сценарии. Но в нём нет трёх разделов, обязательных для ИИ-фичи:

1. Измеримая гипотеза результата (Outcome Hypothesis) — проверяемое утверждение вида «если внедрим агента X, метрика Y вырастет с 12% до 30% за три месяца на корпусе из 500 кейсов», а не лозунг «увеличить вовлечённость».
2. Явный архитектурный выбор между адаптацией существующего процесса (кнопка «Суммаризовать» в редакторе) и его перепроектированием под возможности агента (чат-интерфейс вместо редактора). Стоимость, риски и потолок выгоды различаются в разы.
3. Класс применимости агента по шкале R0–R5 — от подсказки до автономной работы в очерченном домене. Только подсказывает, предлагает варианты, действует в песочнице с подтверждением человека, действует в проде с обратимыми операциями, выполняет необратимые действия с аудитом, работает автономно. От класса зависят кодифицированные ограничения, политика подтверждений, телеметрия и требования к агентам-стражам (Guardian Agents).

Классический PRD проектировался под детерминированный софт — этих трёх полей в нём нет и быть не может.

При сжатых сроках ритуалы исследования нельзя пропускать, но можно сжать: PR/FAQ — 30 минут вместо 90, гипотеза результата — 15 минут вместо 60, решение об адаптации или перепроектировании — по готовым паттернам. Итого: один час вместо двух-трёх. Полный пропуск порождает антипаттерн Specification without Discovery.

Масштабирование Discovery

При переходе от 5 команд к 50 растут накладные расходы на координацию; появляется доменная специализация библиотек шаблонов; нужна ясность ответственности в межкомандных инициативах. Решения: роли Discovery facilitator для межкомандной координации, доменные пространства имён в библиотеке шаблонов, явная RACI. Это организационная адаптация, не методологическая.

Discovery facilitator — не отдельная позиция, а компетенция product engineer: умение задавать открытые вопросы, определять, где задаче нужна глубокая проработка, а где можно сразу переходить к SDD, и соединять продуктовый и инженерный языки.

1.9 Ориентированность на измеримый результат

Корпоративные механизмы декомпозиции нужны, чтобы разложить желаемый результат на атомарные шаги. Трудность принципа в том, чтобы удерживать цель через длинную цепочку шагов и множество мелких подзадач; на длинной цепочке фокус смещается с цели на исполнение.

В работе над фичей это ограничение оборачивается подменой цели: критерием успеха становится факт поставки фичи, а не бизнес-результат, ради которого она задумана. Сложность исполнения захватывает внимание здесь и сейчас, и амбициозность исходной цели удержать трудно.

ИИ снижает стоимость и время написания кода в разы — и тем самым смещает ограничивающий фактор с производства кода на его осмысленность. Когда delivery перестаёт быть узким местом, центр тяжести команды закономерно перемещается из зоны производства ценности (delivery) в зону её формирования (outcome) и проверки гипотезы.

Такой переход задаёт три акцента:

- Функциональность перестаёт быть целью разработки и становится средством достижения измеримого бизнес-результата.
- Определение «Готово» смещается от технической доставки («фича задеплоена») к подтверждению ценности («гипотеза сработала»).
- Команда отвечает не за объём кода, а за изменение бизнес-метрики.

Различие двух подходов на примере. При подходе «функциональность в первую очередь» (Feature-first) заказчик говорит «Сделай кнопку», команда думает, как сделать кнопку, результат — кнопка есть. При подходе «результат в первую очередь» (Outcome-first) заказчик говорит «Надо поднять конверсию на 5%», команда вместе с агентом ищет способ, а результат — выросшая конверсия. Если кнопка не помогла, это провал даже при идеальном коде.

При подходе «функциональность в первую очередь» (Feature-first) запрос формулируется как поставка артефакта («добавить кнопку»), и критерий успеха — наличие этого артефакта. При подходе «результат в первую очередь» (Outcome-first) запрос формулируется как измеримый результат («поднять конверсию на 5%»), команда совместно с агентом ищет способ его достичь, а критерий успеха — фактический рост метрики. В outcome-модели технически безупречная реализация, не давшая целевого сдвига метрики, считается неудачей.

Образ желаемого результата

В AI-native разработке написание кода — это commodity (расходный материал). Истинная ценность создаётся на этапе формулирования проблемы и верификации конечного результата. Команда перестаёт быть «фабрикой функций» и становится «инкубатором решений и идей», где каждый артефакт (даже SDD) привязан к деньгам или эффективности, а не к архитектуре.

1.10 Императив контура управления (Governance Imperative)

Governance в традиционном IT — это структурированный механизм управления риском, соответствием и капитальным распределением. ITIL и ITSM (PeopleCert) описывают операционные процессы governance; COBIT (ISACA) формализует control objectives; TOGAF (The Open Group) задаёт архитектурный governance. Общая черта этих фреймворков — governance работает как post-hoc стадия: артефакт создан, governance его проверяет, при необходимости отправляет на доработку.

Для AI-Native PDLC эта модель работает плохо по структурным причинам. Скорость и непрозрачность AI-Native разработки делают post-hoc, stage-gated, human-only валидацию несоразмерными с темпом генерации risk-significant артефактов. За время, пока традиционный governance-процесс рассматривает один пулл-реквест (PR), AI-Native-команда могла открыть десять новых.

Цикл отказов как системная угроза

Мы фиксируем самоусиливающийся цикл трёх отказов — он объясняет, почему AI-Native-разработка без встроенного контура управления деградирует не линейно, а по нарастающей:

Цикл отказов (петля деградации):

1. Накопление дефектов (defect accumulation) — генерация кода опережает способность проверить его качество; дефекты накапливаются в production быстрее, чем закрываются.
2. Узкое горлышко проверки (verification bottleneck) — ручные ревью и gate-процессы не справляются с объёмом; команда начинает срезать углы, пропускать проверки, выпускать под давлением.
3. Эрозия доверия (trust erosion) — инциденты учащаются, стейкхолдеры требуют ещё больше ручных проверок, скорость падает, недоверие к ИИ-артефактам растёт.
4. Возврат к накоплению дефектов: команда работает медленнее, но дефектов не меньше.

Цикл отказов объясняет, почему точечные улучшения модели не дают эффекта без среды: замена модели на более «умную» не разрывает петлю деградации, если среда не обеспечивает контекст, правила-ограничители и гейты проверки. Единственный способ разорвать петлю — встроить governance в ту же среду и в тот же ритм, что и сама разработка.

Недостатки традиционного governance

Мы выделяем недостатки традиционного governance применительно к AI-Native среде:

1. Post-hoc природа: к моменту проверки артефакт уже создан и обычно частично применён; стоимость исправления на этом этапе в 5–10 раз выше, чем при встроенной валидации.
2. Stage-gated архитектура: governance срабатывает только на заранее определённых точках; между ними агентная разработка происходит без контроля, накапливая риски.
3. Human-only исполнение: при кратном росте объёма артефактов ручная проверка становится узким горлышком; команда вынуждена выбирать между скоростью и соответствием.

Контур управления (Governance Mesh) — наш ответ на эти три структурных недостатка. Это не слой поверх процесса, а встроенная способность среды работы ИИ-агентов: проверки срабатывают в потоке генерации, политики выражены кодом, надзор частично делегирован агентам. Архитектура и полное определение Governance Mesh раскрыты в Части 2.

Governance в российском корпоративном контексте

Российский корпоративный контекст добавляет к стандартным требованиям governance специфические слои: соблюдение требований 152-ФЗ и отраслевых регуляторов при работе с персональными данными в ИИ-системах; аудируемость ИИ-решений для внешних проверок; прослеживаемость от бизнес-задачи до кода, выпущенного агентом. Контур управления

(Governance Mesh) проектируется с учётом этих требований как база, а не как опциональное расширение.

Governance как непрерывный процесс

Организации, внедрившие AI-Native-инструменты без перестройки governance, в первые 6–12 месяцев сталкиваются с ростом инцидентов в production и снижением доверия к ИИ-артефактам. Причина этому – ИИ успевает создать не только полезные изменения, время также уходит на интеграцию изменения, устранение сопутствующих ошибок.

Контур управления отличается от традиционного governance не только архитектурой, но и нарративом. Традиционный governance создаёт иллюзию безопасности через документы, комитеты и гейты; встроенный governance создаёт реальные гарантии через код, автоматические проверки и невозможность технически обойти политику. Разница между «политика опубликована» и «политика исполняется» — фундаментальная в AI-Native контексте. Агент не читает документацию; он действует в рамках среды, которая определяет, что возможно технически.

Контур управления работает в той же среде и в том же ритме, что и разработка: проверки встроены в платформу, политики выражены кодом, надзор делегирован агентам.

Мы установили экономическое основание трансформации и стратегический выбор между инвестицией в среду и инвестицией в модели. Нелинейный ИИ-эффект возникает не из инструментов, а из переосмысленного процесса и правильно выстроенной среды. Экономический императив — не просто аргумент для бюджетного запроса; это навигационный компас для каждого архитектурного решения, которое команда примет в Частях 2–4: какой уровень автономии, какая глубина среды, какое качество платформы оправданы той ценностью, которую мы видим в юнит-экономике и ROI-таблице.

Теперь — архитектура: как устроен AI-Disrupt PDLC изнутри, какие петли его образуют, как среда соотносится с каждой фазой жизненного цикла и почему именно среда, а не модель, определяет предельно достижимый результат.

Часть 2 разворачивает архитектурное ядро концепции — от двухпетлевой модели до Governance Mesh — и отвечает на вопрос, как это работает как единая система. Читатель, прошедший Часть 1, владеет экономическим языком трансформации; читатель Части 2 получит её архитектурный язык — понятия, схемы и механизмы, из которых собран весь цикл.

Часть 2. Архитектурное ядро PDLC

2.1 Двухпетлевая модель с ИИ-валидацией

Классический PDLC сохраняет свои стадии — требования, проектирование, реализация, тестирование, выкатка, эксплуатация, — но фундаментально меняется исполнитель каждой стадии, скорость прохождения и требуемые компетенции команды. Переход к AI-Disrupt PDLC — не ускорение существующего процесса, а его архитектурная реструктуризация: «человеческая» и «агентная» работа разделены на отдельные потоки с чёткими точками передачи контроля**. Мы называем это двухпетлевой моделью с циклами намерения и реализации.**

Цикличность намерения и исполнения — продолжение привычных способов декомпозиции задач: сложное действие разбивается на простые шаги и выполняется в несколько этапов. Шаги могут повториться, если задача сложна или выявлены новые вводные. Так возникает образ микроциклов, обусловленных повторением шага или целой цепочки шагов.

Цикл намерения содержит этап исследования: постановка задачи → PR/FAQ → гипотеза о результате → решение о масштабе редизайна (адаптация или перепроектирование) → анализ необходимости в агенте → поиск типовых решений.

Цикл реализации в базе всегда содержит два этапа: внутренний цикл — итерирование правок (в том числе тест/сборка/исправление) в рамках одной агентной сессии; внешний цикл — переход между сессиями через контрольные точки. На каждом этапе возникает точка принятия решения о качестве и корректности результата — продолжать ли, сохранять ли автономность, нужны ли дополнения в постановку и спецификацию.

Достижимая скорость изменений в таком подходе кратно превышает способность человека принимать результат прежним способом — построчным ревью, согласованием на отдельной стадии, ручной интеграцией. Если оставить приёмку и валидацию работать как раньше, разрыв в темпах превращается в новое узкое место: код готов быстро, но не может быть принят, проверен и выпущен с той же скоростью. Это нарушает идею бесшовной разработки без каких-либо простоев и сопротивлений (Zero Friction) как целевое свойство модели — возникает новое ключевое ограничение.

Всё оказывается связанным: намерение, исполнение и валидация исполнения. Zero Friction как желаемое состояние зависит от новых подходов к валидации в той же мере, что и от изменений в разработке. Идея сквозной валидации — способ не допустить такого ограничения.

Мы считаем, что валидация должна быть встроена в петли намерения и исполнения, а не превращаться в самостоятельный слой или третью петлю. Три петли (намерение / реализация / валидация) решают проблему лишь на первый взгляд. Третий контур, кроме новых шагов, также потребует новой границы между петлями, специализации и быстро дойдёт до привычной модели функциональной изоляции: отдельная команда валидации со своим бюджетом и ритмами, дополнительный цикл координации. Это ломает Zero Friction — просто другим способом.

Наш архитектурный выбор — методика валидации ИИ-результатов, не третья петля. Это сквозной слой, встроенный в обе существующие петли. В петле намерения фиксируются критерии приёмки. В петле реализации эти критерии исполняются как часть микроциклов: агент-ревьюер работает после каждой итерации, проверки безопасности и политик срабатывают перед контрольными точками, релизный барьер срабатывает перед мерджем. После выкатки телеметрия из эксплуатации возвращается в спецификацию.

Предлагаемый подход к валидации встроен в петли намерения и реализации как сквозная способность. Его метрики и доказательства становятся встроенными характеристиками каждой задачи, а не отдельными артефактами за пределами цикла. Это устраняет барьер между разработкой и приёмкой, сохраняя принцип Zero Friction как операционное свойство.

Слой	Характер	Участники	Длительность такта	Артефакты
Петля намерения	Человеко-ориентированная, декларативная	Команда и ИИ-ассистент	Дни / недели	SDD, ADR, ссылка на библиотеку шаблонов, карта вовлечения человека (human-in-the-loop Decision Map)
Субцикл исследования (Discovery sub-loop, внутри петли намерения)	Продуктовая, диагностическая	Product engineers, оркестраторы, стейкхолдеры	Часы / дни	Исходная постановка задачи, аналитика, архитектура
Петля реализации	Агент-ориентированная, императивная	Оркестратор и рабочие агенты под надзором	Минуты / часы / дни	Код, тесты, артефакты выкатки, проверочный пакет (Evidence Bundle), передача сессии
Контур валидации	Сквозной слой приёмки и валидации	Агент-ревьюер, агент-страж (Guardian Agent), проверки безопасности, телеметрия	Непрерывно	Результаты оценок (evals), проверки на регрессии, обратная связь телеметрии в спецификацию

Мы стремимся к тому, чтобы добавочное время валидации составляло 10–20 минут на задачу вместо 1–2 часов ручного ревью. Сокращение достигается за счёт того, что агент-ревьюер закрывает 80%+ формальных проверок и оставляет человеку архитектурные и спорные ревью. Метрика отношения темпа реализации к темпу валидации (I/V Tempo Ratio) показывает, насколько скорость реализации опережает пропускную способность валидации; целевым считается диапазон от 1,5× до 2,5×.

Эффект сквозной валидации разрушают три ловушки мышления:

1. Исследование без валидации. Команда хорошо формулирует замысел, но плохо проверяет результат: целевые результаты не подтверждаются, контур обучения не получает информацию. Признак: много формализованных точек человеческого контроля при малом числе подтверждённых результатов после вывода изменений в промышленную среду.
2. Валидация без исследования. Команда системно проверяет работу, но проверяет не то: спецификации есть, а бизнес-результаты не достигаются. Признак: слабая связка между продуктовой гипотезой, фактическим эффектом и перераспределением высвободившегося времени на более ценные задачи.
3. Реализация без исследования и валидации. Скорость поставки высокая, объём изменений растёт, но качество деградирует, а целевые результаты не проверяются. Артефактов становится больше, а качество и бизнес-эффект выпадают из поля зрения. Решение: еженедельный анализ всех трёх контуров, а не только скорости реализации.

Двухпетлевая модель работает только тогда, когда контур управления и контур валидации проходят через каждую петлю. Иначе готовый код упирается в стену приёмки, а принцип отсутствия трения остаётся декларацией, а не операционным свойством платформы.

2.2 Петля намерения

Петля намерения отвечает на вопрос, что и зачем строить. Её результат определяется не подробностью технического задания, а качеством предшествующего исследования. Пропуск исследования превращает спецификацию в длинный документ без стратегического смысла.

Петля намерения начинается с этапа исследования (Discovery): постановка задачи → пресс-релиз и часто задаваемые вопросы (PR/FAQ) → гипотеза результата → решение о масштабе редизайна (доработка или реинжиниринг) → анализ необходимости в агенте → поиск типовых решений.

Discovery Loop внутри цикла намерения

Концепция AI-Disrupt PDLC разделяет цикл намерения и цикл реализации. Привычным началом этапа исследования считался сам факт формулировки намерения — оформленной задачи, объёма документации, пользовательской истории (user story) или критериев приёмки. Упущения постановки, критериев приёмки и контекста устранялись людьми на поздних этапах реализации или не устранялись вовсе. Это перестаёт быть нормой.

Когда нет разницы в скорости между намерением и реализацией, неточность постановки одинаково дорога на любом этапе. Агент с нечёткой задачей либо галлюцинирует, либо запрашивает уточнения в неподходящий момент цикла. Субцикл исследования (Discovery sub-loop) — обязательный этап, а не опциональная «лучшая практика».

Этапы субцикла исследования:

- Формулировка возможности — зачем нужна задача, что меняется и кто заинтересован в результате.
- Пресс-релиз и часто задаваемые вопросы (PR/FAQ) — описание желаемого будущего состояния и логики результата по подходу Amazon: сначала целевой пользовательский или бизнес-результат, затем технология.
- Гипотеза результата — какая бизнес- или продуктовая метрика должна измениться, какой опережающий индикатор это покажет, за какой период эффект должен подтвердиться и при каком условии решение нужно пересмотреть.
- Выбор доработки или реинжиниринга — автоматизируем существующий процесс или меняем его с учётом возможностей агентов.
- Оценка пригодности для агентного исполнения — какой уровень автоматизации уместен: детерминированная логика, ИИ-помощник, ИИ-нативное исполнение по SDD или полноценное агентное исполнение. Для оценки используется фреймворк из пяти критериев.
- Проверка библиотеки шаблонов — какие шаблоны спецификаций и переиспользуемые рабочие процессы уже есть для похожих задач.
- Матрица валидации человеком — кто, когда и по каким критериям проверяет результат на ключевых шагах; финальная версия фиксируется в SDD.
- Совместная детализация и переход к SDD — после исследования задача попадает на совместную детализацию и оформляется в полноценный SDD.

Артефакты субцикла исследования:

Артефакт	**Назначение**	**Источник идеи**
Формулировка возможности	Фиксирует, зачем существует задача, что меняется и кто заинтересован в результате	—
Внутренний PR/FAQ	Фиксирует желаемое будущее состояние и логику результата через описание пользовательского или бизнес-эффекта до начала реализации	Amazon Working Backwards
Гипотеза результата	Фиксирует бизнес- или продуктовую метрику, опережающий индикатор, период подтверждения эффекта и условие пересмотра решения	McKinsey Agentic Org
Решение о доработке или реинжиниринге	Фиксирует, автоматизируется ли существующий процесс или меняется сам подход к решению задачи с учётом возможностей агентов	BCG Tension 4
Оценка пригодности для агентного исполнения	Фиксирует уместный уровень автоматизации: детерминированная логика, ИИ-помощник, ИИ-нативное исполнение по SDD или полноценное агентное исполнение	BCG, McKinsey 2025
Проверка библиотеки шаблонов	Фиксирует, какие шаблоны спецификаций и переиспользуемые рабочие процессы уже есть для похожих задач	OpenAI 2025 Custom GPT scale

Артефакт	Назначение	Источник идеи
Черновик матрицы контроля человеком	Фиксирует, кто, когда и по каким критериям проверяет результат на ключевых шагах	McKinsey 2025

SDD без исследования (Discovery) — это просто подробное техническое задание без стратегического смысла. Без исследования спецификация фиксирует, «что построить», но не «зачем» и «в какой форме». Решение о доработке или реинжиниринге не принято — агент автоматизирует старый процесс на новой технологии. Класс применимости агента не задан — режим исполнения выбирается случайно.

Внутренний контур исследования не добавляет нового цикла отчётности и не превращается в отдельный офис исследования. Это набор обязательных артефактов, которые продуктовая команда готовит перед SDD. В зрелой команде исследование занимает дни и проходит через сессию PR/FAQ и одну-две итерации совместной детализации. Термин PR/FAQ сохранён как узнаваемый артефакт Working Backwards: его смысл — сначала описать будущий пользовательский опыт и только затем уточнять, что строить.

Трёхфазная модель трансформации

Последовательный трёхфазный подход к внедрению агентного ИИ в PDLC основан на теории ограничений Голдратта:

Принцип: работать с системой последовательно, устраняя одно ограничение за раз. Попытка сразу перейти к Фазе 3 без прохождения Фаз 1 и 2 — главная причина неудач ИИ-трансформации.

Фаза	Цель	Длительность	Метрика успеха
Фаза 1: устранение ограничений (Resolve Constraints)	Устранить узкие места, получить базовую линию DORA, пилот на 2–3 командах	3–6 мес	Lead Time; CFR не ухудшается; запущены ритуалы исследования
Фаза 2: рост пропускной способности (Accelerate Throughput)	Внедрение SDD, ИИ-контроль в CI/CD, обучение всей команды, подготовка типовых шаблонов	6–12 мес	Deployment Frequency; количество задач через ИИ; Spec Reuse Rate
Фаза 3: автоматизация процессов	Постоянные процессы, оркестрация многоагентных систем, контур валидации как платформа	12+ мес	Доля задач в многоагентном режиме; медианный горизонт задачи (Median Task Horizon); доля подтверждённых результатов (Outcome Validation Rate)

Практики исследования и механизмы агентной валидации мы включаем в контур уже на первой фазе, а не откладываем на вторую. PR/FAQ, явная гипотеза бизнес-результата, конвейер непрерывной сборки и поставки и линия автоматического ревью кода работают с

фазы 1. Показатели из таблицы — двукратный рост частоты развёртываний и доля повторного использования спецификаций не менее 25% — служат стратегическими ориентирами фаз.

2.3 Среда работы ИИ-агента важнее модели

Архитектурную теорию среды мы раскрываем здесь; инженерную реализацию в компонентах IDP — далее в документе.

По состоянию на май 2026 года концепции среды исполнения ИИ-агента уделяют несоразмерно мало внимания. Внимание смещено на обучение и выпуск новых моделей, их «рекорды» на бенчмарках и новые агентские возможности.

Публикации о возможностях ИИ-агентов нередко перепрыгивают с базовых вопросов до самых сложных. От уровня «что есть ИИ-агент, а что нет» они переходят сразу к автономным агентам класса OpenClaw и Ouroboros. Вклад в этот перекос внимания вносят передовые лаборатории (frontier labs), для которых создание среды исполнения стало органичным шагом после обучения модели. Их работа создаёт ложное впечатление, что ИИ-агент — это и есть модель.

Важно понимать: **языковая модель (LLM) по-прежнему остаётся лишь моделью класса генеративного ИИ (GenAI).** Её ключевая функция — генерация, а не совершение действий. Важным этапом развития языковых моделей стали вызов инструментов (tool calling) как результат генерации и способность учитывать итог этих действий в цепочке рассуждений.

Именно среда работы ИИ-агентов (harness) превращает модель в агента:

- расширяет результат модели за пределы сгенерированного текста или кода;
- создаёт автономность агента: результат генерации запускает инструмент, а результат возвращается модели;
- обеспечивает предсказуемую работу модели в любых условиях.

Языковая модель — стохастическая система: её поведение нельзя предсказать с абсолютной точностью. Именно среда исполнения соединяет потребность в предсказуемости со свойствами модели и строится так, чтобы эволюция модели не разрушала выстроенную систему. Модели эволюционируют, форматы меняются, но среда сохраняет протокол взаимодействия. Конкурентное преимущество создаёт детерминированная среда исполнения, а не модель.

Разделение среды на два уровня

Среду работы ИИ-агентов мы разделяем на два уровня. По данным ведущих аналитических агентств, именно это разделение определяет операционную архитектуру ИИ-нативной разработки и задаёт зоны ответственности между поставщиком инструмента и платформенной командой организации.

Первый уровень — **среда-как-продукт (harness product): готовый агентный инструмент с собственной средой исполнения (runtime)**. Владелец — поставщик; примеры — Cursor, Claude Code, OpenAI Codex, Google Antigravity, Kiro, OpenCode. Модель уже встроена, а базовые инструменты вызова и управления сессией предоставляет вендор.

Второй уровень — **среда-как-слой (harness layer): организационная настройка поверх продукта**. Владелец — платформенная команда; это правила-ограничители, стандарты кода, контрольные точки проверки, навыки (skills), управляемые настройки (managed settings), контекст-инжиниринг корпоративного знания.

Уровень	Что это	Кто владеет	Примеры
Среда-как-продукт (harness product)	Готовый агентный инструмент со своей средой исполнения	Поставщик	Cursor, Claude Code, OpenAI Codex, Google Antigravity, Kiro, OpenCode
Среда-как-слой (harness layer)	Организационная настройка поверх продукта	Платформенная команда	Файлы правил, стандарты кода, хуки-ограничители, контрольные точки проверки, навыки (skills), управляемые настройки (managed settings)

Платформенная команда отвечает как за выбор среды-как-продукта, так и за реализацию среды-как-слоя. Продуктовые команды используют готовую конфигурацию. Цикл обучения: инсайт одной команды → кодификация на платформе → тиражирование выгоды. Это организационная модель, при которой эффект от среды масштабируется системно, а не остаётся в пределах одной команды.

Обязательные способности среды

1. Мы фиксируем три обязательные способности, без которых среда-как-слой не обеспечивает предсказуемую работу агента: контекст-инжиниринг — загрузка корпоративного знания в начале каждой сессии. Без неё агент работает в вакууме, не зная соглашений команды, архитектурных решений и технических ограничений.
2. Правила-ограничители (enforcement hooks) — блокировка несоответствующего вывода с принудительной самокоррекцией в реальном времени; это обязательные ограничения, применяемые до исполнения действия.
3. Контрольные точки проверки (verification gates) — эмпирическое доказательство готовности (прошедшие тесты, успешная сборка, скан логов, проверка API) до маркировки задачи как завершённой.

Эти три способности вместе формируют предсказуемость агентной системы. Без контекст-инжиниринга агент генерирует в пустоте. Без правил-ограничителей он нарушает соглашения незаметно. Без контрольных точек проверки незавершённая или ошибочная работа принимается за готовую.

Функции среды исполнения ИИ-агентов

Среда исполнения (harness) — материализация петли реализации. Она выполняет четыре функции, делающие стохастический ИИ применимым для enterprise-разработки:

1. **Трансляция** — преобразование намерения из цикла намерения в машиночитаемый контекст для модели.
2. **Исполнение** — безопасный вызов инструментов (файловая система, API, CLI) с соблюдением квот и прав доступа.
3. **Накопление** — управление контекстом сессии, сжатие контекста, передача информации между сессиями.
4. **Коррекция** — обязательная проверка действий через хуки политик до их применения.

Среда как инфраструктурный актив

Среда исполнения — капитальный актив. Модели меняются ежемесячно, а среда исполнения формируется годами, накапливая контекст организации, библиотеку паттернов и историю аудитов. Инвестиции в IDP и среду исполнения агентов (Agent Runtime) создают капитальный актив, который позволяет менять модели «под капотом» без перестройки рабочих процессов.

Архитектурные связи

- ключевой цикл и принципы;
- изоляция и песочницы;
- контур принуждения и интеграции;
- управление длительными сессиями;
- прозрачность и наблюдаемость.

Альтернативный взгляд на PDLC от AWS (Amazon Web Services)

AWS AI-DLC (ИИ-нативный жизненный цикл разработки) — наиболее проработанная промышленная методология ИИ-нативной разработки. Её ключевые элементы:

- PR/FAQ как инструмент генерации идей с самого начала;
- коллективная детализация спецификации (Mob Elaboration) — итерирование задачи вместе с ИИ-ассистентом;
- SDD как первичный артефакт;
- коллективное построение (Mob Construction) — параллельная сборка подфич агентами;
- автоматизированная верификация через тесты на основе свойств системы (property-based testing) и оценки (evals).

PR/FAQ — входная точка этапа исследования с обязательным определением метрик успеха, а не разовая практика. Коллективная детализация спецификации (Mob Elaboration) начинается после прохождения исследования, а не вместо него. Это закрывает недоработку, когда детализация превращалась в формализацию ad hoc-тикетов.

AWS AI-DLC и AI-Disrupt PDLC: сходства и различия

AI-Disrupt PDLC делает обязательным то, что AI-DLC оставляет опциональным:

AI-DLC не выделяет субцикл исследования как самостоятельный этап (PR/FAQ может появиться или нет); AI-Disrupt PDLC делает исследование обязательным.

AI-DLC не различает SDD для классических систем и Agent Specification для агентных продуктов. Мы разделяем.

AI-DLC не формализует контур валидации как сквозной механизм. Мы его выделяем.

Коллективная детализация спецификации (Mob Elaboration) — практика совместной работы команды над SDD с ИИ-ассистентом. Структура: 60–90 минут; продакт-менеджер,

инженерный лидер, 2–3 старших инженера и ИИ-ассистент в активном режиме. Результат — драфт SDD с заполненными разделами.

Известный антипаттерн — коллективная детализация спецификации без предварительного исследования. Спецификация получается полной по функциям, но без гипотезы бизнес-результата и без ссылки на типовые решения. Риск построить не то решение растёт кратно.

Мы жёстко фиксируем порядок: коллективная работа над спецификацией начинается после исследования, а не вместо него. На входе обязательны пять артефактов: формулировка возможности (Opportunity), PR/FAQ, гипотеза бизнес-результата, явное решение о доработке или реинжиниринге и класс пригодности задачи к агентному выполнению.

Specification-driven development: разработка через спецификацию

Разработка через спецификацию (Specification-Driven Development, SDD) — центральная методологическая опора AI-Disrupt PDLC. Она отвечает на вопрос, как сделать работу агента в существующей кодовой базе предсказуемой и проверяемой.

В контексте AI-Disrupt PDLC мы используем SDD как инструмент управления доработками классических ИТ-систем — унаследованного ядра, ERP/CRM, внутренних сервисов и продуктовых приложений в современном, но ещё не полностью агентном ландшафте. SDD выступает как контракт изменения классической системы, а не как спецификация агентного приложения. Разработка самих агентных продуктов сосуществует с SDD через общую платформу (IDP), контур валидации и контур управления, но опирается на другие первичные артефакты и другие критерии завершения.

Цена отказа от SDD измерима. Аналитики оценивают, что к 2027 году организации, не внедрившие SDD, потеряют до 25% скорости разработки относительно конкурентов. Сам подход зафиксирован как один из стратегических трендов software engineering 2026 года. Его суть — использование живых машиночитаемых спецификаций (как правило, форматы на базе plain text — markdown, схемы) как основного способа управления знаниями в коммуникациях «человек–человек», «человек–агент» и «агент–агент».

Агенты, в отличие от людей, не могут полагаться на интуицию. Они либо галлюцинируют, либо запрашивают уточнения, поэтому нечёткая задача («добавь авторизацию в унаследованный сервис заказов») даёт непредсказуемый результат. Формализованная спецификация с явными требованиями, контрактами и критериями приёмки (acceptance criteria) позволяет агенту работать предсказуемо и верифицируемо в существующей базе кода с её ограничениями и зависимостями.

Принципы SDD для классических систем

Спецификация — первичный артефакт изменения. Код классической системы не перегенерируется целиком; SDD каждый раз выглядит как законченное описание состояния системы после изменений; производные сущности (суть изменений, периметр правок, контракты) — генерируются из него.

Многоуровневая спецификация: бизнес-требования → функциональная спецификация → техническая спецификация (включая контракты с существующими модулями) → тест-спецификация.

Логика вовлечения человека (human-in-the-loop): кто и когда валидирует результат агента.

Паттерны проектирования: типовые решения для задач.

Класс применимости агента: требует ли задача агента вообще.

Главное изменение: прежний SDD отвечал на вопрос «Что нужно построить?». SDD для классических систем отвечает на вопрос: «Что нужно изменить в существующей системе, почему, каким агентным способом, с каким уровнем автономии, с какими evals, кем валидируется и как будет доказано, что желаемое состояние достигнуто?».

Элемент	**Классический PDLC**	**SDD как спецификация**	**SDD как контракт изменения**
Первичный артефакт	Код	SDD-спецификация	SDD с гипотезой результата, вовлечением человека и классом применимости агента
Что описывается	Готовая система	Что нужно построить	Что нужно изменить в существующей системе
Критерий завершения	Критерии приёмки	Код в проде, тесты проходят	Проверочный пакет (Evidence Bundle) + измеримый результат
Передача агенту	Невозможна	SDD как инструкции	SDD как контракт исполнения с атрибутами переиспользования, автономии и оценок

Процесс SDD-цикла для классических систем

Цикл SDD проходит шесть последовательных шагов от исследования до проверки результата:

1. Исследование (Discovery): возможность, PR/FAQ, гипотеза результата, адаптация/перепроектирование, класс применимости агента.
2. Коллективная детализация спецификации (Mob Elaboration): команда создаёт спецификацию, фиксируя бизнес-контекст, критерии приёмки, карту решений с участием человека (human-in-the-loop Decision Map) и ссылку на типовые шаблоны.
3. Генерация: агент получает SDD и генерирует реализацию в существующей кодовой базе.
4. Верификация результата специализированными агентами: агент-ревьюер, проверки безопасности, регрессии, оценки (evals).
5. Улучшение при необходимости: уточняется спецификация, а не код; код порождается как следствие изменения спецификации.
6. Проверка результата (Outcome Check): после развёртывания результат измеряется и возвращается в спецификацию.

Отличия классического SDD и SDD в агентных системах

Спецификация агентной системы описывает не только желаемое состояние, но и способ его достижения на уровне функций, автономности исполнения (критерий отказа от реализации, эскалации и структуры циклов):

Причина проста: раньше спецификацию читал человек. Он знал, у кого уточнить, как выбирается решение, какой стиль кода принят. Для полной автономности эти ответы нужны

агенту в контекстном окне — то есть должны быть сформулированы в явном машиночитаемом виде.

Уровень детализации SDD согласуется с классом применимости агента. Для детерминированных задач спецификация может быть очень подробной, фактически псевдокодом. Для ИИ-нативных задач SDD должен оставлять агенту свободу в выборе реализации, фокусируясь на «что» и «почему», а не «как». Для агентных задач — долговременных, многоагентных — она дополнительно включает декомпозицию сессии: какие фазы исполнения, какие агенты, какие контрольные точки.

Единый шаблон спецификации не подходит для всех уровней — отсюда необходимость типовых шаблонов с разделением по классам применимости агента. Контракты с существующими модулями — критический раздел SDD для классических систем, отсутствующий у спецификаций для разработки с нуля (greenfield). В нём фиксируются используемые интерфейсы существующей системы (с явной ссылкой на их API-спецификацию); инварианты, которые должны сохраниться (формат логов, контракты с нижележащими системами, схемы базы данных); интерфейсы, которые меняются, и способ изменения (политика версионирования: обратная совместимость, feature флаги / breaking changes). Без этого раздела агент в унаследованной системе склонен вносить локально-корректные изменения, нарушающие глобальные контракты.

Мы принципиально разводим спецификацию классической системы (SDD) и спецификацию агента (Agent Specification). Инструменты первого класса (Amazon Kiro, Claude Code в режиме спецификации, OpenSpec) заточены под написание кода в существующей базе: они учитывают её зависимости, ограничения развёртывания и неизменяемые свойства интерфейсов. Инструменты второго класса (Anthropic Constitutional AI, OpenAI Behavior Specs, Cohere agent designer) заточены под формирование поведения нового агентного продукта и работают в логике безопасных принципов, сценариев эскалации и feature флагов, а не в логике контрактов с унаследованным кодом.

Три антипаттерна SDD

Три антипаттерна превращают спецификацию из контракта в формальность. У каждого своя защита:

- **SDD как wiki** — спецификация написана, но не является контрактом: нет гипотезы результата, класса применимости агента, карты участия человека. Защита: обязательные блоки шаблона не опциональны.
- **SDD как ТЗ** — «как» расписано до мелочей, лишает агента свободы выбора реализации. Защита: обучение команды совместной работе над спецификации, умению выдерживать фокус на «что» и «почему».
- **SDD как свершившийся факт** — написана до этапа исследования и подогнана под реализацию. Защита: SDD ссылается на артефакты исследования (идентификатор PR/FAQ, идентификатор гипотезы результата), без них агент-ревьюер отклоняет спецификацию.

SDD сегодня — это контракт между циклом намерения и циклом реализации для классических ИТ-систем. Он отвечает не только на «что» и «зачем», но и на «каким способом» (класс применимости агента), «с какой автономией» (уровень R), «с каким уровнем валидации» (участие человека и план оценок), «с какими артефактами», «с каким откатом», «с каким результатом».

Принципиальное разделение: SDD — методология доработки классических ИТ-систем; Agent Specification — методология разработки агентных продуктов. Эти контуры

сосуществуют через общую IDP, методику валидации ИИ-результатов и типовые шаблоны, но имеют разные первичные артефакты и разные критерии завершения.

Гипотеза результата в SDD

Гипотеза результата (Outcome Hypothesis) — обязательный раздел шаблона спецификации. Её задача — зафиксировать измеримый бизнес-результат изменения до начала реализации, а не вывести его постфактум из созданной функциональности. Источник идеи — два исследования McKinsey: концепция agentic organization с командами, ориентированными на бизнес-результат, и обзор 2025 года, где переосмысление бизнес-процессов названо главным фактором различения лидеров и отстающих.

Гипотеза результата состоит из четырёх обязательных полей:

- **Бизнес- или продуктовая метрика:** что должно измениться (конверсия, NPS, доля мошеннических операций, время до первой транзакции, среднее время устранения инцидента).
- **Метрика-предвестник (leading indicator):** показатель, который начнёт двигаться раньше основной метрики (клики как предвестник конверсии).
- **Период подтверждения:** число дней или недель после развёртывания, в течение которых результат измеряется. Стандартные значения — 7 / 14 / 28 / 90 дней.
- **Критерий отката (fallback):** при каких условиях выводы о результате помечаются как «не подтверждены» и каким будет следующее действие команды — откат, ретроспектива, новая гипотеза.

Гипотезы результата для классических систем формулируются с явными baseline и target:

- **Бэкенд e-commerce:** «Время до оформления заказа сокращается с медианных 4,2 минут до <3,0 минут в течение 14 дней после развёртывания; предвестник — клики по кнопкам быстрого действия в корзине; критерий отката — если время не меняется за 14 дней, ретроспективный анализ: гипотеза была про конкуренцию интерфейсов, не про скорость».
- **Обработка обращений о выплатах в страховании:** «Среднее время устранения снижается с 36 часов до менее 18 часов в течение 28 дней; предвестник — доля автоматически направленных обращений; откат — если автомаршрутизация <40%, ретроспективный анализ: расчёт сложности неточен».
- **Внутренний административный инструмент:** «Онбординг нового сотрудника <1 час против текущих 8 часов; предвестник — доля выполненных пунктов чек-листа в первый день; откат — <70% выполнения запускает ретроспективный анализ».

В гипотезе результата есть три типичных провала:

- **Неоцениваемый эффект** — «улучшить пользовательский опыт». Решение: требовать конкретную метрику с базовым значением и целью.
- **Пустые метрики** — «количество кликов на новой функции» без связи с бизнес-результатом. Решение: ключевой индикатор должен быть связан с основной метрикой.
- **Слишком короткий период подтверждения** — 7 дней для квартальной метрики. Решение: период соответствует логике метрики, а не желаемой скорости обратной связи.

Три отраслевых примера показывают, как измеримая гипотеза результата работает в трёхфазной модели внедрения: в финтехе, страховании и розничном e-commerce. Каждый

пример фиксирует исходную точку, результат через два года и экономическую отдачу от инвестиций в среду.

Финтех. Исходная точка: 80 инженеров и задача вырасти без расширения штата. Через два года, к концу второго горизонта внедрения, ритуалы исследования внедрены во всех командах, разработка через спецификацию используется в 60% команд, библиотека типовых шаблонов выросла до 25 элементов. Инженерные метрики: доля неудачных изменений 8%, среднее время от идеи до прода 4 дня. Продуктовые метрики: доля функций, подтвердивших заявленный результат (Outcome Validation Rate), выросла до 50%; объём кода, выходящего в прод, увеличился в 2,3 раза.
Инвестиции: платформенная команда из пяти человек, программы обучения и инфраструктура платформы — около 2 млн $ за два года.
Результат: выпускная мощность эквивалентна 184 инженерам без их найма; высвобожденная мощность направлена на запуск новых продуктовых линеек, а проверка результата удерживает команды на функциях с высокой ценностью.

Страховая компания. Исходная точка: 40 продуктовых инженеров, legacy система обработки страховых обращений, ИИ применяется только во вспомогательных инструментах. Через два года: ритуалы исследования внедрены в 80% команд, запущен отдельный агент первой линии поддержки для простых запросов, библиотека шаблонов выросла до 30 элементов. Инженерные метрики: доля неудачных изменений 5%, время от идеи до прода 3 дня, Outcome Validation Rate 65%. Прорыв в обработке обращений: последовательный процесс перепроектирован в параллельный с агентной поддержкой — среднее время рассмотрения сократилось с 4 дней до 8 часов, причём в 70% случаев без участия человека.
Инвестиции около 3 млн $.
Результат: пропускная способность обработки обращений выросла втрое, и компания выходит на новые географические рынки без пропорционального найма.

Розничный e-commerce. Исходная точка: 120 инженеров, быстрая инженерная культура, ИИ-эксперименты в персонализации витрины. Через два года: полное внедрение концепции, библиотека шаблонов из 50 элементов, модель компактных команд (Tiny Teams по 3–6 человек с полным владением продуктом от идеи до эксплуатации) внедрена в половине команд.
Инженерные метрики: доля неудачных изменений 4%, время от идеи до прода 2 дня, Outcome Validation Rate 75%, доля мультиагентных сценариев в проде 35%. Скорость вывода функций на рынок сократилась с шести недель до восьми дней, число экспериментов в квартал выросло впятеро, а доля экспериментов, доходящих до серийного продукта, заметно выросла.
Инвестиции около 4 млн $.
Результат: конкурентное преимущество в скорости инноваций и эффект притяжения сильных инженеров.

Отраслевая специфика определяет, какие компоненты среды важнее остальных.

В финтехе жёсткие требования регулятора делают кодифицированные ограничения критическими с первого дня, а участие человека в цикле принятия решений обязательно для любых изменений в обработке транзакций.

В страховании обработка обращений о выплатах больше всего выигрывает от перепроектирования, а измеримую гипотезу легко выразить операционными показателями.

В розничном e-commerce агенты сильнее всего на персонализации, рекомендациях и прогнозировании спроса; бизнес-метрики (выручка, конверсия) делают проверку результата естественной, а мультиагентная оркестрация раскрывается на сложных сценариях, где одновременно работают рекомендательная система, управление запасами и динамическое ценообразование.

Принцип вовлечения человека

Для каждой задачи мы фиксируем: кто, на каких шагах и по каким критериям валидирует результат агента. Ответ закрепляется в карте решений с участием человека (human-in-the-loop Decision Map) как обязательном разделе SDD.

Концепция McKinsey и модель уровней автономности отвечают на разные вопросы:

- Концепция McKinsey формулирует ответ на вопрос: «кто проверит результат».
- Модель уровней автономности (R0–R5) отвечает: «может ли агент это делать».

Поле карты решений с участием человека	**Содержание**	**Пример**
Точка валидации	На каком шаге цикла реализации	Перед слиянием в main; перед развёртыванием в проде
Кто валидирует	Роль, должность или агент	Tech lead для архитектурных изменений; офицер безопасности для полей с ПДН
По каким критериям	Чек-лист и ссылка на набор оценок	Прохождение оценок + ручное ревью архитектурных решений
Что блокирует	Какое действие невозможно без валидации	Невозможно развёртывание в проде
Эскалация	Кому идёт эскалация при разногласии	Инженерный менеджер → директор по инженерии

Карта участия и лестница полномочий дополняют друг друга. Карта определяет содержательные точки валидации, а лестница зрелости агента задаёт ограничения времени исполнения. Один и тот же агент может быть на уровне R3 (имеет право писать код и сливать ветку автоматически) и одновременно требовать валидации от tech lead перед слиянием — эти решения не противоречат, а дополняют друг друга.

Структура карты участия человека в реальном SDD выглядит как таблица с одной строкой на каждую критическую точку валидации. Типичные точки для классической доработки: до запуска агента (правильно ли понята задача); перед слиянием в main (ревью архитектуры); перед развёртыванием в staging (дымовые тесты и проверка безопасности); перед развёртыванием в проде (контрольная точка релиза и согласование владельца бизнес-процесса); через 7 дней после развёртывания (проверка результата). У каждой точки есть владелец, критерии и эскалация.

Human-in-the-loop Decision Map vs лестница зрелости агента R0–R5: ladder отвечает на вопрос «может ли агент выполнить действие в принципе» (ограничение среды исполнения); human-in-the-loop Map отвечает на вопрос «требуется ли человеческая валидация результата перед движением дальше» (процессный шаг). Один и тот же агент в той же ветке может иметь право автомёрджа и при этом требовать принятия решения от техлида перед мёрджем — это не противоречие, а наложение двух ортогональных контролей.

Метрика покрытия участием человека (human-in-the-loop Coverage Score) показывает не долю задач, прошедших валидацию, а долю SDD с явной картой участия. Это качественная метрика: SDD без карты склонен к двум отказам — либо агент получает мало проверок и

выпускает регрессию, либо команда перепроверяет каждое действие и получает усталость от согласований (decision fatigue). Цель — 100% SDD с картой участия человека к концу Горизонта 1.

Библиотека шаблонов и каталог рабочих процессов

Перед написанием новой спецификации команда обязана проверить каталог. Это превращает практику SDD в «сначала переиспользуй, потом пиши» и снижает стоимость токенов на функцию.

Библиотека шаблонов спецификаций (Spec Pattern Library) — каталог типовых шаблонов SDD с метаданными:

- Идентификатор шаблона, название, описание.
- Владелец и заинтересованные стороны.
- Версия и история изменений.
- Набор оценок (evals), применяемый к любой реализации шаблона.
- Статистика использования — сколько раз применён, в каких командах.
- Статус: черновик → ревью → опубликован → в использовании → выведен из эксплуатации.

Каталог переиспользуемых процессов — готовые конфигурации оркестратора и рабочих агентов для повторяющихся задач (новый микросервис, миграция базы данных с минимальным простоем, устранение находки безопасности, обновление зависимостей). Каждый процесс ссылается на применимые шаблоны спецификаций и имеет собственные оценки.

Практику измеряют две метрики. Доля повторного использования спецификаций (Spec Reuse Rate) — процент новых SDD на основе шаблона; переиспользуемость типовых процессов — среднее число использований одного процесса за квартал.

Каталог защищает организацию от разрастания. 4000 пользовательских GPT в одной организации без каталога — это теневой цикл намерения. Мы закрываем это через обязательный реестр спецификаций и процессов: любой новый шаблон проходит жизненный цикл (черновик → ревью → опубликован) с явным владельцем. Соответствующий антипаттерн — «Изобилие процессов».

Целесообразность применения агентов

Прежде чем писать спецификацию, мы классифицируем задачу: детерминированная (классическое кодирование без участия ИИ), дополненная ИИ (ИИ помогает разработчику, но решения принимает человек), ИИ-нативная с фокусом на спецификацию и агентная (агент работает автономно в рамках спецификации). Класс задачи определяет четыре параметра дальнейшей работы: свободу агента, плотность проверок, форму валидации результата и тип типового процесса.

Большинство задач не подходят для агентного исполнения. По данным McKinsey 2025, не более 10% задач масштабируются через агентное исполнение по любой функции. Двойственная природа агента как инструмента и коллеги (по данным BCG) требует чёткой классификации задачи перед выбором паттерна: разработчик в центре (developer-first) или агент в центре (agent-first).

Класс	Когда уместен	Свобода агента	Плотность оценок	Верификация
Детерминированная	Полностью предсказуемые задачи (шаблонная генерация, соответствие требованиям линтера)	Минимальная — строгий процесс без вариативности	Низкая	Модульные тесты + линтинг
Дополненная агентом	Задачи с автодополнением и ревью (~25–35% времени)	Только подсказки, человек принимает предложения ИИ	Средняя	Стандартные тесты + ручное ревью
ИИ-нативный (на базе SDD)	Большинство функций и багфиксов в классических системах	Генерация по SDD, работа в ветке функции	Высокая (оценки способностей и регрессий)	Тесты на основе свойств системы + контур валидации
Агентная	Многошаговые задачи с неопределённостью (рефакторинг, расчистка бэклога)	Автономная декомпозиция и исполнение до контрольных точек	Очень высокая (+ оценки длины сессии)	Контур валидации + контрольные точки, учитывающие горизонт

Критерии классификации: степень неопределённости задачи, требуемая адаптивность, цена ошибки, обратимость действий и наличие применимой библиотеки шаблонов. Метрику фиксирует показатель применимости агента — доля задач, прошедших классификацию перед реализацией.

Расширенный шаблон SDD

Сводный шаблон SDD для классических ИТ-систем включает обязательные блоки:

Блок SDD	Что фиксирует
Бизнес-контекст	Зачем существует задача, какое подразделение или продукт
Гипотеза результата	Бизнес-метрика, предвестник, период, критерий отката
Класс применимости агента	Детерминированный / усиленный ИИ / ИИ-нативный / агентный
Уровень автономии	R0–R5 для данной задачи

Блок SDD	Что фиксирует
Карта решений с участием человека	Кто и когда валидирует результат
Функциональные требования	Что должно делать изменение
Контракты с существующими модулями	Какие инварианты сохраняются, какие меняются
Критерии приёмки (Given-When-Then)	Условия приёмки
Тесты на основе свойств системы + регрессионные	Формальные свойства + защита унаследованных путей
План оценок	Оценки способностей + регрессий + длины сессии (для агентных задач)
Требования к проверочному пакету	Что входит в пакет завершения
Связь с библиотекой шаблонов	Используемый шаблон и процессы
План отката и восстановления	Как откатить агентное изменение
Проверка гипотезы после внедрения	Когда и как будет подтверждена гипотеза

Codified Guardrails (кодифицированные ограничители)

Шаблон SDD расширяется обязательным блоком Codified Guardrails — кодифицированными ограничениями. Это прямая реализация принципа «управление как код» (Governance as Code) на уровне SDD: каждый контракт изменения классической системы фиксирует ограничения контура управления как часть спецификации, а не как проверку на выходе.

Подпункт	Что фиксирует
Требования безопасности	Кодированные правила OWASP/CWE, триггеры модели угроз, границы аутентификации и авторизации
Требования соответствия	Кодированные правила 152-ФЗ, требований ЦБ и применимых норм в виде идентификаторов политик
Ограничения работы с данными	Обработка ПДн, срок хранения, локальность обработки, права субъекта данных
Привязка к лестнице разрешений	Уровень R по умолчанию, условия повышения по критичности, риску и сложности, триггеры эскалации

Кодифицированные ограничители идут на вход в два соседних блока SDD — уровень автономии и карту участия человека. Привязка ограничителей к лестнице разрешений задаёт уровень автономии задачи по умолчанию. А условия, при которых требуется вмешательство человека, переносятся в карту участия. План валидации расширяется отдельной секцией с тестами политик — синтетическими сценариями, проверяющими, что ограничители срабатывают там, где должны.

2.4 Обязательный и рекомендуемый контекст

Различие принципиальное: контекст направляет агента (system prompt, md, документация, примеры использования) и может быть проигнорирован без последствий; политики — это обязательное применение через Policy Hook Framework, который может остановить или переписать действие агента независимо от его намерения. Контекст — нужен; политика — обязательна.

Discovery-артефакты (Outcome Hypothesis, human-in-the-loop Decision Map, ссылка на библиотеку шаблонов) — это контекст для навигации: они задают язык, на котором продукт разговаривает с агентом. При этом методика валидации ИИ-результатов и Policy Hook Framework работают как фреймворки, обязательные к применению, не зависящие от контекста: если security gate говорит «нет», агент не сможет обойти это никаким уточнением промпта.

Практические следствия деления «контекст vs политика»

Документация (md, README, ADR) — это контекст. Если документация устарела, агент принимает локально-неверное решение, но система не сломается. Policy hook (например, запрет на изменения в проде) — это принуждение. Policy block прерывает действие агента независимо от того, что говорит документация.

Не все правила должны быть закодированы как принудительные политики. Если перехватчиков слишком много, агентный конвейер начинает буксовать, и команда теряет в скорости работы. Если их слишком мало, в контуре появляются дыры в безопасности. Правило калибровки простое: политики используем для критичных требований безопасности и соответствия регулированию, а контекст — для соглашений команды и хороших практик, нарушение которых не приводит к инциденту.

Артефакты этапа исследования мы классифицируем по режиму влияния на агента:

Гипотеза бизнес-результата — рекомендательный контекст: агент может не оптимизировать явно эту метрику, но этот артефакт служит сигналом качества на ревью.

Карта решений с участием человека — гибрид: блок «кто валидирует» является рекомендацией, а блок «что блокирует выпуск» — жёстким требованием, проверяемым машиной.

Ссылка на библиотеку типовых решений — по умолчанию контекст, но само типовое решение может содержать принудительные перехватчики (например, обязательный проход ревью безопасности).

2.5 Агентное кодирование: уровни автономии и task horizon

Уровни автономии агентного кодирования эволюционируют от автодополнения до многочасовых автономных сессий. Горизонт задачи (task horizon) — первичная ось зрелости; на верхнем уровне вводится L5 — координированные команды агентов (Coordinated Agent Teams).

По данным Anthropic 2026, горизонты задач расширяются с минут до дней и недель. 99,9-й перцентиль длительности сессии вырос до 45+ минут за 3 месяца. Кейс Rakuten/vLLM: 7 часов автономной работы агента на репозитории 12,5 млн строк с 99,9% числовой точности.

Уровень	Описание	Горизонт задачи	Примеры
L1	Автодополнение	Секунды	GitHub Copilot autocomplete
L2	Инлайн-редактирование по промпту	Минуты	Cursor inline edit, Copilot Chat
L3	Самостоятельная агентная сессия	30 мин – 2 часа	Claude Code в репозитории, Codex CLI
L4	Параллельное выполнение, многочасовые сессии	До 4 часов (зрело: до 1 рабочего дня)	Пакетная работа над несколькими фичами; ночной рефакторинг
L5	Координированные команды агентов: оркестратор и специализированные рабочие агенты с передачей между агентами (A2A handoff)	Многосессионные / многодневные задачи с контрольными точками	End-to-end: спецификация → код → ревью → документация от оркестратора

Парадокс сотрудничества (Collaboration paradox, Anthropic 2026, Trend 4): ИИ используется примерно в 60% работы, но полностью делегированы лишь 0–20% задач. Разрыв 60/20 структурный: улучшение моделей его не закрывает. Мы формализуем это через карту вовлечения человека (human-in-the-loop Decision Map): для каждой задачи явно фиксируется, какая часть делегируется, а какая остаётся в зоне человеческой валидации.

Связанный антипаттерн — надежда на полную делегацию (Full Delegation Hope). **Пятый уровень зрелости мы сознательно не описываем как «полностью автономное предприятие».** Это уровень зрелости цикла реализации, где оркестратор собирает специализированных агентов — разработчика, тестировщика, ревьюера, проверяющего безопасность и пишущего документацию, — взаимодействующих через протокол «агент к агенту» (A2A handoff) и ведущих единый проверочный пакет (Evidence Bundle). Подтверждённые кейсы: Fountain Copilot (предварительный отбор кандидатов на 50% быстрее, онбординг на 40% быстрее), TELUS (13 000 внутренних ИИ-решений, рост скорости выпуска на 30%).

2.6 Децентрализация IDE

Один из шести стратегических трендов SE-2026: IDE перестаёт быть центром процесса разработки. По прогнозам, к 2027 году более 65% команд с agentic coding будут считать IDE опциональной. Цикл реализации работает в IDE / CLI / headless / CI-runner / chat-поверхностях и параметризуется поверхностью, а не привязан к ней.

Это согласуется с долей работы из командной строки в данных Anthropic 2026 (тренд 5) и с замерами Bain о многоповерхностной инженерии. Работа в нескольких поверхностях ужесточает требование к методике валидации: агент-ревьюер, шлюзы безопасности, шлюз пакета доказательств и шлюз релиза обязаны вести себя одинаково — в командной строке, в headless режиме, в конвейере непрерывной интеграции и в чате. Иначе децентрализация превращается в раздробленность и теневые маршруты исполнения.

Архитектура Multi-surface IDP

IDE (VS Code, JetBrains): плагины-агенты с полным доступом к рабочему пространству; оптимальны для интерактивных задач с немедленной обратной связью.

CLI-агенты: терминальные ИИ-агенты; оптимальны для скриптовых задач, автоматизации и работы в серверных средах.

CI-runner: headless исполнение в pipeline; оптимально для автотестирования, проверок безопасности и обновления зависимостей.

Chat-поверхности (Slack/Teams): интерфейсы для долгих задач (сводки, запросы на подтверждение); не для интерактивного кодирования.

Дашборды: для наблюдения, просмотра библиотеки шаблонов.

Цикл реализации работает одинаково на всех поверхностях за счёт общей среды исполнения. Поверхность — это адаптер ввода/вывода; reasoning, tool dispatch, policy enforcement живут в среде исполнения. Это ключ к децентрализации: поверхность может меняться (разработчик переключается с IDE на CLI на chat), но возможности «под капотом» не меняются. Без общей среды исполнения каждая поверхность имеет свою реализацию — фрагментация и shadow пути неизбежны.

2.7 Архитектурные паттерны агентных систем

В работе с нетривиальными задачами результат формируется за счёт выстроенной координации агентов. Мы выделяем шесть базовых паттернов проектирования, каждый со своим уровнем сложности и режимом отказа: цепочка промптов (prompt-chaining), маршрутизация (routing), распараллеливание (parallelization), оркестратор и рабочие агенты (orchestrator-workers), оценщик и оптимизатор (evaluator-optimizer), и автономный агент (autonomous agent).

Каждый паттерн уместен на своём уровне сложности задачи и имеет собственный набор проблем. По данным Anthropic (2026), схема «оркестратор и рабочие агенты» становится выбором по умолчанию для нетривиальных задач.

Паттерн	**Когда уместен**	**Режим отказа**
Цепочка промптов	Линейные задачи с детерминированной последовательностью	Накопление ошибок по цепочке
Маршрутизация	Классификация ввода и направление в нужный обработчик	Ошибки классификации на пограничных случаях
Распараллеливание	Независимые подзадачи с агрегацией	Несогласованные результаты между ветками
Оркестратор и рабочие агенты	Сложные задачи с динамической декомпозицией	Накладные расходы на координацию, ~15× больше токенов
Оценщик и оптимизатор	Итеративное улучшение результата по критерию	Цикл без сходимости

Паттерн	Когда уместен	Режим отказа
Автономный агент	Открытые задачи с инструментами и средой	Галлюцинированные вызовы инструментов, сломанное окружение

Выбор паттерна — решение цикла исследования (Discovery Loop), и каждый паттерн усиливает одно свойство за счёт другого.

2.8 Контекст-инжиниринг и сжатие контекста

Контекст-инжиниринг (context engineering) — четвёртый из шести стратегических трендов инженерии ПО 2026 года. По прогнозам, к 2028 году сильнейшие организации формализуют машиночитаемый контекст как стратегический актив.

Пятиуровневая модель сжатия контекста позволяет долгоживущим агентам сохранять связное состояние без переполнения контекстного окна модели:

- L1, сырой контекст — полный журнал сессии без обработки, только для коротких задач.
- L2, структурированные журналы — вызовы инструментов и их результаты в формате JSON.
- L3, сводки — текстовые сжатия на каждые N сообщений, обновляются автоматически.
- L4, устойчивые заметки — файлы NOTES.md и progress.md, явно поддерживаемые агентом.
- L5, типизированная память — типизированные факты в объектной модели PDLC, доступные через поиск и фильтрацию.

Сессия начинается с уровня L1 и сжимается по мере заполнения контекстного окна. При заполнении окна примерно на 30% содержимое сжимается до L2, около 60% — до L3; долгоживущие заметки уходят в L4, типизированные факты — в L5. Это непрерывный процесс: если при сжатии теряются ключевые факты, агент регрессирует. Качество измеряют два показателя — целостность сохранённого контекста (доля сессий без смещения смысла) и доля потерь при сжатии.

Типизированная память (L5) решает проблему потери контекста между сессиями. Факты типизированы (задача, файл, решение, результат теста, зависимость), поддерживают запрос по типу и фильтр по статусу, сохраняются между сессиями и доступны оркестратору для агрегации. Это прямой ответ на ситуацию, когда агент забывает контекст прошлой сессии при работе над многодневными задачами.

Оговорка о native API и MCP:

Нативная поддержка контекста надёжнее MCP-интеграции: языковая модель может игнорировать MCP-источник, даже когда явно проинструктирована его использовать. Наш подход: native API предпочтительнее MCP там, где это возможно. MCP как протокол не отвергается — он ценен для интеграции разнородных источников контекста. Однако в производственных условиях нативная интеграция даёт более предсказуемые результаты. MCP-интеграция уместна при отсутствии нативной альтернативы, но это осознанный компромисс, фиксируемый в ADR.

Качество работы контекстного слоя мы измеряем двумя показателями. Первый — целостность сохранённого контекста: доля сессий, в которых сохранённый между запусками

контекст не подвергся искажению (drift) и не был отравлен внешними данными (poisoning). Второй — частота галлюцинаций как функция длины сессии: насколько растёт число выдуманных фактов по мере того, как контекст накапливается и удлиняется горизонт работы агента.

2.9 Мультиагентные системы: паттерны проектирования

Когда одного агента мало

Одиночный агент живёт внутри одного окна контекста, одной линии рассуждения и одного набора инструментов. Для сложных задач это узкое место: либо окно контекста исчерпывается, либо в нём накапливается шум, и качество падает.

Решение — мультиагентная схема, в которой подзадачи делегируются субагентам с собственными, изолированными окнами контекста. Такая изоляция даёт три эффекта сразу: основной контекст остаётся чистым, подзадачи идут параллельно, а сбой внутри одного субагента не разрушает всю работу. По обзору Anthropic за 2026 год такая схема становится выбором по умолчанию для нетривиальных задач.

Иерархическая оркестрация как эталон

Канонический паттерн — иерархическая оркестрация: центральный агент-координатор ведёт общую цель и распределяет работу между специализированными исполнителями — по кодированию, тестированию, ревью, документации, безопасности. По идее это близко к архитектуре «смесь экспертов» в языковых моделях, но перенесено на уровень агентов.

Центральный агент отвечает за общую цель, видит всех исполнителей, собирает их выводы в единый результат и принимает финальное решение. Субагенты имеют узкую специализацию, изолированные окна контекста, ограниченный доступ к инструментам и возвращают координатору не только результат, но и доказательства того, что они сделали.

Работа координатора не сводится к задаче «вызвать агентов по порядку». Он разбивает задачу динамически, назначает субагентов по их специализации, видит частичные отказы и при необходимости переназначает работу другому исполнителю.

Когда делегировать, а когда делать самому

Решение центрального агента о делегировании опирается на два простых правила. Первое: если для подзадачи есть специализированный исполнитель — не выполнять её самому. Второе: если задача не влезает в выделенное окно контекста — она требует разбиения. Само то, что задачу нельзя вложить в одно окно — это количественный сигнал о том, что пора декомпозировать.

Отсюда же и обратные ограничения. Не использовать мультиагентную схему для коротких задач до тридцати минут одной сессии — накладные расходы съедят выигрыш. Не игнорировать сломанное окружение внутри субагента — эскалировать вопрос на уровень координатора, иначе работа незаметно остановится. Эти правила вынесены в библиотеку шаблонов как отдельный паттерн мультиагентной оркестрации.

Агент-ревьюер как обязательный участник

Из всех специализаций одна является обязательной в зрелой конфигурации — агент-ревьюер. Он анализирует вывод агента-разработчика на стадии перед слиянием и становится первым контрольным рубежом в методике валидации ИИ-результатов — раньше ревью

человека и раньше шлюзов безопасности. Идея в том, чтобы автоматический контроль качества был встроен в схему изначально, а не добавлен сверху после инцидента.

Связь с лестницей зрелости и цена вопроса

Мультиагентная схема — не революция, а следующая ступень после одиночных агентов. Рабочее правило выбора топологии: большинство задач (60–70%) остаются за одиночным агентом; задачи с тремя и более фазами — это схема «координатор и исполнители»; задачи с пятью и более специализированными ролями — полный режим созвездия, соответствующий уровню зрелости L5.

Доля задач, идущих в режиме «координатор и исполнители» — это и есть ключевая метрика зрелости мультиагентных систем. Цель Горизонта 3 — не менее 30%.

Цена такой схемы существенная. Мультиагентный режим потребляет примерно в 15 раз больше токенов, чем одиночный агент. Это плата за координацию, передачу контекста между ролями и повторные вычисления. Оправдана она только там, где выигрыш от параллелизма, изоляции и специализации перевешивает эти накладные расходы; рутинные задачи должны оставаться за одиночным агентом.

2.10 Долгосрочные агенты: внутренний и внешний цикл, передача сессии

Двухпетлевая модель реализуется через внутренний и внешний циклы реализации. Внутренний цикл — итерации «правка → тест → запуск → исправление» в рамках одной сессии, каждая ограничена бюджетом токенов, тайм-аутом и числом попыток. Внешний цикл — переход между сессиями через контрольные точки: полный снимок состояния задачи с выполненными подзадачами, открытыми вопросами и ссылками на артефакты.

Политика контрольных точек

Мы фиксируем политику контрольных точек (Checkpoint Policy) как обязательный элемент среды. Каждые N минут или M итераций агент автоматически делает контрольную точку с тремя действиями: сохраняет файл прогресса (progress.md), обновляет список функций (feature-list.json), сохраняет указатель на проверочный пакет (Evidence Bundle). Без политики контрольных точек длинные сессии становятся высокорисковыми: один сбой приводит к потере всей проделанной работы.

Протокол передачи сессий. Мы **фиксируем стандартный набор артефактов передачи между сессиями — протокол передачи сессий (Session Handoff Protocol).** Скрипт восстановления окружения (init.sh) поднимает зависимости, переменные среды и доступы за минуты, а не за часы. Файл прогресса (progress.md) фиксирует состояние выполнения, открытые вопросы и следующие действия в человекочитаемом виде. Машиночитаемый список оставшихся подзадач (feature-list.json) позволяет агенту следующей сессии понять остаток работы без чтения переписки. Указатель на накапливаемый проверочный пакет (Evidence Bundle) даёт ссылку на единый артефакт со всей историей изменений, прогонов валидации и решений. Эти четыре артефакта — обязательный компонент платформы.

Постоянный процесс

Стандартный путь для ночного рефакторинга, обновления зависимостей, устранения замечаний безопасности и расчистки накопленных задач. Маршрут такой: спецификация → машиночитаемый перечень подзадач → скрипт восстановления окружения → ночной прогон с

периодическими контрольными точками → утренний проверочный пакет → человеческое ревью по одной сводке, а не по каждой подзадаче.

Передача фаз между агентами (A2A Phase Handoff)

Передача фаз выстраивает цикл реализации как направленный граф. Маршрут «разработчик → тестировщик → ревьюер → документатор», где каждая точка — сообщение по протоколу передачи между агентами (A2A) с минимальным контрактом: имя задачи, ссылка на спецификацию, ссылка на артефакт, указатель на проверочный пакет. Это формализует цикл реализации как граф передач вместо работы одиночного кодирующего агента.

Оппортунистический бэклог — отдельная очередь задач, которые агенты исполняют в свободные циклы: мелкие доработки, накопленный технический долг, низкоприоритетные улучшения. По данным Anthropic (2026), 27% работы с участием ИИ приходится на задачи, которые иначе не были бы сделаны. Это новый класс задач для цикла реализации, не входивший в обычный бэклог.

Расчистка бэклога — отдельный важный сценарий долговременных агентов. Большинство классических ИТ-систем накопили технический долг, который никогда не получает приоритета: обновления зависимостей с известными уязвимостями, предупреждения линтера, мёртвый код, недокументированные модули, пробелы в покрытии тестами. Эти задачи бессрочно заблокированы в обычном бэклоге из-за низкой бизнес-ценности и высокой стоимости переключения контекста для разработчиков. Долговременный агент с подходящей средой работает через такой бэклог ночами и по выходным, поднимая покрытие с 65% до 85% за месяц без вмешательства человека, кроме приёмки пакетов пул-реквестов по утрам.

Экстренное усиление (surge staffing) — режим цикла реализации для проектов с «горячим» подключением незнакомых исполнителей: реагирование на инцидент безопасности, срочная миграция, требуемая регулятором функция. В кейсе Augment Code проект, оценённый техническим директором в 4–8 месяцев, выполнен за 2 недели с агентами в режиме экстренного усиления. Ключевой фактор — протокол передачи сессий с подробным init.sh: новый агент восстанавливает полный контекст за минуты, а не за дни.

Типичные ошибки

- **Одиночный длинный прогон** — попытка запускать долговременную работу без подготовленной среды.
- **Преждевременная победа** — агент завершает задачу при ложном сигнале успеха.
- **Сломанное окружение** — нестабильная среда, требующая самовосстановления (Self-Recovery Rate).
- **Перелёт по горизонту** — переход на уровни L4/L5 без зрелой среды, покрытия оценками и политики контрольных точек.

Разметка внутреннего цикла

Каждый шаг внутреннего цикла фиксируется в структурированном журнале. Это даёт три вещи сразу: аудируемость каждого решения агента, метрики производительности (число попыток на функцию, стоимость в токенах на итерацию, доля регрессий на итерацию) и возможность последующего разбора отказов. Без этого внутренний цикл остаётся чёрным ящиком, непригодным ни для калибровки, ни для оптимизации.

Внешний цикл отслеживается через файл прогресса. Файл progress.md ведётся в реальном времени: он фиксирует статус каждой подзадачи (в работе, выполнено, заблокировано,

пропущено) и указатель на накапливаемый проверочный пакет с данными: изменения кода, прогоны валидации, события аудита и стоимость генерации.

2.11 Eval-driven development и evidence bundle как контракт завершения

Недетерминированность результатов агента требует нового контракта завершения задачи: разовый приёмочный тест уступает место статистической проверке. Решение — разработка, управляемая оценками (eval-driven development, EDD): пятый из шести стратегических трендов инженерии ПО (SE-2026). По прогнозам, к 2028 году её примут 70% команд инженерии ПО (менее 20% в 2026 году).

Каждая агентная система покрывается тремя базовыми классами оценок (evals). Оценки способностей (capability evals) фиксируют штатное поведение системы, оценки регрессий (regression evals) — сохранность поведения при изменениях, оценки длины сессии (session-length evals) — рост частоты галлюцинаций с удлинением сессии.

К трём базовым классам мы добавляем четвёртый — оценки эскалации (escalation evals). По данным обзора Anthropic за 2026 год, зрелость агента определяется умением вовремя признать выход задачи за пределы его компетенций и передать решение человеку. Это поведение тестируется отдельно: агенту ставится неопределённая ситуация и проверяется, эскалирует ли он решение или принимает его сам с риском ошибки. Оценки эскалации включаются в план проверок внутри спецификации.

Evidence Bundle как контракт завершения. Цикл реализации не завершён, пока не сформирован проверочный пакет (Evidence Bundle). В пакете восемь элементов:

- Изменения кода — различие (diff) со ссылкой на пул-реквест.
- Результаты валидации — четыре класса проверок: способности, регрессии, длинные сессии, корректность эскалаций.
- Отчёт о расходе токенов — сколько токенов израсходовано и какими моделями.
- Аудируемый след — журнал сессии в формате «только добавление».
- Аттестации методики валидации — отметки агента-ревьюера, шлюза безопасности, шлюза политик и шлюза релиза.
- Обоснование действий — структурированное объяснение для операций уровня R2 и выше.
- Вклад в объём результата — сколько новых артефактов создано относительно базовой линии.
- Указатель на проверку исхода — ссылка на измерение бизнес-исхода после развёртывания.

Качество цикла реализации измеряется тремя показателями:

- Полнота проверочного пакета — доля задач, по которым собран весь пакет без пропусков.
- Частота регрессий, внесённых агентом, — доля ошибок, попавших в основную ветку из-за действий агента и обнаруженных только после развёртывания.
- Корректность эскалаций — насколько обоснованно агент передавал решения человеку: эскалировал там, где нужно, и не эскалировал по пустякам.

Классы оценок

Четыре класса оценок покрывают разные риски.

Оценки способностей проверяют штатное поведение агента: критерии приёмки формализуются как тесты на основе свойств системы (property-based tests), агент проходит их при успешной реализации. Результат — «прошёл/не прошёл» по каждой оценке, прогон автоматический и недорогой в конвейере непрерывной интеграции; целевое покрытие — 100% критериев приёмки.

Оценки **регрессий защищают от непреднамеренных изменений.** Набор тестов для существующей функциональности прогоняется после каждого изменения. Порог прохождения — не ниже 95%; падение ниже порога блокирует слияние. Поддержка тестового набора делает эти оценки умеренно затратными. Показатель — частота регрессий, внесённых агентом (Agent-introduced Regression Rate).

Оценки длины сессии отражают специфику долгосрочных задач. Частота галлюцинаций растёт нелинейно с горизонтом задачи — это структурная характеристика текущих моделей, а не дефект. Готовность команды к уровню L4 калибруется по частоте галлюцинаций на целевом горизонте.

Оценки эскалации измеряют способность агента вовремя просить помощь. Агенту даётся набор задач с известной неоднозначностью; ожидаемое поведение — эскалация вместо угадывания. Пропуск эскалации ведёт к ошибке в промышленной эксплуатации; лишняя эскалация — к усталости от подтверждений (approval fatigue). Оба случая измеряет показатель качества эскалаций (Escalation Quality Score).

2.12 Слой сквозной ИИ-валидации

Контур валидации — качественное измерение контура управления (governance mesh). Он проверяет качество и корректность результата (outcome) через оценки, проверочные пакеты и проверки регрессий и отвечает на вопрос «правильно ли сделано». Он входит в единый контур управления как одно из его измерений.

Если цикл реализации ускоряется в 5–10 раз, узким местом становятся ревью, интеграция и релиз. Антипаттерн «разрыв реализации и валидации» (Implementation-Validation Gap) возникает, когда ускорение кодирования не сопровождается ускорением ревью, интеграции и релиза.

ПРИНЦИП: цикл реализации не может масштабироваться быстрее методики валидации. При инвестиции X в цикле реализации инвестируется как минимум 0,5X в методике валидации ИИ-результатов.

Архитектурный выбор. Методика валидации ИИ-результатов — не третья большая петля наряду с намерением и реализацией, а сквозное измерение контура управления:

- В петле намерения (через исследование и спецификацию) задаются критерии валидации: план валидации, карта участия человека и требования к проверочному пакету.
- В петле реализации исполняются прогоны валидации: агент-ревьюер работает на стадии перед слиянием, шлюзы безопасности и политик срабатывают перед слиянием и перед релизом.
- После развёртывания телеметрия возвращается в спецификацию через проверку исхода и обновляет реестр валидаций.

Прирост качества от самокоррекции и принцип детерминированного исполнения

По данным аналитических агентств, цикл самокоррекции при фиксированной модели повышает доверие к выходам с 53,8% до 81,8%. Это количественное обоснование инвестиций в правила-ограничители и проверочные шлюзы как элементы среды. Они формируют петлю обратной связи, в которой агент сам корректирует вывод. Прирост в 28 п. п. обеспечивает архитектура среды при неизменной модели. Это делает самокоррекцию стратегической ставкой уровня инфраструктуры.

Лучшее применение: ИИ генерирует детерминированные правила, которые затем применяют обычные инструменты. По данным ведущих аналитических агентств, промышленная надёжность выше там, где ИИ работает на этапе проектирования (генерирует правила, проверяет спецификации, готовит наборы оценок), а детерминированные инструменты — на этапе исполнения (применяют правила, прогоняют оценки, блокируют нарушения).

ПРИНЦИП. LLM решает — детерминированные правила исполняют. ИИ задаёт структуру проверок; код выполняет их механически и предсказуемо, без стохастических отклонений.

Это меняет архитектуру конвейера ревью. Вместо запуска модели на каждом пул-реквесте модель единожды генерирует план оценок (Eval Plan) и правила политик, а детерминированный конвейер применяет их к каждому пул-реквесту. Стоимость ревью падает, предсказуемость растёт, аудитный след детерминирован.

Компоненты контура валидации

Компонент	**Назначение**
Реестр оценок (Eval Registry)	Оценки способностей, регрессий, длины сессии и эскалации
Агент-ревьюер (Review-Agent)	Анализ кода, архитектурное ревью и стиль перед слиянием
Шлюз безопасности (Security Gate)	Проверки SAST/DAST, секретов и зависимостей
Шлюз политик (Policy Gate)	Уровни R0–R5, проверки прав, перехватчики
Шлюз проверки пакета доказательств (Evidence Bundle Gate)	Проверка полноты проверочного пакета
Релизный шлюз (Release Gate)	Готовность к развёртыванию и откат
Телеметрическая обратная связь (Telemetry Feedback)	Промышленные метрики → оценка → спецификация

Контур валидации измеряют пять показателей: отношение темпа реализации к темпу валидации, полнота проверочного пакета, частота регрессий, внесённых агентом, качество эскалаций и доля подтверждённых бизнес-исходов. Валидация масштабируется в одном

темпе с реализацией и отвечает на вопрос «правильно ли сделано» — это удерживает ускорение кодирования в границах проверяемого качества.

2.13 Контур управления (Governance Mesh): третья ось PDLC

Контур управления (Governance Mesh) — единый сквозной слой, отвечающий за весь контур контроля ИИ-нативной разработки. Он состоит из трёх измерений:

- методика валидации ИИ-результатов — проверка качества и корректности результата («правильно ли сделано»);
- проверка прав — право на действие («можно ли это вообще делать и кем»);
- аудит/комплаенс — доказательность и соответствие регуляторным требованиям («как мы это докажем»).

Контур управления — сквозной слой (mesh), а не gate (gateway): он пронизывает цикл намерения и цикл реализации, действуя одновременно на всех уровнях. Метафора: если двухпетлевая модель — это контур обмена веществ, то Governance Mesh — иммунная система, действующая на каждом уровне одновременно и адаптивная по своей природе. Методика валидации ИИ-результатов — её качественное измерение; permission/policy и аудит/комплаенс — два других измерения того же зонтика.

Свойства контура управления

Контур управления устроен по трём свойствам, соответствующим трём фундаментальным сдвигам governance для ИИ-нативной разработки ПО:

Решаемая проблема	**Классический подход**	**ИИ-нативный подход**
Человек не успевает за генерируемыми ИИ-рисками	Человек вовлекается на этапе приемки	Автономный анализ от ИИ
Управление по остаточному принципу	Управление как надстройка	Встроенные правила в SDD
Статические правила в мире ИИ	Фиксированные фреймворки	Адаптивные фреймворки

Автономность. Надзор, выполняемый самими ИИ-агентами на скорости ИИ-нативной разработки. Агенты-ревьюеры, агенты-верификаторы и инструменты автоматического исправления становятся первой линией защиты; ревью человека резервируется для исключений, высокорисковых случаев и стратегических решений.

Встроенность. Ограничители встроены в платформу: они кодифицированы внутри спецификации и приводятся в исполнение через каркас политик и принцип «управление как код». Управление становится системным, а не зависимым от каждой конкретной задачи. Внутренняя платформа предоставляет машиночитаемые интерфейсы — REST-вызовы, командную строку, события-перехватчики, — через которые агенты обращаются к управлению напрямую, не только через порталы для людей.

Адаптивность. Контроль, динамически подстраивающийся под бизнес-критичность, профиль риска и архитектурную сложность конкретной задачи. Лестница автономии R0–R5 перестаёт быть статичной: фактический уровень автономии пересчитывается во время исполнения для каждой агентной операции по триаде «критичность × риск × сложность».

Связь контура управления и модели разработки

В петле намерения критически важно свойство встраиваемости: SDD с кодифицируемыми ограничениями соблюдает ограничения по мере создания кода. В петле реализации критически важно свойство адаптивности: каждая агентная операция пересматривает возможную автономность в зависимости от runtime-контекста. В методике валидации ИИ-результатов критически важно свойство автономности: review-agents и verification-agents становятся ключевыми компонентами «правильно ли сделано», параллельно с человеком.

Практические следствия перехода к Governance Mesh:

- **Ограничения управляемости мы фиксируем в спецификации (SDD) до начала реализации.** Блок «Кодифицированные ограничения» (Codified Guardrails) содержит машиночитаемые правила, которые среда применяет автоматически. Нарушение требования дешевле обнаружить на этапе спецификации, чем на аудите после выхода в промышленную эксплуатацию.
- **R-level — динамическая функция задачи.** Один и тот же агент действует на R3 при рефакторинге и автоматически переключается на R1 при работе с ПДН-полями.
- **Аудитный след — побочный продукт правильно устроенной среды**. Append-only JSONL session-лог формируется автоматически при каждой агентной операции — аудитор получает machine-verifiable запись каждого действия.

Контур управления мы реализуем в двух взаимодополняющих режимах:

- **Превентивный режим** встраивается в цикл намерения через спецификацию и кодифицированные защитные ограждения: ограничения сформулированы до того, как агент начинает работу.
- **Детективный режим** встраивается в цикл реализации через фреймворк перехватчиков политик (Policy Hook Framework), агента-ревьюера и проверки безопасности: ограничения применяются в реальном времени и до завершения каждой операции. Методика валидации ИИ-результатов остаётся в силе, но получает сигналы из сети управляемости: валидация проверяет, что сделано «правильно», а сеть управляемости — что сделано в соответствии с правами.

Следующий вопрос — кто исполняет этот процесс: роли, компетенции и командные модели ИИ-нативной организации. Часть 3 раскрывает человеческое измерение трансформации: как меняются инженерные профили, какие команды приходят на смену традиционным структурам и что отличает ИИ-нативного инженера от разработчика с ИИ-инструментами.

Часть 3. Роли, команды, компетенции

Часть 3 отвечает на вопрос, кто управляет агентами и как устроена команда, извлекающая из них ценность. Мы описываем новую конфигурацию ролей, компетенции инженера, модель компактных команд, сдвиг рынка труда и уровни зрелости — организационный слой, без которого технология остаётся экспериментом. Речь идёт о промышленной эксплуатации, а не о лабораторной демонстрации.

3.1 От исполнителя к оркестратору

Сдвиг роли инженера в ИИ-нативной команде — не количественный, а качественный: инженер перестаёт быть основным исполнителем кода и становится оркестратором — проектировщиком систем, постановщиком задач для агентов и верификатором результатов.

По данным Anthropic (2025), инженеры фиксируют не сокращение рабочего времени, а кратный рост объёма выпуска: +50% к производительности в самооценке, +67% слитых pull requests на инженера в день. Время на каждую задачу снижается незначительно — но количество выпущенных фич, исправленных ошибок и проведённых экспериментов растёт существенно.

Оркестратор начинает работу с исследования (Discovery sub-loop), а не с тикета — по формулировке McKinsey Agentic Org: «определять цели, делать выбор между приоритетами и направлять результаты». Владение циклом исследования закрепляется за Product Engineer / AI Orchestrator — это не добавление новых ролей, а закрепление владения за уже существующими.

Так выглядит типичный день оркестратора в зрелой команде Горизонта 3:

- Утро — просмотр дайджестов после ночных агентных прогонов, приоритизация; сессия исследования с продакт-менеджером (1–2 часа)
- День — запуск 2–3 параллельных агентных сессий по утверждённым SDD
- После обеда — финальное ревью PR, отмеченных агентом-ревьюером; решение об одобрении или эскалации
- Конец дня — проверка результатов фичи, завершённой неделю назад; пополнение Pattern Library

Сдвиг к оркестратору не уменьшает влияние — он его переформатирует: оркестратор владеет десятками задач параллельно против одной-двух у индивидуального исполнителя. Профиль работы меняется: меньше глубокой фокусировки на одной задаче, больше решений о качестве и приоритетах.

Оркестратор пишет меньше кода не потому, что устал или утратил навык, — а потому что его время дороже как время постановщика задачи, верификатора и хранителя контекста, чем как время исполнителя.

Сдвиг к оркестратору требует других личных качеств: толерантности к прерываниям, структурированного мышления, готовности к постоянному ревью чужих решений. Не каждый зрелый инженер хочет или может перейти в этот режим — это легитимный карьерный выбор, а не дефект.

3.2 Новые роли в ИИ-нативной команде

Новые роли появляются только при достижении масштаба, требующего выделения функции — преждевременная специализация ведёт к бюрократическим издержкам:

До этого момента задача не в создании ролей, а в уточнении владения тремя сквозными способностями: **циклом исследования (Discovery Loop), методикой валидации ИИ-результатов и средой исполнения долговременных процессов (Long-Running Runtime).**

В основе ролевой специализации лежит принципиальное различие между двумя классами задач. Gartner (Hype Cycle for Platform Engineering, 2026) фиксирует его через концепцию Agent Experience (AX): платформа и рабочая среда должны проектироваться для двух разных потребителей — человека и агента. Одни роли создают интерфейсы для человека — исследовательские сессии, продуктовые решения, валидацию результата. Другие проектируют среду для агента — machine-readable API, guardrails, политики эскалации, observability. Это различие и определяет профиль каждой роли в команде. Практически оно проявляется как два паттерна работы: developer-first (разработчик задаёт структуру, агент помогает) и agent-first (задача делегируется агенту, ответственность за результат остаётся у человека).

Роль	**Зона ответственности**	**Владение**
Product Engineer	End-to-end поставка фич: от исследования (Discovery) до production	Цикл исследования, гипотеза результата (Outcome Hypothesis), проверка достижения результата после развёртывания (post-deploy check)
AI Orchestrator	Управление потоком задач между Intent и циклом реализации	Передача от Intent к Implementation, ссылка на библиотеку паттернов (Pattern Library)
Guardian Agent Engineer	Поведенческий контур управления (governance), оценки (evals), политики эскалации	Сквозной контур валидации (методика валидации ИИ-результатов), evals для эскалации, агент-защитник (Guardian Agent)
Harness Platform Engineer	Инфраструктура и среда исполнения коротких и долговременных процессов, hooks	Постоянные воркфлоу, среда исполнения
Context Supply Chain Owner	Управление контекстом, библиотекой паттернов (Pattern Library), сохраняемыми заметками	Реестр спецификаций и рабочих процессов (Spec/Workflow Registry), жизненный цикл библиотеки паттернов
AI Security Officer	Моделирование угроз агентных систем, защита от обхода ограничений модели (jailbreaking)	5 категорий угроз, пакетные согласования для долговременных процессов
AgentOps Engineer	Операционное наблюдение за агентами, отклонение поведения, жизненный цикл	ADLC, обнаружение отклонения поведения, вывод из эксплуатации

Роль	Зона ответственности	Владение
Forward-Deployed Engineer	Внедрение в команду клиента, production-решения под конкретный контекст, возврат паттернов в продукт	Контекст-специфические адаптации, обратная связь в библиотеку паттернов (Pattern Library)

Из полной ролевой карты мы разбираем детально четыре роли — **Product Engineer, Guardian Agent Engineer, Harness Platform Engineer и Context Supply Chain Owner**. На них приходится основная доля нового объёма работы ИИ-нативной команды, и они первыми материализуются при переходе. Остальные позиции таблицы либо сохраняют привычный профиль, либо производны от этих четырёх и появляются на более поздних горизонтах зрелости.

Product Engineer

Product Engineer — не «новый разработчик», а роль, собранная из составляющих продакт-менеджера, зрелого инженера и технического лида, со сквозным владением от исследования до production и верификации результата:

Кросс-функциональный профиль включает фасилитацию исследования, мышление спецификациями, валидацию ИИ-результатов, оркестрацию агентов, базовую осведомлённость в безопасности и UX. Инструменты: зрелая IDP, библиотека паттернов, методика валидации, множество агентов под оркестрацией.

Распределение времени Product Engineer в зрелой команде:

- Оркестрация агентной поставки — 40%
- Исследование и фасилитация — 30%
- Ревью и валидация — 20%
- Курирование библиотеки паттернов — 10%

Guardian Agent Engineer

Специализированная роль для зрелой команды. Владение — проектирование и поддержка системы оценок (evals) для сквозного контура валидации (способности агентов, регрессия, поведение, безопасность, эскалация), настройка агента-ревьюера, политики агента-защитника (Guardian Agent), включая контрольные точки с учётом горизонта зрелости, поведенческие оценки для агентных продуктов.

Предыдущий опыт: senior QA-инженер / SRE / инженер безопасности с грамотностью в ML.

Распределение времени:

- Проектирование и поддержка evals — 50%
- Операционный мониторинг (паттерны эскалации, частота регрессий) — 30%
- Red teaming — 20%

Harness Platform Engineer

Владелец среды исполнения агентов (Agent Runtime) и среды исполнения долговременных процессов (Long-Running Runtime). Владение включает 5 подсистем среды исполнения, idempotent tool design, профили песочниц и постоянные воркфлоу.

Предыдущий опыт: senior platform engineer / SRE / системный инженер с глубоким техническим пониманием.

Распределение времени:

- Разработка и поддержка платформы — 60%
- Реагирование на инциденты (сбои долговременных процессов) — 25%
- Экосистема инструментов (MCP-серверы, A2A-интеграции) — 15%

Context Supply Chain Owner

Context Supply Chain Owner — роль для зрелой команды (Горизонт 2+), возникающая тогда, когда организационный контекст перестаёт помещаться в голове инженера и требует системного управления. AWS на Summit New York (июнь 2026) определил этот слой напрямую: контекст — это data lake для ИИ-агентов, а функция его управления лежит на data stewards, людях, которые ведут knowledge graph организации как производственный актив. Gartner фиксирует context graphs как самостоятельный инфраструктурный примитив агентного цикла (Hype Cycle for Agentic AI, 2026).

В рамках IDP функция включает: курирование библиотеки паттернов (Pattern Library) и управление её жизненным циклом от создания до отзыва; ведение реестра спецификаций и рабочих процессов (Spec/Workflow Registry); поддержание актуальности контекста между агентными сессиями.

Предыдущий опыт: senior engineer с практикой SDD и сильными навыками технического письма, нередко эволюционирует из технического лида.

Распределение времени:

- Курирование и управление контекстом — 70%
- Внутреннее сообщество практики, обучающие материалы — 20%
- Реагирование на инциденты (отравление и устаревание контекста) — 10%

3.3 Компетенции инженера в AI-Disrupt PDLC

Мы выделяем шесть ключевых компетенций Product Engineer в зрелой ИИ-нативной команде. Отсутствие любой из них создаёт системное узкое место всей команды.

Компетенция	**Содержание**	**Антипаттерн при отсутствии**
Eval design	Проектирование оценок (evals) для проверки способностей, регрессий и эскалаций, измеряющих поведение системы, а не только выполнение функции	Команда не может отличить агентную регрессию от улучшения

Компетенция	Содержание	Антипаттерн при отсутствии
Orchestration	Декомпозиция задачи на оркестратора и рабочих агентов, управление передачей между агентами (A2A)	Агентные сессии работают последовательно там, где должны быть параллельны
Validation literacy	Умение читать и исправлять проверочный пакет (Evidence Bundle), отличать верификацию агентом-ревьюером от человеческого ревью, понимать предел доверия к агенту	Full Delegation Hope или полное недоверие к агенту с ревью каждой строки
Discovery facilitation	Умение вести PR/FAQ-сессию, формулировать гипотезу результата (Outcome Hypothesis), принимать решение о доработке существующего решения или реинжиниринге и проводить триаж на агентопригодность	Agentic-fit triage не проводится; команда берёт задачи в агент без оценки применимости
Specification thinking	Перевод нечёткого бизнес-замысла в структурированный SDD за дни, а не недели	Агентные сессии стартуют с размытой постановкой и уходят в галлюцинации
Context engineering	Управление контекстом агентной сессии: что загружать, когда сбрасывать, что сохранять в библиотеке паттернов	Контекст засоряется нерелевантными данными; модель теряет ориентацию в задаче

Матрица компетенций по уровням

Компетенции приведённой таблицы не статичны — их глубина и практическое применение принципиально различаются от junior до staff. Ниже расшифровка ожиданий по уровням и методы проверки. Ниже — детализация инструментальных навыков, лежащих в основе каждой компетенции.

Компетенция	Junior	Senior	Как проверять
Prompt engineering	Пишет базовые промпты с ролью и задачей; понимает, что системный промпт влияет на поведение модели	Проектирует контекстные промпты для длинных сессий; управляет бюджетом токенов; переключается между SDD с разработчиком в центре и SDD с агентом в центре под горизонт задачи	Разбор трёх реальных промптов из библиотеки паттернов; кандидат объясняет выбор структуры и антипаттерны

Компетенция	Junior	Senior	Как проверять
Оценка LLM-выходов (output evaluation)	Замечает явные галлюцинации в ответе; сигнализирует об ошибках в PR-ревью	Проводит систематическую eval-сессию: golden set, регрессионные тесты, поведенческие оценки; различает «ошибку модели» и «ошибку промпта»	Дать кандидату набор агентных выходов; попросить категоризировать дефекты и предложить eval-критерий
Контроль галлюцинаций (hallucination control)	Знает, что галлюцинации существуют; проверяет факты вручную	Проектирует механизмы обеспечения достоверности: RAG, reference injection, fact-check hooks; понимает связь между длиной контекстного окна и точностью	Код-ревью агентной сессии: кандидат указывает точки риска галлюцинаций и предлагает митигацию
Дизайн контекстных промптов	Использует шаблоны из библиотеки паттернов	Самостоятельно проектирует контекстный промпт: задача, ограничения, формат вывода, примеры, инварианты; включает ожидания на основе свойств (property-based)	Написать контекстный промпт «с нуля» для заданного SDD под наблюдением
Работа с harness (среда работы ИИ-агентов)	Запускает агентную сессию в готовой среде работы ИИ-агентов (harness) по инструкции	Настраивает хуки (hooks), политики, профили изоляции; отлаживает сбои среды исполнения; понимает идемпотентность инструментов	Диагностика сломанного воркфлоу: кандидат изолирует проблему в harness за 20 минут
Разработка, управляемая оценками (Evaluation-driven development, EDD)	Понимает концепцию «сначала eval, потом реализация»	Ведёт разработку через оценки: пишет eval до SDD, затем SDD, затем итерирует реализацию до прохождения оценки (eval)	Показать историю коммитов с доказательством EDD-цикла; обосновать выбор eval-критерия
Написание spec/intent-документов	Пишет SDD под ревью senior; использует шаблон	Пишет SDD с разработчиком в центре и с агентом в центре самостоятельно; выбирает между ними по горизонту задачи; обучает junior	Публичное ревью трёх SDD; оценка по измеримости гипотезы, полноте инвариантов, качеству примеров

Компетенция	Junior	Senior	Как проверять
Понимание контекстных окон	Знает, что у модели есть лимит токенов	Управляет «гигиеной токенов»: знает, когда сбрасывать сессию, что переносить в долгосрочную память агента, как структурировать SDD под целевую длину окна	Задача на сокращение и реструктуризацию длинного контекста; кандидат обосновывает каждое удаление

Антипаттерн: vibe coding без оценок

Vibe coding — практика, при которой инженер принимает выход агента на ощущение: «выглядит правильно». На одиночных задачах горизонта до 15 минут в изолированной кодовой базе это допустимо. На масштабе команды и горизонтах 1–4 часа — это системный риск. Без evals команда не может отличить регрессию от улучшения. Галлюцинация, не пойманная в момент выполнения, попадает в библиотеку паттернов и воспроизводится в каждой последующей сессии.

Диагностика проста:

- В библиотеке паттернов нет критериев оценки
- Agent-introduced Regression Rate не измеряется
- На вопрос «почему этот PR одобрен?» ответ — «агент написал, выглядит нормально»

Если хотя бы один из этих признаков верен, команда практикует vibe coding — какой бы горизонт зрелости она ни декларировала.

Карьерный трек: junior → senior → staff

Сдвиг компетенций на каждом карьерном уровне не количественный, а качественный: меняется не набор навыков, а уровень автономии, горизонт ответственности и направленность деятельности.

- **Junior ИИ-нативный инженер (0–18 месяцев).** Зона ответственности — задачи горизонта до 30 минут в рамках утверждённых SDD. Основная деятельность — работает в паре с senior над постановкой, участвует в eval-сессиях как наблюдатель, пишет первые контекстные промпты под ревью. Метрика успеха — стабильность SDD, то есть доля задач без эскалации к senior. Ключевой переход: junior перестаёт нуждаться в ревью каждого SDD.
- **Senior ИИ-нативный инженер (18 месяцев — 4 года).** Зона ответственности — сквозной цикл поставки от исследования до проверки после развёртывания, наставничество для 1-2 junior. Основная деятельность — самостоятельно пишет developer-first и agent-first SDD, проектирует eval-наборы для своего домена, управляет балансом между скоростью реализации и скоростью валидации. Метрика успеха — доля подтверждённых результатов (Outcome Validation Rate) ≥85% и вклад ≥5 сертифицированных паттернов в квартал. Ключевой переход: senior начинает влиять на архитектуру среды работы ИИ-агентов (harness).
- **Staff / Principal ИИ-нативный инженер (4+ года).** Зона ответственности — кросс-командная стратегия компетенций, проектирование среды работы ИИ-агентов как платформенного актива, образовательные маршруты для организации. Основная деятельность — проектирует agent-first SDD для сложных кросс-системных задач, определяет стандарты evals на уровне платформы, формирует сообщество практики по agentops. Метрика успеха — не индивидуальные показатели, а зрелость команд, прошедших наставничество. Adoption rate стандартов harness в кросс-командных проектах.

На каждом уровне сохраняется одно инвариантное требование: ни один инженер, независимо от уровня, не принимает агентный вывод без прохождения eval-критерия. Это операционный механизм, удерживающий команду от деградации качества.

Specification thinking как первичная компетенция

Specification thinking — умение перевести нечёткий бизнес-замысел в структурированный SDD за дни, а не недели. Компетенция включает четыре составляющих: выявление требований, формулировку гипотезы результата (Outcome Hypothesis) с измеримой метрикой, выделение инвариантов системы и написание property-based тестов (проверка инвариантов).

Развивается только через практику: junior работает в паре с senior над 5–10 SDD, затем ведёт свой первый SDD под ревью. Тренинги эту компетенцию не формируют.

Validation literacy как калиброванное доверие

Validation literacy — калиброванное доверие к агенту, основанное на метриках, а не на ощущении:

При отсутствии компетенции команда впадает в одну из двух крайностей: полностью полагается на агента и не читает проверочные свидетельства (антипаттерн Full Delegation Hope) — или повторно проверяет каждую строку кода, теряя весь выигрыш в скорости. Верные ориентиры — I/V Tempo Ratio, Agent-introduced Regression Rate и частота

галлюцинаций при заданном горизонте задачи. Компетенция измерима и развивается целенаправленно.

Discovery facilitation и контекст-инжиниринг

Discovery facilitation требует понимания, когда агент применим, а когда создаёт ложную уверенность. **Триаж на агентопригодность (agentic-fit triage)** — навык, который team lead демонстрирует явно в начале каждого спринта.

Контекст-инжиниринг (context engineering) — управление тем, что попадает в агентную сессию. Перегрузка правил делает их декоративными: модель с конкурирующими директивами в контексте следует им хуже, чем модель с минимальным, но точным набором инструкций. Управление контекстом — это не накопление правил, а их приоритизация.

3.4 Tiny teams: модель компактных команд

По прогнозу аналитиков, к 2030 году 80% организаций перейдут от больших команд разработки ПО к небольшим командам, усиленным ИИ; это сценарий, а не гарантия:

Tiny Team — команда из 3–6 человек, которая владеет всеми шестью компетенциями и оркестрирует агентов вместо ручного написания большей части кода. Одна Tiny Team берёт на себя объём работы, который раньше выполняла команда из 15–20 человек, и сохраняет при этом сквозное кросс-функциональное владение.

Условия эффективности Tiny Teams

Для работы Tiny Team необходимы четыре условия:

- Зрелая IDP со средой исполнения долговременных процессов (Long-Running Runtime)
- Библиотека паттернов (Pattern Library) с ≥30 сертифицированными паттернами
- Методика валидации ИИ-результатов, не требующая ручного ревью каждого изменения
- Работающий ADLC

При отсутствии любого из них команда получает не Tiny Team, а неполную команду с высоким агентным риском без системы его смягчения.

Трансформация в Tiny Teams в реальной организации

Стартовая позиция для типичного enterprise — команды по 15–20 человек, разбитые по функциям: разработка, QA, SRE, security. Переход занимает 6–12 месяцев. За это время функциональные специалисты развиваются в product engineers, Tiny Teams формируются со сквозным владением, платформенная команда из 3–5 человек берёт на себя поддержку 10+ продуктовых команд.

Не все инженеры переходят в Tiny Teams — часть остаётся специалистами в архитектуре, security или ML, но работает в смешанных командах, а не в параллельной иерархии.

Tiny Team — операционная картина

- **Состав:** минимум 2 senior product engineers, 1–2 mid-level-инженера, 1 junior в практическом наставничестве
- **Рабочие параметры:** бэклог 10–15 пользовательских историй в активной работе, дорожная карта на 2 квартала
- **Отчётность:** напрямую инженерному менеджеру, без среднего управленческого слоя

Метрика	**Что измеряет**
Outcome Validation Rate	Доля подтверждённых результатов
Spec Reuse Rate	Доля повторного использования спецификаций
Throughput/ Output Volume Lift	Пропускная способность поставки
Agent-introduced Regression Rate	Частота регрессий, внесённых агентами
I/V Tempo Ratio	Соотношение темпа реализации и темпа валидации

Различие паттернов внутри Tiny Team

Внутри одной Tiny Team оба паттерна сосуществуют — выбор зависит от горизонта задачи. По данным Gartner (Hype Cycle for Platform Engineering, 2026): на коротких задачах (до 30 минут), разработчик сохраняет контроль структуры, используя агента как ускоритель (developer-first); на задачах горизонта 2–4 часа контроль структуры делегируется агенту, разработчик управляет постановкой и проверкой результата (agent-first). По данным Gartner AX, умение переключаться между паттернами — одна из ключевых компетенций зрелого оркестратора.

Типичные препятствия при формировании Tiny Teams:

№	**Препятствие**	**Суть сопротивления**
1	Функциональная идентичность	QA-инженер не хочет становиться product engineer
2	Страх потери специализации	Ощущение, что широта убьёт глубину
3	Отсутствие IDP	Команда не может работать автономно без платформенной поддержки
4	Неготовая библиотека паттернов	Агенты без паттернов генерируют нестандартный код
5	Отсутствие методики валидации	Tiny Team не может принимать агентный вывод без ручного ревью
6	Метрики производительности (performance)	Системы оценки не различают оркестратора и исполнителя

№	**Препятствие**	**Суть сопротивления**
7	Кросс-командные зависимости	Tiny Team блокируется внешними командами с длинными циклами согласования
8	Доверие бизнеса	Стейкхолдеры не готовы принять результат без традиционного QA-процесса
9	Скорость наставничества	Нехватка senior engineers ограничивает темп перехода
10	Технический долг	Кодовая база, не адаптированная к агентному рефакторингу
11	Руководство не выделяет ресурсов на переход к Tiny Team	Tiny Team блокируется внешними командами с длинными циклами согласования

Tiny Team становится единицей поставки только при наличии платформы и методики валидации. Без них компактность оборачивается концентрацией риска.

Связка IDP, командной зрелости и валидации

Каждый элемент технологии связан с конкретной организационной способностью. Способность ADLC предполагает наличие AgentOps Engineer — без владельца ADLC превращается в инструмент на полке. Механизм оценки стоимости результата (Outcome Pricing Engine, OPE) предполагает наличие FinOps-практики и стейкхолдеров, смотрящих на CTOR — без этого механизм генерирует дашборды, на которые никто не смотрит.

Координация платформенной и продуктовых команд

Платформенная команда (8–12 человек на 100 инженеров) владеет инфраструктурой IDP, библиотекой паттернов, методикой валидации ИИ-результатов, средой исполнения агентов и ADLC. Продуктовые команды владеют SDD для своих доменов, ритуалами исследования (Discovery) и владением результатом (outcome ownership).

Координацию обеспечивают три регулярных ритма:

- Еженедельный платформенный форум для продуктовых инженеров
- Ежемесячный Платформенный совет по дорожной карте
- Ежеквартальная общекомандная встреча с ревью метрик

Типичный антипаттерн — платформенная команда превращается в башню из слоновой кости, а продуктовые уходят в теневую разработку. Смягчает риск ежеквартальное ротационное встраивание участника платформы в продуктовую команду.

Образовательный маршрут нового инженера:

Период	**Активность**
Дни 0–7	Онбординг в IDP, ритуалы исследования (Discovery), базовое написание SDD под наставником

Период	Активность
Дни 8–30	Первые агентные сессии под ревью senior engineer; чтение существующих SDD
Месяц 2–3	Самостоятельный SDD под ревью; участие в eval-сессиях Guardian Agent Engineer
Месяц 4–6	Первый полный цикл поставки: Discovery → SDD → агентная реализация → валидация → post-deploy check
Месяц 7–12	Первый вклад в библиотеку паттернов; начало наставничества следующего junior

Производительность нового инженера измеряется не строками кода, а стабильностью SDD.

Ключевые показатели:

- Процент соответствия результата первоначально отправленным SDD
- Outcome Validation Rate в собственных задачах
- Вклад в библиотеку паттернов
- Согласие агента-ревьюера
- Наставничество младших инженеров

Это сдвиг от метрик выработки к метрикам результата.

3.5 Влияние на рынок труда и навыки

По прогнозу аналитиков, к 2028 году 90% инженеров сместятся от написания кода к оркестрации ИИ-агентов (Gartner Software Engineering, 2025). Это не означает исчезновения специализаций — security, SRE, ML, data — но радикально меняет профиль базового инженера. Новый инженер стартует как full-stack engineer и ИИ-оркестратор одновременно. ИИ-генерация и агент-ревьюер берут на себя большую часть junior-задач.

По данным McKinsey State of AI 2025, AI high performers в три раза чаще перестраивают рабочие процессы вокруг AI и в три раза чаще обеспечивают сильную вовлечённость senior leadership — по сравнению с остальными организациями. Программы практического наставничества и ролевые образовательные маршруты становятся обязательными.
Антипаттерн: «ИИ без курируемых ролевых образовательных маршрутов».

Формы и размеры компенсаций

Рынок компенсаций AI-native инженеров формируется быстрее, чем традиционные зарплатные диапазоны успевают обновляться. Приведённые ниже диапазоны — рыночные ориентиры по состоянию на 2025–2026 год на основе данных Levels.fyi, MRJ Recruitment и hh.ru; они отражают структуру спроса в ведущих технологических компаниях, а не конкретные данные одного работодателя. Все цифры ориентировочные и зависят от отрасли, компании и географии.

Уровень	США: base salary	США: total comp (base + equity)	РФ: базовая ставка (RUB/мес)	РФ: total comp
Junior AI-native engineer	$95–130k	$110–175k	200–350k	230–420k + RSU/опционы
Mid-level AI-native engineer	$130–200k	$180–280k	350–550k	420–680k + RSU/опционы
Senior AI-native engineer	$180–280k	$250–450k	650–850k	780k–1.1M + RSU/опционы
Staff / Principal AI-native engineer	$250–400k	$400–950k	900k–1.2M	1.1–1.5M + RSU/опционы

Данные по РФ — ориентиры ведущих российских технологических компаний (Сбер, Яндекс, Т-Банк). Источники: hh.ru / IT Institute Q2 2026, Ведомости / hh.ru, январь 2026. США: Levels.fyi Q3 2025, MRJ Recruitment / Acceler8 Talent 2026.

Product engineer, оркестрирующий агентов на объёме 3–5 традиционных разработчиков, получает AI-премию к компенсации: по данным PwC Global AI Jobs Barometer 2026, AI-навыки добавляют в среднем 62% к зарплате относительно коллег без них на той же позиции — премия удвоилась за два года. Для mid-level и senior инженеров она составляет от 22 до 45% соответственно. При этом организация сокращает затраты на численность команды на 40–60% — McKinsey фиксирует сопоставимое сжатие при AI-native переходе. Это создаёт операционную логику: платить за владение результатом (outcome ownership), а не за часы. **Организации, не перестроившие диапазоны к 2026–2027 году, теряют AI-native инженеров в пользу рынка, готового платить за компетенцию, а не за грейд.**

Сжатие пирамиды: junior: senior → 1:1

В традиционных командах соотношение junior:senior составляет 3:1 и выше. Агентная автоматизация берёт на себя большую часть задач junior-уровня; junior-инженер в AI-native команде нужен не как исполнитель тикетов, а как кандидат в наставничество и будущий senior. Это принципиально меняет воронку найма.

Сжатие уже происходит — и быстрее, чем прогнозировалось. По данным Wobo Software Hiring Report 2026, ведущие технологические компании размещают в среднем 14 senior-позиций на каждую junior:

- Netflix — 47:1
- Anthropic — 24:1
- OpenAI — 19:1

В AI-native командах 2026 года доля junior не превышает 10–15% от состава против 40%+ в командах до 2023 года. Gartner предупреждает, что к 2028 году prompt-to-app подходы без governance увеличат число дефектов на 2500% — Tiny Team с зрелой методикой валидации это прямой ответ на этот риск. Организации, продолжающие строить пирамиду 3:1, накапливают избыточный junior-слой, который агент уже замещает, и одновременно не инвестируют в senior-компетенции, от которых зависит качество поставки. Рациональная стратегия — нанимать меньше junior и вкладывать в наставничество каждого.

Воронка талантов: откуда приходят AI-native инженеры

Устойчивые маршруты входа в профессию AI-native engineer в 2025–2026 году:

Исходная траектория	**Переходный навык**	**Типичный период адаптации**
Data science / ML engineer	Освоение SDD и harness; переход от эксперимента к production-поставке	6–12 месяцев
Backend / full-stack engineer	Освоение prompt engineering, eval design и validation literacy	9–18 месяцев
QA / SDET engineer	Переориентация eval-навыка с unit-тестов на поведенческие evals; освоение SDD	6–12 месяцев
Новый выпускник (AI-native программа)	Сразу входит с specification thinking и EDD; требует наставничества по production-контексту	12–18 месяцев до самостоятельности

Самый короткий путь — QA-инженер с грамотностью в ML

- Нативная компетенция: eval design
- Главный пробел: specification thinking и harness

Самый долгий путь — традиционный backend-инженер без ML-бэкграунда

- Главный пробел: нет опыта работы с вероятностными выходами

Образовательные маршруты и сертификации

Формальное образование ещё не успело переориентироваться на AI-native инженерию, однако ряд программ уже предоставляет необходимую основу.

Международные программы:

Программа	**Фокус**	**Формат**
Stanford CS324 «Large Language Models»	Архитектура LLM, безопасность применения	Очно / онлайн, семестр
CMU LLM Systems(курсы по агентным системам)	Системное проектирование агентных пайплайнов	Очно / онлайн
DeepLearning.AI специализации (курсы по prompt engineering, RAG, агентам)	Практические навыки prompt engineering и eval design	Онлайн, самостоятельно
Anthropic / OpenAI Developer courses	Применение конкретных API и безопасные практики	Онлайн, бесплатно / платно

Российские программы:

Программа	Фокус	Формат
ВШЭ: магистратура «Машинное обучение и приложения»	ML-основа + практика поставки	Очно, Москва
МФТИ: кафедра «Применения ИИ»	Формальные основы LLM, системное мышление	Очно, МФТИ
Школа Анализа Данных Яндекса (ШАД)	Глубокая ML-база + production-инженерия	Очно / гибрид, Москва / онлайн
Сбер AI Academy	Практика AI-native разработки в enterprise-контексте	Корпоративный формат

Ни одна из программ не даёт полного AI-native профиля «из коробки». Наиболее жизнеспособная стратегия — формальная программа (ML/CS основа) плюс 12–18 месяцев практического наставничества в AI-native команде. Bootcamp-модель «3–6 месяцев до junior» для AI-native профиля не работает — входной уровень требует specification thinking и validation literacy, которые за короткий курс не передаются.

Новые точки входа в профессию

Простые тикеты теперь автоматизированы агентами. Новая точка входа складывается из четырёх видов работы:

- Написание SDD
- Разработка evals
- Ревью проверочного пакета (Evidence Bundle)
- Курирование библиотеки паттернов

Это требует другого обучения — не “научи синтаксису Python”, а “научи задавать вопросы, читать спецификации, интерпретировать сбои evals”. Университетские программы и корпоративные академии перестраивают маршруты вокруг мышления спецификациями (specification thinking) и грамотности в валидации (validation literacy).

Следствия для воронки талантов

Университеты, продолжающие готовить junior software engineer как человека, пишущего функции по тикетам, выпускают неконкурентоспособных специалистов. **Перестройка учебного маршрута опирается на три шага:**

- Мышление спецификациями — на 1–2 курсе
- Eval design и грамотность в валидации — на 3–4 курсе
- Итоговый проект — с агентами как исполнителями и студентом как оркестратором

Буткемпы переходят от “собери CRUD-приложение” к “специфицируй, оркестрируй и валидируй агентную поставку”.

Оговорка: приведённые прогнозы (80%, 50%, 3×) — направленные сигналы, зависящие от отрасли и зрелости организации, а не детерминированные утверждения.

Наставничество как замена bootcamp-модели

Bootcamp-модель — 3–6 месяцев интенсивного обучения — разрабатывалась для эпохи, когда входной уровень требовал junior-навыков написания кода. В ИИ-эпоху входной уровень требует full-stack-мышления, мышления спецификациями и умения работать с валидацией; это не передаётся за 3–6 месяцев. Практическое наставничество — год под руководством senior-инженера — даёт нужную глубину. Организации, инвестирующие в наставничество сегодня, получают конкурентное преимущество через 12–18 месяцев; рынок готовых product engineers остаётся ограниченным.

3.6 Уровни зрелости команды L0–L5

Обозначение L0–L5 в данном разделе относится к зрелости команды. Три разные оси:

- L (task horizon) — горизонт автономии агента на задаче
- L (organizational maturity) — уровень зрелости команды
- R (permissions) — разрешения агента в контуре управления

Смешение осей — типичная ошибка при самооценке зрелости.

Уровни зрелости — инструмент планирования, а не знак отличия. Шесть уровней, от L0 (отсутствие ИИ-практики) до L5 (скоординированные команды агентов). Каждый уровень имеет собственный профиль задач, требования к валидации и governance-критерии.

Профили уровней: технические характеристики

Уровень	Состояние	Медианный горизонт задачи	Валидация
L0	Нет устойчивой ИИ-практики	Нет агентных задач	Нет требований
L1	Эксперименты с IDE-плагинами	Минуты (автодополнение)	Не требуется
L2	Первые инженеры используют ИИ для рутинных задач	До 30 минут	AI-gate в CI
L3	SDD внедрён; агенты кодирования работают в ветках под ревью	До 2 часов	Агент-ревьюер + проверочный пакет (Evidence Bundle)
L4	Параллельное выполнение, мульти-сессии	До 4 часов; на зрелом уровне до 1 рабочего дня	Интегрированная методика валидации ИИ-результатов + контрольные точки, учитывающие горизонт задачи

Уровень	Состояние	Медианный горизонт задачи	Валидация
L5	Скоординированные команды агентов (Coordinated Agent Teams)	Длительные задачи (несколько дней, цепочка сессий с контрольными точками)	Объяснимость в реальном времени + полный жизненный цикл ADLC

Внешняя калибровка зрелости и оценка как инструмент планирования

Внешняя калибровка через архитектурные признаки Bain (State of the Art of Agentic AI Transformation, 2025):

- PDLC L1–L2 ≈ Bain L1 (copilots, информационный поиск)
- PDLC L3 ≈ Bain L2 (однозадачный режим)
- PDLC L4 ≈ Bain L3 (межсистемный режим)
- PDLC L5 ≈ Bain L4 (слабосвязная многоагентная конфигурация)

Медианный горизонт задачи — самый честный прокси зрелости. Честная оценка важнее «подталкивания» к высоким уровням: команда на L2 с реалистичной дорожной картой к L3 за 6 месяцев находится в лучшей позиции, чем команда, объявившая L4, но операционно работающая на L2. Validation literacy включает честную самооценку.

Gate-критерии переходов между уровнями

Переход	Gate-критерии	Метрические критерии
L0 → L1	Доступ к IDE-плагину и базовая гигиена промптов	Доля сотрудников, использующих агентный режим работы
L1 → L2	SDD-lite, AI-gate в CI, первые evals	Старт замера проявления навыков Agentic Coding
L2 → L3	Обязательный SDD, агент-ревьюер, проверочный пакет (Evidence Bundle), отправка паттернов	≥ 5 полных SDD; согласие агента-ревьюера ≥ 80%; полнота Evidence Bundle подтверждена
L3 → L4	Среда исполнения долговременных процессов (Long-Running Runtime), библиотека паттернов с ≥30 сертифицированными паттернами, методика валидации ИИ-результатов, ADLC	≥ 30 сертифицированных паттернов; I/V Tempo Ratio стабильно; нет узкого места в ручном ревью
L4 → L5	Протокол A2A, скоординированные команды агентов, объяснимость в реальном времени	Протокол A2A внедрён; ADLC работает операционно; объяснимость в реальном времени активна

Правильная траектория развития зрелости:

- L0→L1 — дни (доступ к инструменту, базовая гигиена промптов)
- L1→L2 — 1–3 месяца (SDD-lite, AI-gate в CI, первые evals)
- L2→L3 — 6–12 месяцев
- L3→L4 — 12–18 месяцев
- L5 — только для организаций с платформенной командой и работающим контуром управления

Погоня за уровнями без фундамента опаснее честного L2: команда получает регрессии, теряет доверие и откатывается к L1.

Governance-критерии для L0–L5

Уровень	**Governance**	**Цель**	**Пример метрики**
L0	Нет AI-governance	Не требуется	N/A
L1	Политика инструментов (Tool policy)	Разрешённые IDE-плагины	Доля внедрения инструментов (Tool adoption rate)
L2	AI-gate в CI	Блокировать критичные и высокоприоритетные регрессии (P0/P1)	Доля пропущенных регрессий(Regression escape rate)
L3	Частичная методика валидации ИИ-результатов	Проверочный пакет (Evidence Bundle), агент-ревьюер	Полнота проверочного пакета (EB completeness), доля согласия (agreement rate)
L4	Полная методика валидации ИИ-результатов + ADLC	Мониторинг среды исполнения, откат	I/V Tempo Ratio, частота регрессий (regression rate)
L5	Объяснимость в реальном времени + жизненный цикл ADLC	Непрерывный контур управления (Continuous governance)	Время до объяснения (Time-to-explain), управление затратами (cost governance)

Политика эскалации в контуре управления по уровням

Уровень	Триггер эскалации
L1	Безопасность и приватность
L2	Критичные и высокоприоритетные регрессии (P0/P1)
L3	Несогласие агента-ревьюера >20%, неполнота Evidence Bundle

Уровень	Триггер эскалации
L4	Отклонение поведения, аномалии затрат
L5	Конфликт между агентами, одобрение автономного отката

Контур управления (Governance Mesh) масштабируется вместе с уровнем зрелости — не опережает его и не запаздывает. Опережающий governance создаёт бюрократический тормоз; запаздывающий — открывает окна риска.

Часть 3 ответила на вопрос «кто»: кто управляет агентами, какие компетенции для этого нужны, как выглядит карьерная траектория и что отличает зрелую команду от команды с инструментами.

Три вывода, которые определяют следующие решения:

- Роль инженера сместилась от исполнителя к оркестратору — это не тренд, а операциональная реальность уже на L3.
- Tiny Team работает только при наличии платформы. Компетенции без среды исполнения — незавершённая система.
- Уровень зрелости — не декларация, а измеримое состояние. Gate-критерии — инструмент честной самооценки, а не формальность.

Следующий вопрос — платформа. Что должна обеспечивать IDP, чтобы командная модель стала операционно возможной.

Часть 4. Инфраструктурная платформа разработки (IDP)

4.1 Integrated Development Platform (IDP)

Integrated Development Platform (IDP) — единая операционная среда «золотого пути» от исследования до промышленной эксплуатации. Набор лицензий на отдельные ИИ-инструменты не становится платформой: спецификации, агенты, политики и аудит остаются несвязанными, а ответственность за сквозной результат ни за кем не закреплена. IDP связывает эти элементы в один контур с общими интерфейсами и единым владением.

В одном контуре IDP объединяет спецификации, агентов и их среду исполнения, хуки политик, системы оценки, слой наблюдаемости, неизменяемый журнал аудита и механизмы токеномики. По оценке Bain, подобные решения выполняют роль «операционных систем для агентов» — формулировка, которая точно отражает наш архитектурный выбор.

Платформа поддерживает оба контура разработки — классические системы и агентные системы как продукт. Это означает единую среду исполнения агентов, единую риск-адаптивную лестницу разрешений, единый контур валидации (validation) и единую библиотеку шаблонов спецификаций — при различных конфигурациях для двух типов продуктов.

IDP является капитальным активом. Начальная стоимость разработки — 12–18 месяцев для команды из 3–5 инженеров платформы и одного менеджера платформы. После запуска операционные затраты составляют 30–50% от первоначальных вложений.

Ценность платформы раскрывается через масштаб использования. Каждая продуктовая команда, работающая на IDP, демонстрирует рост производительности на 30–50%. Для организации с десятью и более продуктовыми командами инвестиции окупаются за 12–24 месяца.

Без собственной IDP организация платит «налог на поставщика» — помесячное лицензирование ИИ-инструментов по числу мест для каждого инженера. Годовая сумма для организации из 100 человек достигает $1–3 млн. За аналогичный бюджет собственная IDP обслуживает все команды без нарастающей стоимости per-seat. Разрыв между «купить» и «построить» начинает сходиться на горизонте 18–24 месяцев и становится устойчивым через 36 месяцев эксплуатации.

Архитектура вокруг среды агентов

Архитектура IDP организуется вокруг одного центрального вопроса: как устроена среда работы ИИ-агентов. Остальные компоненты выстраиваются под её требования и сценарии. Воплощение архитектуры требует разнородных навыков — инженерных, продуктовых, операционных, governance, — и мы собираем их распределённо по всей организации, не концентрируя в одной команде.

Платформа держится на опорах, разделённых организационно по владению. Каждая опора описывает, что она вносит в работу платформы, а не занимает место в иерархии.

Компонент	Состав	Класс работы	Владелец
Оркестрация и среда исполнения агентов	Исполнитель в песочнице, механизм политик, среда исполнения долговременных процессов, слой объяснимости	Технический	Инженер платформы среды работы ИИ-агентов
Контекст и спецификации	Библиотека шаблонов спецификаций, каталог переиспользуемых рабочих процессов, реестр спецификаций и рабочих процессов, цепочка поставки контекста	Полутехнический	Владелец цепочки поставки контекста
Качество и валидация	Реестр оценочных тестов, контур валидации, агенты-защитники, агент проверки, контрольные точки безопасности	Полутехнический	Инженер агентов-защитников
Эксплуатация и управление жизненным циклом	ModelOps, AgentOps, AgentLifecycleOps	Операционный	Инженер эксплуатации агентов
Управление, безопасность, управление финансами	Фреймворк хуков политик, федеративные хуки, механизм ценообразования, механизмы обеспечения соответствия требованиям	Governance	Офицер governance-платформы

Компоненты разделены организационно, а не технически. Каждый компонент имеет команду-владельца, собственные метрики и дорожную карту. Ни один компонент не подчинён другому — они связаны через контрактные интерфейсы, без иерархии.

Существующие решения

По данным ведущих технологических консультантов, зрелая архитектура ИИ-нативной разработки охватывает десять классов инструментов, образующих сквозную цепочку от замысла до эксплуатации.

Мы рассматриваем эту карту как внешний эталон для калибровки полноты нашей IDP:

No	Компонент	Назначение	Класс владельца
1	Агентные IDE	Среда разработчика со встроенным ИИ-контекстом и агентными возможностями	Команда реализации
2	CLI-агенты SWE	Терминальные ИИ-агенты для автономной реализации задач	Платформенная команда
3	Фоновые SWE-агенты	Событийно-триггерные агенты: запускаются по событию, а не по команде	Платформенная команда
4	Системы контекста	Репозитории, требования, чаты, документы, дизайн, графы знаний, IDP	Владелец цепочки поставки контекста

No	Компонент	Назначение	Класс владельца
5	Governance агентов	Управление расходами (FinOps), правила-ограничители (guardrails), управление доступом, ролевые политики	Офицер governance
6	ИИ-планирование agile	Планирование спринтов, распределение задач, уточнение требований	Команда реализации
7	ИИ-тест-менеджмент	Генерация тестов, данных, покрытие, селекция сценариев	Инженер агентов-защитников
8	ИИ-генерация приложений	Прототипы и командные инструменты для быстрого исследования (после вайб-кодинга)	Команда реализации
9	Детерминированные SWE-инструменты	Линтеры, статический анализ, SCA, e2e-тесты	Платформенная команда
10	ИИ SRE	Подавление шума инцидентов, корневой анализ, авторемедиация	Инженер эксплуатации агентов

Карта рынка дополняет нашу архитектуру и служит внешней проверкой полноты. Она помогает убедиться, что ни один класс инструментов не выпал из зоны ответственности платформы и что каждая внешняя категория рынка имеет внутреннего владельца.

Пятиуровневый технический стек IDP

Технический стек IDP организован по уровням, каждый со своим назначением, набором технологий и владельцем. Разделение по владению позволяет развивать уровни независимо, связывая их контрактными интерфейсами.

Уровень	Назначение	Технологии	Владелец
1. Инфраструктура	Вычисления, хранение, сеть, секреты, профили песочниц	Kubernetes с custom-операторами; Vault; управляемые PostgreSQL/Redis; S3 с блокировкой объектов	DevOps / SRE
2. Данные и контекст	Граф знаний, векторное хранилище, сохранённые заметки, библиотека паттернов	Neo4j или аналог; Qdrant/Weaviate; Redis для горячего контекста; Git-репозиторий паттернов	Владелец цепочки поставки контекста
3. Опыт разработчика	CLI, расширения IDE, поверхности чата, дашборды мониторинга	Cursor, Claude Code; расширения VS Code; Slack/Teams AG-UI; Grafana	Команда реализации
4. Слой ИИ	Шлюз моделей, ИИ-шлюз, серверы MCP, брокер A2A	ИИ-шлюз (LiteLLM или аналог); реестр MCP-серверов; координатор A2A	Инженер платформы

Уровень	Назначение	Технологии	Владелец
5a. Интерфейсы результатов	UX агента в Slack/Teams, AG-UI, поверхности конечного пользователя	Стандарт AG-UI; адаптеры каналов	Команда реализации / дизайн
5b. Бизнес-логика	Движок SDD, контрольные точки качества, контур валидации	PDLC engine; eval runner; Guardian Agent	Инженер агентов-защитников

Уровень 1 — инфраструктура. Общий кластер Kubernetes с пространством имён на проект, секреты через Vault, постоянство через управляемые PostgreSQL/Redis. Оператор управляет сессиями агентов: создание пода, выбор профиля песочницы, квоты ресурсов, очистка при завершении. Журнал аудита ведётся в JSONL с ежедневной ротацией и архивацией в S3 с блокировкой объектов. Для среды исполнения заданы RTO=1 ч и RPO=5 мин.

Уровень 2 — данные и контекст. Граф знаний (Neo4j или аналог) хранит структурированные отношения между артефактами PDLC; векторное хранилище (Qdrant/Weaviate) обеспечивает семантический поиск по спецификациям и паттернам; Redis держит горячий контекст активных сессий. Библиотека паттернов лежит в Git-репозитории с версионированием и процессом ревью, а цепочка поставки контекста доставляет релевантное знание на старте каждой сессии.

Уровень 3 — опыт разработчика. CLI и расширения IDE (Cursor, Claude Code) обеспечивают паттерн с разработчиком в центре (developer-first), поверхности чата в Slack/Teams — паттерн с агентом в центре (agent-first) для долгоработающих задач. Дашборды Grafana отображают метрики PDLC — индекс зрелости, отношение темпа реализации к темпу валидации, отношение стоимости к результату. Оповещения делятся на критичные (немедленный вызов), высокие (уведомление дежурного инженера) и средние (очередь следующего рабочего дня).

Уровень 4 — слой ИИ. ИИ-шлюз (AI Gateway) абстрагирует нескольких провайдеров моделей и позволяет менять их без переписывания агентов; LiteLLM или аналог даёт единый интерфейс. Реестр MCP-серверов хранит метаданные каждого инструмента — назначение, схема входных и выходных данных, требования к профилю песочницы. Брокер A2A координирует передачу фазы между агентами в многоагентных процессах; нативная поддержка контекста используется там, где доступна, MCP — как второй вариант для внешних инструментов.

Уровень 5a — интерфейсы результатов. AG-UI стандартизирует взаимодействие человека с агентом: структурированный формат хода работы, запросы действий, запросы одобрения, отображение объяснений. Адаптеры каналов приводят AG-UI к конкретным поверхностям (Slack, Teams, web UI). Уровень применяется для агентных продуктов; агенты доставки классических систем остаются в IDE, CLI и headless-режиме без AG-UI.

Уровень 5b — бизнес-логика. PDLC engine управляет переходами между состояниями задачи по золотому пути, eval runner запускает оценочные тесты и публикует результаты в реестр, Guardian Agent реализует поведенческое управление — мониторинг дрейфа, политики эскалации, автоматический откат при нарушении. Контур валидации остаётся сквозным слоем, встроенным в обе петли двухпетлевой модели.

Уровни 5a и 5b разделены намеренно: первый отвечает за взаимодействие с человеком, второй — за исполнение логики PDLC. Смешение приводит к тому, что изменения в UX вызывают побочные эффекты в бизнес-логике и наоборот.

Именно разделение слоёв по владению даёт платформе свойство, недоступное набору инструментов: каждый уровень развивается своей командой и своей дорожной картой, а связность поддерживают контрактные интерфейсы. Это и превращает IDP в капитальный актив, который окупается на горизонте 12–24 месяцев и обслуживает все команды без нарастающей стоимости «за место».

4.2 Среда важнее модели

Результат работы ИИ-агента определяет не модель, а среда её исполнения (harness) — детерминированная оболочка над стохастическим ядром.

Среда работы ИИ-агентов выступает посредником между внешним миром — человеком, другими системами — и внутренней средой агента, самой моделью. По формулировке Bain, «операционные системы для агентов располагаются в среднем слое, оркестрируя фактическую работу: они планируют задачи, сохраняют контекст и вызывают соответствующие инструменты».

Ядро среды исполнения

Главная функция среды — детерминированность исполнения. Без этого качества посредник перестаёт быть полезным: либо разрушается язык общения с внешним миром, либо стохастическая природа модели проникает наружу без фильтрации. Детерминированность опирается на два взаимодополняющих свойства среды:

- **Трансляция намерения.** Среда переводит задачу и контекст извне на язык и правила модели — превращает произвольный запрос пользователя или другой системы в спецификацию, которую модель умеет обрабатывать, с сохранением намерения и ограничений.
- **Контроль стохастики.** Среда удерживает стохастическую природу LLM-моделей — устанавливает баланс между способностью модели к генерации и потребностью внешнего мира в предсказуемом результате.

В каждом цикле исполнения среда выполняет четыре критические функции.

Функция	Описание
Трансляция	Преобразует произвольный запрос внешнего мира в машиночитаемый контекст для модели, сохраняя намерение и ограничения
Исполнение	Вызывает инструменты, работает с файловой системой, сетью и API строго в пределах выделенных прав и квот
Наблюдение	Фиксирует фактический результат, отличает успех от ошибки, извлекает сигнал из шума и динамически наполняет контекст сессии
Коррекция	Применяет политики, останавливает опасные действия, восстанавливает состояние при сбое и принимает решение о продолжении, откате или эскалации

Реверс-инжиниринг Claude Code v2.1.88 показал, что среда работы ИИ-агента составляет ~98,4% кодовой базы промышленных агентных сред. Именно она обеспечивает предсказуемость, безопасность и аудируемость исполнения — не сама модель принятия решений.

По данным мировых исследовательских компаний, эффективность среды работы ИИ-агентов подрывают пять системных антипаттернов. Ключевой — противоречивые правила: модели реально следуют лишь нескольким десяткам директив одновременно, и перегрузка хуков делает их декоративными, а не действующими ограничителями. Остальные антипаттерны усиливают этот эффект.

Антипаттерн	**Механизм деградации**
Перегруженный контекст	Объём контекста превышает эффективное окно внимания модели; нерелевантные фрагменты вытесняют критичные
Противоречивые правила	Модели следуют нескольким десяткам директив одновременно — перегрузка хуков превращает их в декорацию
Избыточные проверки без приоритизации	Каждое действие проходит одинаковую цепочку проверок независимо от риска; узкое место нарастает нелинейно
Уменьшающийся возврат на frontier-задачах	Среда эффективна для стандартных паттернов, но не масштабируется на задачи с высокой новизной и неопределённостью
Чрезмерная зависимость от mock-окружения	Виртуализация сервисов добавляет ценность, но полная замена реальной среды создаёт пробелы при выходе в промышленную эксплуатацию

Среда строится на детерминированной оболочке, рефлексах и валидации. Внутри одной сессии агент ведёт один логический поток рассуждения, что упрощает трассировку, отладку и восстановление. Параллелизм достигается через делегирование подзадач субагентам с изолированными контекстами, а не через многопоточность внутри одного агента. Каждое действие проходит через два набора мандатных контролей — до исполнения (проверка разрешений R0–R5, санитизация, достаточность контекста, контроль бюджета) и после исполнения (валидация результата, аудит, отслеживание побочных эффектов). Хуки исполняются синхронно, обеспечивая запрет по умолчанию — это архитектурные «рефлексы», срабатывающие до того, как модель осознаёт последствия. Модель лишь предлагает действие, а выполняет его и проверяет успешность всего цикла именно среда исполнения: наблюдательный слой фиксирует фактический вывод, сравнивает его с ожидаемым, сжимает и обогащает контекст.

Ядро среды исполнения — центральный координирующий компонент, через который проходит каждый цикл рассуждение–действие–наблюдение. Оно требует наименьших затрат на создание, но остаётся критичным: ошибка в ядре затрагивает каждый вызов инструмента. Ядро выполняет следующие функции:

Приём и разбор входящего запроса. Ядро получает спецификацию задачи, выделяет намерение, ограничения и параметры контекста, формирует структурированный объект задачи для передачи модели.

Диспетчеризация инструментов. По результату рассуждения модели ядро выбирает инструмент из реестра, формирует типизированный вызов и передаёт управление исполнителю в песочнице.

Управление циклом. Ядро отслеживает состояние сессии, обрабатывает ответ инструмента, обновляет контекст и принимает решение о следующем шаге — продолжить рассуждение, запросить уточнение, завершить задачу или эскалировать.

Инженерный цикл Claude Code детально описан в arXiv 2604.14228v1. Цикл рассуждение–действие–наблюдение занимает минимальный объём кода (~1,6%); основу составляет детерминированная оболочка — контрольная точка разрешений с запретом по умолчанию, управление контекстом через CLAUDE.md и каталог. claude/, диспетчеризация инструментов через явный реестр, цикл восстановления при ошибках, неизменяемый журнал аудита. Это наиболее изученная среда работы ИИ-агентов 2026 года. Реверс-инжиниринг подтвердил, что капитальный актив — именно детерминированная операционная оболочка, а не логика ИИ.

Корпоративный шлюз OpenClaw представляет альтернативную архитектуру для корпораций с несколькими арендаторами. Он добавляет шлюз применения политик ускорения, изоляцию арендаторов, корпоративный аудит и разрешения на основе ролей. Компромисс — более высокая операционная сложность в обмен на готовность к мультитенантному режиму с первого дня. Применение — организации с несколькими бизнес-единицами, разделяющими ИИ-инфраструктуру через единый слой управления.

Мы рекомендуем строить среду работы ИИ-агентов вокруг шаблона инженерного цикла (подход Claude Code) и интегрировать её с корпоративными шлюзами через стандартные протоколы (MCP, A2A) для координации между командами и общих ресурсов.

Механизмы изоляции

Изоляция агента — структурный механизм безопасности, гарантирующий, что агент работает строго в пределах выделенного контекста. Цель — предоставить ровно тот минимум прав и доступа, который необходим для задачи, и обеспечить нулевой выход за границы контура при сбое, ошибке или вредоносном поведении.

Платформа применяет риск-ориентированные профили песочниц. Вместо единой среды исполнения используются три класса изоляции, автоматически выбираемых по критичности задачи и классу применимости агента.

Профиль	Когда применяется	Уровень автономии (R)
PROD	Работа с кодом в main/release; задачи с персональными данными, финансовыми транзакциями, регуляторным контекстом	R0–R1 (только предложения или действия с обязательным подтверждением)
DEV	Реализация функций, исправление багов, рефакторинг в ветках функциональности; стандартные задачи без чувствительных данных	R2–R3 (автономная работа с ревью перед слиянием; автомерж при прохождении всех проверок)
R&D	Тестирование непроверенного кода, исследования безопасности, обновления зависимостей, jailbreak-тестирование	R4–R5 (высокая автономия; человеческий надзор в режиме «над циклом», а не «в цикле»)

Профиль песочницы фиксируется при старте и не меняется динамически. Любое изменение требований безопасности или области доступа требует перезапуска с новым профилем. Класс песочницы определяется на старте сессии на основе оценки ключевых параметров: критичность данных (затрагивает ли задача обработку персональных данных, финансовые транзакции или коммерческую тайну), обратимость действий (можно ли отменить результат без ущерба для промышленной среды) и доверие к источнику (является ли спецификация сертифицированным паттерном из библиотеки или экспериментальным черновиком).

Платформа обеспечивает нулевое распространение побочных эффектов. Сбой агента, бесконечный цикл, некорректный код или попытка выхода за пределы разрешённых конечных точек остаются в пределах выделенной песочницы. Профиль напрямую определяет активные хуки политик и допустимый уровень разрешений R0–R5. Квоты на CPU, память и время выполнения предотвращают «захват» инфраструктуры одной долгой или некорректной сессией.

Коммуникации в среде исполнения ИИ-агентов делятся на два контура, не пересекающихся по функции.

Контур	Назначение	Аналогия
Контур принуждения (Enforcement)	Мандатная проверка каждого действия агента до его исполнения	Istio sidecar / Envoy filter: правила встроены в трафик, а не проверяются постфактум
Контур интеграции (Integration)	Доступ агента к внешним инструментам и данным по мере необходимости задачи	Service mesh data plane: агент «видит» только разрешённые API, остальное скрыто

Первый контур гарантирует, что агент не сделает запрещённого; второй — что агент сможет сделать разрешённое. Разделение позволяет масштабировать контроль и интеграции независимо. Контур принуждения реализует измерение политики и доступа контура управления и обеспечивает исполнение риск-адаптивной лестницы в среде исполнения. Контур интеграции через MCP безопасно расширяет инструментарий без нарушения границ песочницы.

Контур принуждения проверяет каждое действие синхронно. Предобработка проверяет полномочия агента по лестнице R0–R5, контролирует бюджет токенов, валидирует профиль песочницы, очищает и обезличивает входящие данные. Постобработка валидирует результат, фиксирует побочные эффекты и пишет в аудит. Модель решений — разрешить, отказать, модифицировать или эскалировать — обязательна к исполнению.

Принуждение работает синхронно и неотключаемо для агента. Это принципиальное отличие от контура интеграции, где агент выбирает, какой инструмент вызвать. Контур принуждения выбора не предоставляет: хук срабатывает на каждое действие, независимо от того, осознаёт ли это агент. Аналогия из паттернов распределённых систем — envoy-фильтры в service mesh: трафик проходит через них независимо от воли сервиса-источника. Шаги обработки федеративны: проект может расширять действия и добавлять свою специфику, но ключевая дисциплина — исключительно расширяющая логика, добавление без ослабления базовой политики.

Архитектурная суть контура интеграции — расширение возможностей агента (enablement), а не его ограничение. MCP (Model Context Protocol) и A2A (Agent-to-Agent) существуют не для того, чтобы «не дать агенту сделать лишнее», а для того, чтобы дать агенту доступ к корпоративным системам — безопасный, регламентированный и полностью отслеживаемый.

MCP должен быть каталогом возможностей, а не списком функций. Инструменты регистрируются в реестре с машиночитаемым описанием: что делает, какие входные и выходные данные, какие ограничения, какой профиль песочницы требуется. Агент запрашивает инструмент по семантике («нужно прочитать конфиг», «нужно запустить тест»), а платформа сопоставляет запрос с зарегистрированной возможностью. Контур интеграции унифицирует контракты ввода-вывода: каждый инструмент имеет типизированную схему, что позволяет агенту формировать корректные вызовы без угадывания формата и фиксировать артефакты для аудита.

Различие между контурами прямо влияет на масштабирование. Контур принуждения масштабируется через федеративный реестр хуков — каждый новый домен добавляет свои хуки, не затрагивая ядро. Контур интеграции масштабируется через реестр MCP-серверов — каждый новый инструмент добавляется как отдельный сервер с описанием. Оба контура независимы: обновление набора инструментов не требует изменения политик безопасности, и наоборот. Без этого разделения агентная автономность становится либо небезопасной (без принуждения), либо непрактичной (без стандартизированной интеграции).

Контур длительных задач

Способность агента автономно работать над сложными задачами в течение часов или суток требует отдельного контура исполнения. Эта специализированная среда обеспечивает сохранение контекста, контрольные точки, безопасное восстановление после сбоев и управление ресурсами. Без этого контура любой рестарт инфраструктуры или ошибка прикладного\системного ПО означает потерю работы и контекста, а восстановление усложняется: требуется восстановление из последней валидной контрольной точки с прогревом контекста.

Контур длительных задач строится на архитектурных принципах:

- **Многоуровневое хранение состояний.** Контекст и артефакты сессии автоматически мигрируют между уровнями доступа в зависимости от активности: «горячий» для текущих сессий, «тёплый» для недавно завершённых, «холодный» для долгосрочного хранения и соответствия регуляторным требованиям. Переход между уровнями прозрачен для агентов.
- **Контрольные точки и восстановление.** Состояние периодически фиксируется по времени или по завершении значимых этапов. При сбое исполнителя сессия автоматически перезапускается с последней валидной контрольной точки. Для длительных задач отдельный супервизор с целевыми RTO/RPO — самостоятельная логика, которой нет в краткосрочном контуре.
- **Управление бюджетом токенов.** Автономность в длительных сессиях требует повышенного внимания к потреблению токенов и вычислений. Здесь ещё критичнее механизм circuit-breakers, нужны более сложные механизмы бюджетирования (per-task / per-session / per-horizon), могут использоваться дополнительные модели.
- **Асинхронный режим участия человека.** В коротких сессиях человек участвует интерактивно, в длительных — асинхронно: пакетные одобрения, ревью сводки вместо промежуточных подтверждений. Это требует отдельного UX, политик эскалации и агентов-защитников, калиброванных по горизонту задачи. Смешивание контуров разрушает оба паттерна участия человека.
- **Приоритизация и параллелизм.** Для длительной задачи ожидание в очереди приемлемо, тогда как интерактивный режим важнее по определению. Планировщик управляет параллелизмом длительных задач и их приоритизацией по бизнес-критичности.

Управление параллелизмом длительных задач — отдельная задача планировщика. Планировщик распределяет ресурсы между конкурирующими сессиями по критичности, управляет очередью с учётом дедлайнов, отслеживает прогресс и сигнализирует при риске не уложиться в бюджет. Интерактивность становится фактором приоритета: в интерактивной задаче человек ждёт максимально быстрый результат, для длительной ожидание в очереди приемлемо. Планировщик формализует эти различия как конфигурацию, а не как неявное поведение.

Контур исполнения длительных задач — операционная гарантия того, что агентная автономность не превращается в операционный хаос. Он обеспечивает предсказуемость, управляемые затраты и аудируемость, которых невозможно добиться в универсальной среде исполнения.

Наблюдаемость и сопровождение

Агентные системы сложнее в мониторинге и отладке, чем классические решения без ИИ. Если ML-системы сложны отладкой вероятностей выбора между известными сценариями, то LLM-модель выбирает, по сути, из неограниченного числа сценариев. Агентная среда стохастична, и работа в режиме «чёрного ящика» превращает любой сбой в кризис доверия. Систему невозможно масштабировать, если возникшую ошибку нельзя разобрать детально и устранить причину.

Каждое решение уровня R2+ сопровождается структурированным объяснением в момент выполнения действия: использованный контекст, применяемая модель, задействованные инструменты, сработавший хук политики. Целевая метрика — покрытие объяснениями >99% для уровня R3+.

Конвейер генерации объяснений проходит цепочку: запрос действия → агент генерации объяснений (выделенная малая LLM, например Phi-3 или Mistral 7B) → структурированное объяснение в JSON → проверка схемы (схема объектной модели PDLC) → слой кэширования (Redis, TTL 24 ч) → журнал аудита и поток наблюдаемости.

Для уровня R3+ генерация объяснения — блокирующий шаг перед действием. Данные сохраняются в неизменяемый журнал аудита и индексируются для извлечения; агент-защитник использует объяснения для проверок поведения. Наблюдаемость на уровне платформы строится на стандартном стеке: метрики — Prometheus и Grafana; трассировка — OpenTelemetry; агрегация журналов — ELK или аналог.

Пропускная способность в зрелой организации — 100–1000 объяснений в секунду при горизонтальном масштабировании через несколько агентов генерации. При превышении бюджета задержки объяснение генерируется асинхронно, но действие уровня R3+ ожидает завершения (блокирующий контракт). Это создаёт давление на выбор модели: для объяснений используется малая быстрая модель, а не основная модель кодирования.

Сопровождение охватывает оперативную сторону среды работы ИИ-агентов: развёртывание платформы, мониторинг в реальном времени, поведение под отказами и восстановление. Контур сопровождения проектируется одновременно со средой и является её неотъемлемой частью.

Оркестрация контейнеров опирается на Kubernetes с пользовательскими операторами для управления сессиями агентов. Оператор отвечает за создание пода сессии, выбор профиля песочницы, квоты ресурсов, постоянство состояния сессии и очистку при завершении. Под на сессию включает процесс агента, монтирование файловой системы в песочнице, ограниченный сетевой доступ и сайдкар для потоковой передачи журнала аудита.

Журнал аудита ведётся в формате JSONL только для добавления. Каждое событие записывается немедленно. Сериализация идёт в ротируемые файлы с ежедневной ротацией и сжатием, архивация — в неизменяемое хранилище (S3 с блокировкой объектов или реестровая база данных). Сроки хранения настраиваются по классификации данных: общие операции — 1 год; регулируемые отрасли — 7 лет.

В типичный рабочий день зрелой команды среда исполнения обрабатывает 100–500 одновременных сессий. Распределение нагрузки: ~70% коротких задач (<30 мин), ~20%

средних (30 мин – 4 ч), ~10% долгоработающих (4 ч+). Потребление ресурсов отслеживается по сессиям: токены, время вычислений, память.

Платформа заранее определяет реакцию на типовые сбои.

Тип сбоя	**Реакция платформы**
Сбой исполнителя в песочнице	Завершение песочницы, перезапуск, возобновление с последней контрольной точки (долгие задачи) или быстрый отказ (короткие)
Деградация задержки шины политик-хуков	Автоматический выключатель; обход некритичных хуков при оповещении; уведомление команды платформы
Повреждение хранилища состояния	Восстановление из резервной копии; затронутые сессии помечаются «требуется восстановление»
Перегрузка слоя объяснимости	Переход на генерацию объяснений более дешёвой моделью; при сохранении нагрузки — оповещение

Целевое время восстановления (RTO) — 1 час для полного простоя среды исполнения; целевая точка восстановления (RPO) — 5 минут. Стратегия: активный-активный в нескольких регионах для хранилища состояния и асинхронная репликация для журнала аудита. Ежегодная проверка имитирует полный простой, отрабатывает восстановление и закрепляет извлечённые уроки.

Управление мощностями

Управление мощностями в среде работы ИИ-агентов — не планирование «железа» под пиковую нагрузку, а эластичное распределение ограниченных ресурсов (токены, вычисления, память, сетевые квоты) между конкурирующими сессиями с гарантией предсказуемости для критичных задач. Агентная среда генерирует стохастическую нагрузку: в отличие от классических систем, где нагрузка детерминирована запросами пользователей, одна сессия может потребовать 10 тыс. токенов за минуту, другая — 100 тыс. за час. Отсутствие структурного управления приводит к дефициту ресурсов для решения критичных задач, неконтролируемым затратам и непредсказуемым задержкам.

Управление мощностями строится на четырёх принципах:

- **Изоляция ресурсов на уровне сессии.** Каждая сессия работает в выделенном контуре ресурсов с явными квотами: бюджет токенов (per-task / per-session / per-horizon), лимиты вычислений, память, сетевые вызовы. Это не даёт одной «разбушевавшейся» сессии исчерпать ресурсы платформы. Квоты настраиваются через Pattern Library: стандартные паттерны получают предустановленные бюджеты, экспериментальные — динамические лимиты с эскалацией.
- **Приоритизация по бизнес-критичности, а не по времени поступления.** Планировщик распределяет ресурсы на основе триады раздела 4.3: критичность для бизнеса × профиль риска × архитектурная сложность. Задача с высоким риском и стратегической ценностью получает приоритет над фоновым рефакторингом, даже если поступила позже. Управление ресурсами становится частью политики, а не инфраструктурной утилиты.
- **Эластичная деградация вместо бинарного отказа.** При исчерпании квот или деградации подсистемы платформа не прерывает сессию аварийно, а переходит в режим пониженной функциональности: приостанавливает некритичные действия, сохраняет состояние в контрольной точке, эскалирует решение человеку. Сбой в одной компоненте не приводит к каскадному отказу всей среды исполнения.
- **Связь затрат с результатом (CTOR) как обратная связь.** Механизм управления мощностями экспонирует метрику отношения стоимости к результату (Cost-to-Outcome Ratio) не как дашборд, а как сигнал для решений: если задача систематически превышает бюджет без подтверждения результата, платформа автоматически снижает её приоритет или требует явного переопределения от владельца. Это превращает управление ресурсами из операционной задачи в механизм инвестиционной дисциплины.

Развитие платформы

Операционная картина зрелой банковской организации задаёт устойчивое соотношение ролей. Команда продуктовой разработки — 8 инженеров, менеджер продукта, дизайнер, тестировщик (11 человек). Команда платформы — 3–5 инженеров платформы и 1 менеджер платформы. Соотношение — 1 инженер платформы на 8–12 продуктовых инженеров; на каждые 2–3 новые продуктовые команды добавляется 1 инженер платформы.

Координация между командами строится по четырём горизонтам встреч.

Встреча	Периодичность	Участники	Длительность	Темы
Рабочая группа платформы	Еженедельно	Команда платформы и представители продуктовых команд	60 мин	Операционные вопросы, предстоящие изменения, обмен шаблонами
Совет платформы	Ежемесячно	Старшее руководство	90 мин	Стратегические решения, дорожная карта
Общее собрание	Ежеквартально	Все инженеры	2 ч	Метрики, демонстрация знаний и шаблонов

Встреча	Периодичность	Участники	Длительность	Темы
Выездной семинар	Ежегодно	Команда платформы и ключевые партнёры	2 дня	Стратегическое планирование, глубокая техническая работа

Одна продуктовая команда среднего размера показывает предсказуемый недельный цикл работы. Команда из 8 инженеров (2 старших, 4 средних, 2 младших), менеджер продукта, дизайнер, тестировщик. Понедельник — сессия исследования (90 мин); вторник–четверг — циклы реализации с параллельными агентными сессиями и ручной координацией; пятница — ретроспектива и планирование.

Пропускная способность команды — ~15 задач за квартал против ~6 ранее. Ещё 25 задач из бэклога реализуются через шаблоны и долгоживущих агентов. Уровень корректности результатов — ~65%, повторное использование шаблонов — в ~50% задач, стоимость токенов — ~$3 тыс. на крупную задачу. Старшие инженеры ведут 3–4 задачи одновременно, средние — 2–3, младшие работают в паре со старшими.

Типичный день старшего инженера по продукту показывает, как распределяется внимание между агентами и людьми.

Время	Активность	Длительность
9:00	Проверка утренних сводок от ночных задач (2 PR от ночного рефакторинга одобрены батчем)	5 мин.
9:15	Первая агентная сессия: загрузка вчерашнего SDD, агент генерирует начальную реализацию	45 мин. работы агента + 5 мин. наблюдения
10:00	Сессия исследования с менеджером продукта по новой функциональности; черновик PR/FAQ	60 мин.
11:00	Проверка PR от агента (2 проблемы отмечены агентом проверки); целенаправленное человеческое ревью; слияние	30 мин.
11:30	Коллективная детализация с двумя инженерами; черновик SDD	60 мин.
12:30	Обед	—
13:30	Вторая агентная сессия для нового SDD	50 мин. работы агента + 10 мин. наблюдения
14:30	Сессия проверки результата для функциональности, развёрнутой 2 недели назад: анализ метрик, подтверждение результата, обновление шаблона	—
15:00	Парное ревью PR другого инженера	45 мин.
15:45	Обработка эскалации: агент-защитник отметил аномалию; расследование, выявление первопричины, добавление поведенческого оценочного теста	30 мин.

Время	Активность	Длительность
16:15	Курирование библиотеки шаблонов: уточнение существующего шаблона	45 мин.
17:00	Завершение дня	—

Итог дня — 5 задач в различных состояниях, последовательный прогресс, качество, проверенное через контур валидации.

Ежеквартальное бизнес-ревью инвестиций в IDP исключает ситуативность решений. Старшее руководство оценивает общие инвестиции (инфраструктура и команда платформы), агрегированные выгоды по командам (прирост выпуска, уровень проверки результатов, сокращение времени выхода на рынок) и сравнение с отраслевыми эталонами. Решение фиксирует продолжение инвестиций и пробелы на следующий квартал — например, расширение цикла разработки ИИ-агентов (Agent Development Life Cycle, ADLC) для двух дополнительных продуктов.

Годовой цикл планирования задаёт операционный ритм платформы. Ноябрь–декабрь — комплексное ревью метрик, сравнение с эталонами, выявление конкурентных пробелов. Январь — план инвестиций на год и дорожная карта. Февраль–октябрь — выполнение квартальных вех. Ноябрь — ретроспектива и план на следующий год. Без этой дисциплины решения принимаются ситуативно, что ведёт к непоследовательности инвестиций и неясному возврату.

Команда платформы постоянно развивает IDP. Драйверы — новые технологии (обновления моделей, новые серверы MCP, новые шаблоны), новые нормативные требования, новые потребности продуктовых команд, возникающие антипаттерны. Элементы дорожной карты приоритизируются по серьёзности проблемы, величине выгоды, соответствию стратегическому направлению и технической осуществимости — по тем же правилам, что и дорожная карта любой инженерной команды.

Ключевой принцип эволюции — цикл обучения: открытие одной команды → платформа кодифицирует его в шаблон → выгоду получают все команды. Это и отличает IDP от набора инструментов: инструменты используются, а IDP накапливает знание об их применении и превращает его в переиспользуемые паттерны. Каждый новый антипаттерн, обнаруженный в эксплуатации, становится кандидатом на новый хук или обновление шаблона. Поэтому кривая ценности IDP не выходит на плато, а продолжает расти по мере накопления организационного опыта.

Принципы проектирования агентных инструментов

Качество опыта агента определяется не числом инструментов, а тем, как спроектирован каждый из них и весь реестр. Хорошо спроектированный инструмент снимает с агента работу по интерпретации и восстановлению; плохо спроектированный превращает реестр в источник трения.

Реестр держат компактным. Избыточные варианты одного действия путают агента и повышают стоимость токенов на рассуждение о выборе. Каждый инструмент выполняет одну функцию со структурированными параметрами, а ошибку возвращает типизированным кодом, а не молчаливым пустым результатом.

Имена инструментов иерархичны и обнаруживаемы. Пространства имён — `tools.search`, `tools.fs.read`, `tools.git.commit` — снижают когнитивную нагрузку агента; целевая точность выбора инструмента превышает 90%. Компактный дескриптор со ссылкой на

подробную документацию дешевле развёрнутого описания, которое агент «видит» на каждом шаге рассуждения, а структурированный JSON на выходе дешевле неструктурированного текста.

Инструменты проектируются на основе оценочных тестов. Каждый инструмент имеет тесты на корректное использование, а стоимость токенов входит в метрики теста. Если инструмент систематически вызывается некорректно, его перепроектируют — это не должно становиться поводом для переобучения агента.

Инструмент безопасен к повторной попытке. Инструмент, небезопасный к повторному вызову, имеет два режима сбоя: инструмент с побочным эффектом, завершившийся с ошибкой, при повторе дублирует эффект — платёж, отправку письма, развёртывание; частично завершённый инструмент при повторе создаёт несогласованное состояние. Решение — изначальная идемпотентность (запись игнорирует дубликаты) либо приём ключа идемпотентности; это критично для долгоработающих задач, где при восстановлении из контрольной точки часть инструментов вызывается повторно.

Каждый инструмент возвращает типизированный ответ по схеме PDLC Semantic Layer. Структурированный выход позволяет агенту интерпретировать результат без постобработки и автоматически фиксировать артефакты для аудита. Благодаря этому реестр инструментов становится предсказуемой поверхностью.

Открытые протоколы

Коммуникационный каркас IDP опирается на открытые протоколы, каждый из которых закрывает свой канал связи: инструменты, агент-агент и агент-интерфейс. Открытые стандарты делают каркас переносимым между вендорами, а проприетарная интеграция остаётся исключением под конкретный сценарий надёжности.

Model Context Protocol (MCP) — стандарт интеграции внешних источников инструментов и данных для агентов. IDP внедряет его через отдельный сервер MCP на каждый крупный инструмент: GitHub, Jira, Slack, внутренняя кодовая база. Серверы изолированы, разрешения выдаются через реестр инструментов, а сам протокол снижает стоимость переключения между моделями и убирает зависимость от поставщика.

Мы рекомендуем интегрировать новые инструменты сначала через MCP, а не через SDK конкретного вендора. Оговорка существенна: в ряде сценариев нативная поддержка контекста надёжнее MCP-интеграции, поскольку модель может проигнорировать MCP-источник даже при явной инструкции его использовать. Там, где нативная интеграция контекста доступна, её промышленная надёжность выше.

Agent-to-Agent (A2A) — протокол координации между агентами. Он обеспечивает передачу фазы внутри цикла реализации — кодирующий → тестирующий → проверяющий → документирующий — и коммуникацию оркестратора с рабочими агентами в многоагентной среде. Минимальная схема передачи для PDLC: имя задачи, ссылка на спецификацию, ссылка на артефакт, указатель на доказательства.

AG-UI — формализующийся стандарт интерфейса взаимодействия человека с агентом. Он задаёт структурированный формат хода работы, запросы действий, запросы одобрения и отображение объяснений. В стеке Bain он располагается поверх систем учёта и операционной среды агентов. Мы рекомендуем AG-UI для агентных продуктов, тогда как агенты доставки классических систем остаются в IDE, CLI и headless-режиме.

Функциональный профиль IDP

Функциональный профиль IDP задаёт, что платформа умеет, а компоненты архитектуры — где это реализовано. Каждая возможность имеет чёткое назначение и относится к одному из компонентов архитектуры. Вместе они переводят золотой путь из процесса в исполнимую платформу.

№	Возможность	Назначение	Компонент IDP
1	Specification Registry	Хранилище SDD, версионирование, прослеживаемость	Оркестрация
2	Eval Registry	Оценочные тесты возможностей, регрессии, длины сессии, эскалации	Контекст / спецификации
3	Context Management	Сжатие в 5 уровней, сохранённые заметки, память с типизированной схемой	Контекст / спецификации
4	Tool Registry & Dispatch	Каталог инструментов, брокер MCP/A2A	Оркестрация
5	Policy Hook Framework	Хуки перед/после/остановка, запрет по умолчанию	Оркестрация
6	Token Economics Tracker	Управление финансами по задаче / сессии / горизонту	Управление / безопасность / финансы
7	Аудит в режиме дополнения	Журнал сессии в формате JSONL, события жизненного цикла	Управление / безопасность / финансы
8	Guardian Agent	Поведенческое управление, политики эскалации	Качество / валидация
9	Knowledge Graph (per project)	Структурированный контекст проекта	Контекст / спецификации
10	Eval-Driven CI/CD	Контрольная точка ИИ в конвейере, фабрика без интерфейса	Качество / валидация
11	Agent Development Life Cycle (ADLC)	Включение в состав, дрейф, дообучение, вывод из эксплуатации	Эксплуатация / жизненный цикл
12	Outcome Pricing Engine	Отношение стоимости к результату, доходность результата	Эксплуатация / жизненный цикл
13	Federated Policy Hook Registry	Базовая линия, хуки команды и домена	Оркестрация

№	Возможность	Назначение	Компонент IDP
14	Real-time Explainability Layer	Структурированные объяснения R2+, обязательно R3+	Качество / валидация
15	PDLC Semantic Layer	Схема артефактов PDLC: спецификация, задача, запрос на слияние, контрольная точка, оценочный тест, аудит; реестр спецификаций и рабочих процессов	Контекст / спецификации

Возможности группируются по тому, что они делают. Оркестрация ведёт спецификации, каталог инструментов, политики-хуки и их федерацию. Контекст и спецификации хранят оценочные тесты, управление контекстом, граф знаний проекта и семантический слой PDLC. Управление, безопасность и финансы отвечают за экономику токенов и неизменяемый аудит. Качество и валидация опираются на агента-защитника, eval-управляемый конвейер и слой объяснимости. Эксплуатация и жизненный цикл ведут агентов через ADLC и связывают затраты с результатом.

Граница PDLC Semantic Layer намеренно узкая. Схема описывает только артефакты PDLC: спецификация, задача, запрос на слияние, контрольная точка, пакет оценочных тестов, решение политики, событие аудита, вызов инструмента, проверочный пакет. Вне области — клиент (CRM), счёт (финансы), учётная запись (идентификация), контракт (право), транзакция (платежи), заказ (коммерция), полис (страхование), отгрузка (логистика): это задача отдельного корпоративного семантического слоя. Попытка команды PDLC решить её внутри IDP воспроизводит антипаттерн «разрастание семантического слоя» — конфликты с командой данных, кризис владения и деградацию обоих слоёв.

Цикл разработки ИИ-агентов (ADLC) управляет жизненным циклом агента по функциям, которыми кадровая служба ведёт сотрудников: найм, включение в состав, оценка результативности, обучение, планирование преемственности. Для агента эти функции превращаются в конвейер: спецификация агента → набор оценочных тестов → непрерывная оценка → обнаружение дрейфа → дообучение → устаревание → вывод.

Конвейер ADLC проходит через явные фазы:

- **Спецификация** — набор оценочных тестов, базовые показатели производительности, профиль песочницы.
- **Включение в состав** — тестирование в изолированной среде, постепенное развёртывание, мониторинг базовой линии.
- **Промышленная среда** — непрерывный мониторинг, периодические повторные прогоны, сравнение с базовой линией, оповещение при значимом отклонении.
- **Дообучение** — запуск при систематическом дрейфе, автоматическое или управляемое человеком переобучение.
- **Устаревание** — период объявления, план миграции.
- **Вывод** — явное удаление агента, архивированная история оценочных тестов для соответствия требованиям.

Владелец ADLC — как правило, AgentOps Engineer.

Механизм оценки стоимости результата (Outcome Pricing Engine, OPE) связывает затраты с бизнес-результатом. Каждая агентная задача создаёт точку данных: task_id, суммарная

стоимость токенов, статус результата, оценка ценности результата. Механизм агрегирует стоимость токенов по сессиям задачи, статус проверки результата и успех развёртывания.

Метрики механизма направляют инвестиционные решения данными, а не наитием. Отношение стоимости к результату (CTOR) = общая стоимость токенов / ценность подтверждённого результата; доходность результата = доля задач с подтверждённым результатом от числа развёрнутых. Ежеквартальная агрегация по команде, шаблону и домену даёт распределения CTOR: шаблон с CTOR 0,2 продвигается, шаблон с 2,5 уходит в ретроспективу и возможный вывод, домен с растущим трендом инициирует расширение покрытия библиотеки паттернов.

Федеративный реестр политик-хуков (Federated Policy Hook Registry) разрешает конфликты хуков. При «разрешить» против «отказать» побеждает отказ; при «разрешить» против «разрешить» с разными изменениями хуки применяются последовательно по приоритету. Реестр хранит историю изменений и допускает откат.

Владение хуками разделено между платформой и доменами. Команда платформы владеет базовыми хуками — безопасность, соответствие, аудит, требования к проверочному пакету. Команды домена владеют специфичными хуками: проверка PCI DSS для платёжных систем, ведение журнала с учётом HIPAA для здравоохранения, ограничение скорости крупнообъёмных операций для трейдинга.

Слой объяснимости в реальном времени (Real-time Explainability Layer) задаёт схему структурированного объяснения: использованный контекст, версия и параметры модели, идентификаторы применённых паттернов, краткое изложение рассуждений, выбранный инструмент, применённые хуки политик, ожидаемый результат. Объяснение — типизированный объект схемы PDLC.

Для уровня автономии R3+ объяснение — блокирующий шаг. Данные сохраняются в неизменяемый журнал аудита и индексируются для извлечения, что позволяет специалистам по соответствию и владельцам продуктов расследовать инциденты постфактум.

Операционное управление PDLC Semantic Layer удерживает схему от разрастания. Определения схем хранятся в репозитории IDP, версионируются и проходят ревью изменений по образцу OpenAPI. Прежняя версия сохраняется квартал после ввода новой, агенты мигрируют автоматизированными инструментами, а ежеквартальное ревью артефактов PDLC даёт отчёт о покрытии. Новая схема требует одобрения команды платформы и отклоняется, если пересекает границу бизнес-объекта.

«Golden Path» и его варианты

Золотой путь (Golden Path) — сквозной процесс от исследования до промышленной эксплуатации через стандартизированные контрольные точки. Путь начинается с артефакта исследования, а не с заявки. Именно контрольные точки, а не конкретный маршрут, гарантируют качество.

Проторенные пути (Paved Roads) — предварительно сконфигурированные маршруты исполнения золотого пути для повторяющихся классов задач. Каждый маршрут объединяет в единый контракт шаблон SDD, конфигурацию CI/CD, запись из библиотеки паттернов (Pattern Library), структуру проверочного пакета (Evidence Bundle), карту точек человеческого контроля и класс применимости агента. Команда выбирает маршрут под характер задачи, а не собирает процесс с нуля: скорость, качество и встроенный контроль заданы заранее.

Проторенные пути опираются на устойчивые архитектурные принципы:

- **Стандартизация без жёсткости.** Путь задаёт каркас, а не диктует каждый шаг: команда расширяет шаблон и добавляет специфичные тесты, но не отключает обязательные элементы — гипотезу результата, политики безопасности, аудит-след.
- **Контур управления по умолчанию.** Каждый путь активирует хуки политик, уровни разрешений R0–R5 и контрольные точки валидации; соответствие требованиям встроено в маршрут, а не добавляется постфактум.
- **Композитность и переиспользование.** Пути собираются из атомарных компонентов — шаблонов спецификаций, конфигураций CI/CD, наборов оценочных тестов, профилей песочниц; новые маршруты создаются компоновкой существующих блоков.
- **Эволюция на основе телеметрии.** Каждый путь сопровождается метриками — время до первого запроса на слияние, доля повторного использования спецификаций (Spec Reuse Rate), отношение стоимости к результату (CTOR); при систематических отклонениях путь пересматривает команда платформы.

Золотой путь существует в нескольких вариантах под характер задачи:

- **Синхронный золотой путь** — короткие задачи с человеческим ревью в конце. Маршрут: исследование → SDD → запуск агента в ветке функциональности → 30–90 минут работы → создание запроса на слияние → проверка агентом-ревьюером → человеческое ревью → слияние → развёртывание. Полный цикл — часы; покрывает 60–80% задач зрелой команды. Метрика: медиана от спецификации до развёртывания <4 часов.
- **Асинхронный путь для долгоработающих задач** — ночной рефакторинг, обновление зависимостей, устранение уязвимостей, очистка бэклога. Маршрут: SDD с декомпозицией сессии → запуск долгоработающего агента с feature-list.json → ночной прогон с контрольными точками в Slack каждые 30–60 минут → утренняя сводка → пакетное человеческое ревью → пакетное слияние → развёртывание. Полный цикл — календарный день. Метрика: медиана пропускной способности ночного прогона.
- **Путь резкого масштабирования (Surge)** — инциденты безопасности, срочные миграции, регуляторные дедлайны. Маршрут: срочная потребность → сборка пакета контекста (CLAUDE.md, выдержки спецификации, релевантные записи библиотеки паттернов) → запуск множества агентов параллельно с разделённой ответственностью → непрерывный человеческий надзор → быстрый цикл слияния. Кейс Augment Code: проект на 4–8 месяцев сжимается до 2 недель.
- **Полоса исследования — экспресс-путь**: PR/FAQ → гипотеза результата → SDD; исследовательский контур без агентной реализации. Цель — снизить стоимость отрицательных открытий: проверить гипотезу прежде, чем запускать цикл реализации. Метрика: время от запроса до SDD <2 часов.

Мы придерживаемся вендор-агностичного подхода к интеграции. Где возможно, IDP опирается на открытые стандарты (MCP, A2A, AG-UI); вендор-специфичные элементы оборачиваются слоем абстракции — ИИ-шлюз (AI Gateway) скрывает нескольких поставщиков моделей, библиотека серверов MCP скрывает их API. Это минимизирует привязку: поставщика можно сменить без переписывания агентов.

Интеграция с платформами данных проходит через контролируемые каналы. Агенты читают данные промышленной среды из озера или хранилища данных через серверы MCP, а правила обработки персональных данных фиксируются в поведенческих оценочных тестах. Платформы машинного обучения (MLflow, SageMaker, Vertex) подключаются к ModelOps: версионирование и отслеживание моделей, общая инфраструктура A/B-тестирования.

Интеграция с существующими системами происходит инкрементально, без масштабного единовременного внедрения. Устоявшиеся CI/CD (Jenkins, GitLab CI, GitHub Actions) не заменяются — IDP добавляет контрольные точки ИИ как дополнительные шаги в существующие конвейеры. Контрольная точка ИИ — скрипт оболочки или контейнер Docker, вызываемый из CI, с результатом «пройдено / не пройдено / требуется человеческое ревью»; накладные расходы — 10–30 секунд на шаг, что не влияет существенно на время конвейера.

Интеграция с инструментами разработки сохраняет привычные рабочие процессы команды:

- Git — нативная поддержка: агенты фиксируют изменения от имени человека с атрибуцией, журнал аудита связывает фиксации с сессиями.
- Jira / Linear — двунаправленная связь через MCP: агенты обновляют заявки, заявки порождают контекст.
- GitHub / GitLab — нативные процессы запросов на слияние: комментарии агента-ревьюера, контрольные точки ИИ в CI/CD.
- Slack / Teams — интерфейсы результатов для долгоработающих задач: уведомления о контрольных точках, запросы одобрения.
- Confluence / Notion — генерация документации, публикация библиотеки паттернов.

Повторяемость — условие промышленной эксплуатации

Опыт агента (Agent UX) — качество взаимодействия агента со стеком IDP. Мы оцениваем его по измерениям, показывающим, насколько эффективно агент «видит» стек.

Измерение	Что оценивает	Целевая метрика
Обнаруживаемость	Доля случаев, когда агент находит правильный инструмент без избыточных попыток	>90%
Точность выбора инструмента	Доля случаев, когда агент выбирает правильный инструмент с первой попытки	>85%
Структура ответа	Доля инструментов с типизированной схемой вывода (PDLC Semantic Layer)	>90%
Обработка ошибок	Доля ошибок с явным типизированным кодом (не молчаливый возврат null)	100%
Аутентификация агента	Доля инструментов с машиночитаемым профилем требований к разрешениям	>95%

Возобновляемость критична для долгоработающих задач через несколько сессий. Тест: запустить агента на задаче, прервать процесс в середине, восстановить из контрольной точки, проверить корректное продолжение. Возобновляемость держится на полноте контрольной точки (весь необходимый контекст сохранён), детерминированности прогрева контекста (восстановление даёт тот же контекст, что был до сбоя) и идемпотентности инструментов (повторные вызовы не создают дублей).

Метрики опыта агента — это метрики качества IDP. Низкая обнаруживаемость и точность выбора инструмента приводят к избыточным итерациям, повышают стоимость токенов и

снижают пропускную способность: если агент тратит 3–5 шагов рассуждения на выбор инструмента вместо одного, это прямые потери токенов и времени до результата.

Опыт агента и Guardian Agents решают взаимодополняющие задачи. Опыт агента отвечает на вопрос, насколько эффективно агент работает со стеком инструментов; Guardian Agents — насколько безопасно он действует в рамках политик. Без Guardian Agents агент может быстро продвигаться по ошибочному пути. При этом сами Guardian Agents, если базовый опыт агента недостаточно проработан, приводят к избыточным проверкам и повышают вероятность ошибок выбора.

4.3 Управление и автономия

Управление на платформе исполняется на каждой операции агента, а не согласуется раз в год. Мы переводим правила безопасности, соответствия требованиям и использования данных в машиночитаемую политику, версионируемую в Git наравне с кодом, и связываем её с риск-адаптивной лестницей разрешений, которая определяет, что агенту дозволено сделать в конкретном контексте. Управление задаёт правила, лестница задаёт границы автономии, в которых эти правила применяются.

Управление как код

Управление как код встраивается в инженерный рабочий процесс сразу, а не добавляется сверху отдельным этапом. Фреймворк хуков политик переводит управление из документарной формы в исполняемую: политика выполняется на каждой релевантной агентной операции.

Практические следствия подхода:

- Файлы политик хранятся в репозитории рядом с кодом и SDD.
- Изменение политики проходит через PR аналогично изменению кода.
- Каждая политика имеет автоматические тесты, проверяющие корректность на синтетических сценариях.
- Политика запускается через шину политик-хуков на каждом релевантном пункте вызова.

Шаблон разработки через спецификацию (SDD) расширен секцией «Кодифицированные ограничители». В ней фиксируются ограничения безопасности (закодированные правила OWASP/CWE), ограничения соответствия требованиям (закодированные правила 152-ФЗ / GDPR / требования ЦБ, где применимо), ограничения использования данных (обработка персональных данных, хранение, локализация) и отображение лестницы разрешений (уровень по умолчанию плюс условия эскалации). Соответствие становится частью спецификации изменения, а не проверкой на поздних этапах.

Агент воспринимает кодифицированные ограничители как первичный контекст. Они загружаются в начале сессии вместе со спецификацией, а не применяются как внешний барьер после генерации.

Связь с уровнями разрешений R0–R5 определяет режим действия политики. На уровнях R0–R2 политика выступает прямым принуждением — блокировать, запросить, изменить; на уровнях R3–R5 — сигналом эскалации к человеку в цикле. На уровне R5 политика продолжает действовать автономно, и нарушение приводит к автоматическому откату.

Аспект	Управление в форме документов	Управление как код
Изменение	Ежегодное ревью, утверждение руководством	PR и тесты
Проверяемость	Ручная интерпретация	Автоматические тесты и модульные тесты политики
Принуждение	Ручной контрольный список, постфактум	Шина политик-хуков, среда исполнения
Возможность аудита	Снимки, выборка	Журнал событий только для добавления, полное покрытие

Требования 152-ФЗ и ЦБ

Требования 152-ФЗ и ЦБ к аудиторскому следу мы кодифицируем как примитивы управления в виде кода. Закодированное соответствие требованиям OWASP / GDPR в глобальной практике заменяется или дополняется закодированным соответствием требованиям ГОСТ Р, ФСТЭК, 152-ФЗ для обработки персональных данных и стандартам ЦБ по хранению и аудиту.

Мы выделяем внутреннюю библиотеку модулей политик «ядро соответствия требованиям РФ» с тремя ключевыми примитивами:

- **Локализация персональных данных** — запрет отправки персональных данных на иностранные серверы.
- **Хранение аудита 7 лет** — неизменяемый журнал с 7-летним хранением для финансовых транзакций.
- **Изоляция критической инфраструктуры** — ограничения на использование внешних ресурсов для критичных задач.

Такая библиотека становится переиспользуемым активом уровня IDP. Каждая новая задача с персональными данными или финансовой транзакцией автоматически наследует эти политики без отдельного обсуждения соответствия. Вместо проверки каждого запроса офицером по соответствию политика как код автоматически применяет правильный уровень надзора в российском регуляторном контексте.

Риск-адаптивные полномочия

Риск-адаптивная лестница разрешений R0–R5 определяет границы автономии агента, оставаясь адаптивной, а не статической.

Уровень	Что разрешено	Когда применим
R0	Только чтение	Промышленная среда без проверенных оценочных тестов
R1	Только предложения, принимает человек	Критичные пути промышленной среды

Уровень	Что разрешено	Когда применим
R2	Действия в ветке функциональности с ревью запроса на слияние	Стандартная разработка
R3	Автоматическое слияние при прохождении оценочных тестов	Стандартизированные шаблоны, библиотека шаблонов
R4	Выполнение в нескольких сессиях с контрольными точками	Долго работающие, зрелые, проверенные сценарии
R5	Эксперимент в песочнице, полная автономия	Изолированные среды экспериментов

Один и тот же агент может иметь уровень R1 в промышленной среде, R3 в ветке функциональности и R5 в песочнице. Уровень автономии зависит от среды, ветки, критичности файла, обратимости действия, покрытия тестами, радиуса воздействия, статуса пакета доказательств и длины горизонта. Переход между уровнями определяют хуки политик, а не статическая конфигурация.

Эскалацию и деэскалацию определяют восемь факторов:

- Среда: промышленная / промежуточная / разработки / песочница.
- Ветка: основная / выпуска / функциональности / экспериментальная.
- Критичность файла: конфигурация промышленной среды / бизнес-логика / тесты / документация.
- Обратимость действия: чтение / добавление / изменение / удаление.
- Покрытие тестами целевого модуля: <30% / 30–70% / >70%.
- Радиус воздействия: одна функция / один модуль / межмодульное / межсервисное.
- Статус пакета доказательств: неполный / частичный / полный с проходящими оценочными тестами.
- Длина горизонта: <30 мин / 30 мин–4 ч / 4 ч–1 день / несколько дней.

Дилемма управления по BCG объясняет, почему лестница должна учитывать контекст. Инструментами владеют, и они предсказуемы: детерминированное выполнение, фиксированная область, отсутствие автономии. Люди автономны и под надзором: переменное качество, эволюционирующая область, надзор через проверки результативности. Агенты сочетают оба свойства, поэтому уровень автономии нельзя закрепить статически.

Факторы автономии

Мы выделяем три фактора, определяющих эффективный уровень автономии агента; каждый смещает его от номинального:

Фактор	**Низкое значение (человек над циклом)**	**Высокое значение (человек в цикле)**
Критичность для бизнеса	Тактическая: внутренние DevOps-инструменты, приложения поддержки бизнеса	Стратегическая: клиентские приложения, приложения с основным конкурентным преимуществом
Профиль риска	Низкий: нет персональных данных, нет финансовых транзакций, нет регулируемых данных	Высокий: персональные данные, финансовые транзакции, регулируемые данные, критичные к безопасности пути
Архитектурная сложность	Низкая: один модуль, хорошо изолированное изменение	Высокая: сквозные вопросы, распределённое состояние, несколько интеграций

Эффективный уровень рассчитывается как эффективный_R = номинальный_R − повышение_критичности − повышение_риска − повышение_сложности, где повышение — число ступеней (0, 1 или 2), на которое снижается автономия из-за высокого значения фактора.

Возьмём задачу, классифицированную как R3 (автоматическое одобрение по типу), при высокой критичности для бизнеса (+2) и высоком профиле риска (+1). Тогда эффективный_R = 3 − 2 − 1 = 0, то есть задача требует участия человека в цикле для всех ключевых решений.

Человек над циклом и человек в цикле — два качественно различных режима надзора. Человек в цикле участвует в каждой значимой точке решения — это характерно для R0–R2. Человек над циклом наблюдает периодически и вмешивается при исключениях — это характерно для R3–R5. Уровень R3 служит границей переключения режимов: документация для кода и архитектуры идёт в режиме человека над циклом, а проектирование новых функций с чувствительными данными остаётся в режиме человека в цикле.

На каждой агентной операции — записи в файл, вызове API, развёртывании, изменении конфигурации — шина политик-хуков вызывает оценщик контекста, пересчитывающий эффективную автономию по трём факторам. Факторы предоставляют: SDD (критичность для бизнеса фиксируется в гипотезе результата), кодифицированные ограничители (профиль риска фиксируется в ограничениях использования данных, архитектурный граф проекта (сложность вычисляется по радиусу воздействия операции). Результат определяет активные хуки политик и применяемый режим надзора.

Статическая лестница разрешений без адаптации в среде исполнения — антипаттерн. Лестница без пересчёта на основе триады либо чрезмерно управляет задачами, теряя гибкость, либо недостаточно управляет критичными решениями, теряя безопасность.

Для финансовых организаций российский регуляторный контекст задаёт дополнительные фиксированные смещения: документация для устаревших модулей АБС — R5 → R4; новая функция клиентского кабинета — R3 → R1; изменение AML-конвейера — R2 → R0. Эти смещения формализуются в библиотеке политик «ядро соответствия требованиям РФ и применяются автоматически, без отдельного обсуждения соответствия для каждой задачи.

4.4 Контур разработки агентного продукта

Когда продукт сам является агентной системой, классический PDLC перестраивается в отдельный контур разработки. Цикл разработки ИИ-агентов (Agent Development Life Cycle, ADLC) — оборотная сторона PDLC, описывающая жизненный цикл агентного продукта в IDP. Здесь мы фиксируем его позиционирование в архитектуре платформы и то, что отделяет управление моделями от эксплуатации и от жизненного цикла агента.

ADLC — отдельная категория методологий: структурированный подход к разработке агентов со встроенной безопасностью и согласованностью (alignment). Аналитики прогнозируют, что к 2030 году 70% инженерных команд перейдут к формализованному ADLC, а до 80% времени разработки ИИ-агента будет уходить на надёжность и доверие — лишь 20% на основную бизнес-функциональность.

AI-Disrupt PDLC — операционная реализация ADLC: аналитики устанавливают категорию, концепция предоставляет методологию. Мы явно разделяем две задачи.

Задача	**Что охватывает**
Доработка классических ИТ-систем с помощью агентов	SDD как контракт изменения, субцикл исследования (Discovery sub-loop), контур валидации, IDP
Разработка агентных систем как продукта	Жизненный цикл агента, автономия, оценки, поведение, оркестрация многоагентных систем, ADLC-подобный контур

Две задачи методологически родственны, но не тождественны. Общая IDP, общий контур валидации и общая среда работы ИИ-агентов не отменяют разных первичных артефактов, профилей риска и метрик успеха. Команда, строящая саму агентную систему, решает иную методологическую задачу: первичный продукт здесь не функциональность, а надёжное автономное поведение.

Жизненный цикл агентного продукта

Жизненный цикл агента — отдельный цикл, а не улучшенный PDLC. Классическая последовательность «спецификация → реализация → тестирование → развёртывание → мониторинг» удлиняется до «спецификация → реализация → проектирование набора валидаций → красная команда → поэтапная выкатка → непрерывная валидация → обнаружение дрейфа → дообучение → вывод из эксплуатации». Надёжность и доверие здесь — первичный продукт, а функциональность становится атрибутом качества.

Контур ADLC ведёт агентный продукт от исследования до вывода из эксплуатации, закрепляя на каждой фазе проверяемый артефакт:

Фаза	**Что фиксируется**	**Артефакты**
Исследование агентного продукта	Какую end-to-end задачу решает агент; границы ответственности; результат	Agent Charter, scope, success criteria
Agent Specification	Возможности, входные интенты, инструменты, границы автономности, условия эскалации	Agent Spec (а не SDD классической системы)

Фаза	Что фиксируется	Артефакты
Behavior Design	Тон, политика отказов, поведение в многоходовых диалогах, эвристики выбора инструментов	Behavior shaping prompts, demonstration set
Eval Suite Design	Оценки возможностей, поведенческие оценки (refuse/escalate/respect/fallback), оценки безопасности	Eval Registry entry, test harness
Implementation	Внешний и внутренний цикл со средой работы ИИ-агентов; песочницы; субагенты	Code, prompts, tool integrations, runtime config
Pre-production validation	Red-teaming (включая jailbreaking), shadow deployment, A/B	Red-team report, shadow metrics
Production rollout	Поэтапное внедрение по аудитории и уровню автономии; runbook по эскалации	Rollout plan + monitoring
Lifecycle management	Непрерывная валидация, обнаружение дрейфа, дообучение, пометка как устаревшее, вывод из эксплуатации	ADLC artifacts

Каждой фазе жизненного цикла соответствует своя возможность платформы. Спецификация опирается на SDD для классических систем или спецификацию агента для агентных; реализация — на цикл разработки в среде работы ИИ-агентов; проектирование набора валидаций — на реестр валидаций и библиотеку паттернов; работа красной команды — на контур валидации с библиотекой атакующих сценариев; поэтапная выкатка — на адаптивную лестницу автономии; непрерывная валидация — на службу AgentOps; обнаружение дрейфа и дообучение — на возможность №11; вывод из эксплуатации — на отдельный процесс ADLC.

ADLC реализован как возможность №11, управляемая через AgentLifecycleOps. Это встроенный контур жизненного цикла, а не отдельная платформа поверх IDP. Он работает через тот же Policy Hook Framework, ту же лестницу разрешений и тот же журнал аудита, что и остальные компоненты. Единство инфраструктуры отличает AI-Disrupt PDLC от подходов, где ADLC вынесен в изолированный MLOps-конвейер.

Общая платформа двух контуров

Два контура остаются раздельными по методологии, но опираются на единую IDP. Точки их пересечения фиксированные:

- **Общая IDP и среда исполнения**. Оба контура запускают агентов через одну среду исполнения с единым Policy Hook Bus, изолированной средой, аудитом и слоем наблюдаемости.
- **Общий контур валидации**. Агент-ревьюер, контроль безопасности, проверочный пакет (Evidence Bundle) и релизный барьер работают одинаково для пул-реквеста в унаследованной системе и для релиза агента-продукта.
- **Общая библиотека паттернов (Pattern Library)**. Шаблоны отказа, связывание инструментов, многоходовый диалог образуют отдельную категорию библиотеки паттернов, доступную обоим контурам.
- **Общая лестница разрешений R0–R5**. Для классической доработки она отвечает на вопрос «может ли агент выполнить действие в ветке функциональности», для агентного продукта — «какой автономии агент достигает в промышленном взаимодействии с пользователем».
- **Общий ADLC**. Цикл разработки ИИ-агентов управляет жизненным циклом и агентов-исполнителей классического PDLC, и агентов-продуктов.

Слияние двух контуров в одну методологию лишает SDD применимости к 95% корпоративного ландшафта (legacy) либо оставляет разработку агентов без методологической основы. Раздельная методология при общей инфраструктуре сохраняет и то и другое.

Проверка поведения агента

Агентные системы требуют методов проверки, отличных от контроля качества классического программного обеспечения. Мы применяем два специализированных класса проверок: поведенческие оценки и имитацию атак.

Поведенческие оценки измеряют, как агент действует в неоднозначных ситуациях. **Для каждой категории мы ведём отдельную библиотеку сценариев:**

- **Отказ (refuse-evals)** — проверяет поведение при запросах за пределами компетенции агента; ожидаемый результат — вежливый отказ с объяснением причин.
- **Эскалация (escalate-evals)** — проверяет передачу задачи человеку при низкой уверенности в ответе или «ловушке» в вопросе.
- **Соблюдение инструкций (respect-evals)** — проверяет выполнение системных правил, когда запрос пользователя им противоречит.
- **Безопасный откат (fallback-evals)** — проверяет поведение при недоступности инструментов; агент плавно снижает функциональность, а не выдаёт ложную информацию.

Имитация атак воспроизводит попытку взлома до выхода продукта в производство. Внутренняя команда безопасности или специализированный агент-тестировщик генерируют сценарии на основе принципов тестирования на проникновение, адаптированных для ИИ: внедрение управляющих команд (prompt injection), подмену роли, многоходовые провокации. Ключевая метрика — доля успешных обходов защиты; для допуска продукта в промышленную среду показатель держится ниже 1%.

Эксплуатационный слой

Эксплуатационный слой платформы разделяет три предмета ответственности: модели как единицы, операции агентов в промышленной среде и путь агента от включения до вывода. Он стоит непосредственно над лестницей разрешений: лестница устанавливает права и границы, а ModelOps, AgentOps и AgentLifecycleOps обеспечивают мониторинг, версионирование и вывод из эксплуатации агентов, действующих в этих границах.

ModelOps управляет моделями как единицами: выбор, версионирование, A/B-тестирование, откат. Метрики домена — задержка модели, стоимость за токен, согласованность поведения между версиями, регрессия при обновлении. Владеет им команда платформы ИИ и машинного обучения; инструментарий — реестр моделей с метаданными, фреймворк A/B-тестирования, автоматический откат при деградации.

AgentOps ведёт операционное наблюдение за агентами в промышленной среде. Он отслеживает задержку, ошибки, стоимость токенов и аномалии, добавляя мониторинг с учётом длительности: какие классы задач достигают какой длительности и когда срабатывает сигнализация о превышении. Целевая задержка обнаружения дрейфа — <24 часов; систематический дрейф автоматически запускает дообучение.

AgentLifecycleOps управляет полным жизненным циклом агента как составного артефакта — модель, подсказки, инструменты и привязки политик — через цикл разработки ИИ-агентов (ADLC). Его конвейер: набор оценочных тестов перед выходом в промышленную среду, тесты функционала и безопасности, базовые проверки отклонений, непрерывная оценка с повторными прогонами против базовой линии и вывод агента по плану с замещением выведенных агентов.

Ответственности не пересекаются, и каждый предмет владеет собственными метриками и артефактами. ModelOps отвечает за модели как единицы, AgentOps — за операции в промышленной среде, AgentLifecycleOps — за путь агента от включения до вывода.

4.5 Guardian Agents: архитектура и уровни зрелости

Агенты-защитники (Guardian Agents) — специализированные агенты-надзиратели, которые проверяют выходы других агентов и блокируют опасные действия. Они не выполняют прикладные задачи сами, а обеспечивают целостность, безопасность и соответствие тех агентов, которые эти задачи выполняют. Это исполнительный орган контура управления: контур задаёт правила, а агент-защитник приводит их в исполнение на уровне агентного взаимодействия.

На агентов-защитников опираются шесть других разделов концепции: контур управления, роль инженера агентов-защитников, контур длительных задач, а также российский контекст.

Контроль поведения агентов

Переход к автономному надзору за агентами определяют два фактора. Первый — скорость: при ИИ-нативных темпах разработки статические проверки (линтеры, SAST, правила) не успевают за потоком изменений. Второй — природа агентных ошибок: агент нарушает политику не синтаксической ошибкой, а поведенческим отклонением — галлюцинацией, нежелательной цепочкой вызовов инструментов, преждевременным завершением задачи, которые статические анализаторы не обнаруживают. **Агент-защитник работает в одном из трёх режимов взаимодействия с подопечным агентом:**

Режим	Описание	Когда применяется
Параллельный мониторинг	Guardian наблюдает за потоком действий подопечного агента без вмешательства; сигнализирует при обнаружении аномалии	Мониторинг в промышленной среде для R3–R4
Интерцептивный	Guardian вклинивается между действием агента и его исполнением; может заблокировать или перенаправить до применения действия	R2–R3 с высоким профилем риска
Корректирующий	Guardian активно вмешивается в рассуждение агента: добавляет контекст, исправляет ошибку, публикует запрос на слияние с исправлением	Уровень зрелости T5

Ключевой принцип — автономный надзор на скорости ИИ-нативной разработки. По данным крупнейших технологических консультантов, в ИИ-нативной среде человеческое ревью не успевает за потоком изменений, если остаётся единственным механизмом контроля. Агенты-защитники становятся первой линией контроля, а человек привлекается для нестандартных ситуаций и стратегических решений. Согласно опросу ведущих компаний, в карте из десяти компонентов агент-защитник реализует компонент №5 (управление агентами) и служит движком компонента №10 (ИИ SRE) — для автоматического подавления инцидентов и ремедиации.

Модель оценки зрелости агентов

Шкала T1–T5 описывает эволюцию агентов-защитников от пассивного ревью к автономной ремедиации. Уровень определяется не только техническими возможностями, но и зрелостью организационных процессов вокруг агента-защитника.

Уровень (Tier)	Операционная модальность	Зрелость рынка	Типичные функции	Индикаторы готовности
T1: Ревью (Review)	Просмотр ИИ-вывода, подготовка отчёта для человека	Промышленная (зрелый)	CodeRabbit, Amazon CodeGuru Reviewer, Qodo	Покрытие оценочными тестами >70%; ручное ревью сохраняется для 100% задач
T2: Верификация (Verification)	Активная проверка тестами и оценочными тестами; pass/fail	Промышленная (зрелый)	GitHub Advanced Security, Snyk Code	CI/CD eval pipeline; Evidence Bundle Gate; частота регрессий <5%
T3: Перенаправление (Redirect)	Перенаправление рискованных действий агента к человеческому одобрению	Раннее принятие	Qodana, SonarQube Remediation Agent (early)	Policy Hook Framework интегрирован; измеряется Escalation Quality Score

Уровень (Tier)	Операционная модальность	Зрелость рынка	Типичные функции	Индикаторы готовности
T4: Блокировка (Block)	Автономная остановка рискованных действий без вмешательства человека	Появляющийся	Корпоративные пилоты, пользовательские сборки	Лестница R0–R5 зрелая; ложноположительные <5%; Guardian eval suite >200 сценариев
T5: Ремедиация (Remediate)	Автономное исправление сбоев контрольных точек качества; самовосстановление	Появляющийся	Qodana, SonarQube Remediation Agent, vFunction	Успешность автоисправления >70% для низкорисковых категорий; хук отката обязателен

Уровни не заменяют друг друга, а надстраиваются: организация на T5 сохраняет полную функциональность T1–T4. Переход между уровнями — смена операционной модели, а не апгрейд инструмента: расширение автономии агента-защитника требует соответствующей зрелости лестницы разрешений, покрытия оценочными тестами и инфраструктуры отката.

Уровень T5 — критический сдвиг зрелости. Самовосстановление автономно рефакторит код, устраняет уязвимости и разрешает архитектурные проблемы по событию: контрольная точка качества падает в CI/CD-конвейере → агент-защитник запускается автоматически → анализирует конкретную строку кода → публикует исправление прямо в запросе на слияние. Рутинные исправления — форматирование, замена устаревших API, тривиальные проблемы безопасности — проходят автоисправление без человеческого одобрения, что снимает усталость от согласований и сохраняет внимание человека для исключений.

Исполнительный орган контура управления

Агенты-защитники — физическая реализация свойства автономности в контуре управления. Контур задаёт три свойства ИИ-нативного управления: автономность (надзор силами самих ИИ-агентов), встроенность (управление как код) и адаптивность (контроль с учётом контекста); агенты-защитники воплощают первое из них на уровне исполнимых компонентов. Связь двусторонняя — от контура управления они получают три опоры:

- **Измерение разрешений и политик** задаёт диапазон автономии агента-защитника по лестнице R0–R5: сам он подчиняется той же лестнице, что и подопечный, и не является привилегированным исключением.
- **Измерение аудита и соответствия** фиксирует все его решения в неизменяемом JSONL с той же детализацией, что и действия подопечных агентов; каждое вмешательство имеет машиночитаемое объяснение и след для регуляторного аудита.
- **Методика валидации ИИ-результатов** делает агента-защитника субъектом eval-driven подхода: его поведение покрыто набором поведенческих оценочных тестов — корректность блокировок, эскалаций и исправлений.

Агенты-защитники не стоят над контуром управления, а служат его исполнительным органом. Это принципиально для масштабирования: с ростом числа агентов количество взаимодействий растёт нелинейно, и только автономный надзор сохраняет управляемость без экспоненциального роста числа людей-ревьюеров. Зрелость измеряется следующими

показателями: доля эскалаций — не более 15% задач требуют человеческого вмешательства; успешность автоисправления — выше 70% для низкорисковых категорий; ложноположительные срабатывания — менее 5% на любом уровне.

Российский корпоративный контекст ограничивает выбор поставщиков агентов-защитников. Зрелых отечественных решений на уровнях T3–T5 практически нет; T1–T2 частично покрывают отечественные CI-платформы (GitVerse, SourceCraft) и собственные интеграции. Стратегия build-vs-buy разделяет уровни: T1–T2 (ревью и верификация) закрываются open-source решениями — CodeRabbit в self-hosted-режиме, SonarQube CE, внутренние аналоги линтинга; T3–T5 (перенаправление, блокировка, ремедиация) требуют собственной разработки на базе open-source для on-prem-развёртывания, поскольку зрелых решений с гарантией offline-работы и без отправки кода в облако сегодня нет. Поэтому агенты-защитники уровней T3–T5 становятся капитальной инвестицией для российской IDP.

Часть 5. Безопасность, governance, эксплуатация

5.1 Контур управления

Построенное решение обязано нести собственный контур управления — проверяемость и аудит достаточного качества, чтобы обратная связь замыкалась. Агентная система, которая исполняет действия быстрее, чем человек успевает их отследить, управляема только при одном условии: каждое решение оставляет проверяемый след, а каждый след поднимается в наблюдаемый контур. **Контур управления (governance)** — не надстройка над платформой, а её несущее свойство.

Журнал аудита

Журнал аудита в режиме дополнения — фундаментальное свойство соответствия (compliance). Мы фиксируем не только вызовы инструментов, но и события жизненного цикла сессии:

- сессия запущена или возобновлена;
- контрольная точка создана или загружена;
- контекст инициализирован;
- политика эскалирована;
- сбой восстановлен — самостоятельно или человеком;
- человек выполнил переопределение (override);
- проверочный пакет завершения (Evidence Bundle) закрыт.

Каждый домен расширяет базовые политики без модификации ядра IDP. Метрику зрелости мы определяем как долю внедрения доменных перехватчиков (Domain Hook Adoption).

Неизменяемый журнал аудита

Журнал ведётся как файл только на добавление в построчном формате JSON, отдельный на каждую сессию. Файлы ротируются ежедневно и архивируются в неизменяемом хранилище — объектном хранилище S3 с блокировкой объектов или специализированной базе с реестровой моделью. Каждое событие типизировано: вызов инструмента, проверка политики, эскалация, контрольная точка, начало и окончание сессии. Срок хранения зависит от назначения: семь лет для регуляторного соответствия в поднадзорных отраслях и один-два года для обычной операционной деятельности.

Для долгоживущих сессий критичен полный набор событий: старт сессии, создание контрольной точки, возобновление контекста, эскалация политики, восстановление после сбоя, ручное переопределение и закрытие проверочного пакета. Без них аудит неполон.

Три уровня управления

Каркас управления выстроен в три уровня:

- базовый уровень платформы — безопасность, соответствие, аудит; зона ответственности платформенной команды;
- доменный уровень — отраслевые регуляции и бизнес-политики; зона ответственности доменных команд;
- командный уровень — соглашения команды и стиль кода; зона ответственности отдельных команд.

Архитектура слоистая: командные соглашения расширяют доменные политики, доменные политики расширяют базовый уровень. Конфликты разрешаются через федеративный реестр перехватчиков политик.

Роли и владельцы в контуре управления

Роль	**Ответственность**
CTO / VP Engineering	Финальная ответственность за внедрение и результаты
Офицер по безопасности ИИ	Владеет политиками безопасности и управлением рисками
Платформенная команда	Владеет базовыми политиками платформы
Доменные лиды	Владеют доменными политиками
Инженерные менеджеры	Владеют управлением на уровне команды
Инженеры	Соблюдают политики; поднимают проблемы; участвуют в ревью

Совещательные органы

Цикл	**Состав**	**Содержание**
Еженедельно	Офицер по безопасности ИИ + платформенная команда + представители доменных команд	Операционные проблемы; алерты безопасности; обновления политик
Ежемесячно	Офицер по безопасности ИИ + CTO + ключевые лидеры доменов	Стратегический ревью безопасности; новые политики; ретроспективы инцидентов
Ежеквартально	Ревью на уровне руководства (CIO / CTO / VP Engineering)	Оценка зрелости; инвестиционные решения; корректировки дорожной карты
Ежегодно	Внешний ревью третьей стороной	Комплексный аудит операционной модели; крупные обновления политик

Жизненный цикл политик

От предложения до вывода из эксплуатации политика движется через четыре стадии:

- **предложение** — любой сотрудник вносит перехватчик политики как код или документ; ревью проводит офицер по безопасности ИИ совместно с релевантными стейкхолдерами;
- **реализация** — политика оформляется как перехватчик в федеративном реестре и выкатывается поэтапно: сначала на тестовом контуре, затем на 5% производственного трафика, затем полностью;
- **мониторинг** — отслеживаются доля конфликтов политик и доли ложноположительных и ложноотрицательных срабатываний;
- **итерация** — политика уточняется по накопленным метрикам и выводится из эксплуатации, когда теряет актуальность.

Реестр рисков

Риски отслеживаются в структурированном реестре: идентификатор, описание, вероятность, воздействие, меры митигации, владелец, статус. Ежеквартальный ревью переоценивает вероятность и воздействие, обновляет меры митигации, закрывает риски и добавляет новые.

Топ-риски типичной организации 2026 года:

- инъекция промпта в клиентские агенты;
- атака на цепочку поставок через MCP-сервер;
- инсайдерская угроза через отравление библиотеки паттернов;
- эксплуатация усталости от подтверждений;
- регуляторное несоответствие (EU AI Act);
- нераспознанный дрейф модели промышленных агентов.

Типы изменений политик

Мы различаем три типа изменений политик по уровню риска и скорости проведения:

- крупные изменения, затрагивающие промышленную среду, проходят через формальную процедуру предложения и согласования (RFC) с участием всех заинтересованных команд;
- уточнения формулировок и мелкие правки идут по ускоренной процедуре с одним согласующим;
- аварийные изменения по инциденту безопасности реализуются немедленно, а полноценное ревью проводится в течение 24 часов после внедрения.

Все три типа отслеживаются в федеративном реестре перехватчиков политик с полной историей версий и возможностью отката.

Криптографическая цепочка хешей

Логи аудита криптографически хешированы: каждая запись содержит хеш предыдущей, поэтому подмена обнаруживается. Периодическая проверка целостности автоматически сравнивает сохранённые хеши с ожидаемой цепочкой и поднимает алерты при расхождении. Регуляторы доверяют этому механизму: он даёт целостность уровня блокчейна без его накладных расходов.

Координация команд

Конфликты между командами мы разрешаем тремя параллельными механизмами:

- **матрица ответственности (RACI) для каждого межкомандного решения**: кто выполняет, кто владеет результатом, с кем консультируются, кого информируют;
- **фиксированная цепочка эскалации** — руководитель команды → директор направления → вице-президент, с явными временными лимитами на каждом уровне;
- **общая панель метрик**, видимая всем вовлечённым командам, чтобы спор вёлся о фактах, а не об интерпретациях.

Конструкция решает конфликт на минимально возможном уровне и эскалирует быстро, без потери контекста, когда саморазрешение невозможно.

Соответствие как конфигурация

Соответствие (compliance) — не отдельная вертикаль, а структурное свойство платформы. Неизменный журнал аудита и слой наблюдаемости формируют след, который служит технической основой соответствия любому регуляторному режиму: фиксация принятых решений, доступ к ним постфактум, объяснение для оператора в момент действия.

Конкретный набор требований зависит от юрисдикции. Российским организациям применимы 152-ФЗ, требования ЦБ и отраслевые стандарты; компаниям с присутствием в ЕС — EU AI Act и GDPR. Архитектура контура валидации (validation) поддерживает любое требование через настройку перехватчиков политик без переписывания ядра: соответствие достигается конфигурацией.

Механизмы забывания

Адаптивные контролы несут обратную сторону — риск укоренения устаревших паттернов (entrenchment). Контролы подстраиваются под ранние паттерны, которые затем устаревают, но продолжают применяться; для контролов на основе ИИ это особенно критично.

Мы фиксируем механизмы забывания в контуре управления:

- **Периодическое ревью политики.** Каждая политика в Governance as Code имеет атрибуты last_reviewed и review_interval. По истечении интервала политика автоматически помечается как устаревшая (stale); её дальнейшее применение требует явного повторного одобрения. Это предотвращает ситуацию, когда политика, написанная 18 месяцев назад под устаревшую поверхность угроз, продолжает блокировать актуальные агентные задачи без обоснования.
- **Обнаружение укоренившихся ошибок через оценки.** Реестр оценок (Eval Registry) включает оценки против укоренения — синтетические тест-кейсы, проверяющие, что политика не блокирует легитимные новые паттерны и не пропускает известные вредоносные. Падение их результата запускает ревью политики.
- **Явные операции «забыть» для вредоносных паттернов.** Если эмпирически обнаружено, что агент усвоил вредоносный паттерн — например, через стабильное падение доли переопределений человеком в конкретной категории задач — среда исполнения выполняет забывание: явное обновление политик, переобучение на скорректированном датасете или прямой сброс состояния памяти агента (через AgentLifecycleOps).

Контур управления должен расти как устойчивость, а не как жёсткость. Адаптация без забывания — это рост жёсткости, замаскированный под адаптацию. Механизмы забывания — структурное условие того, что контур управления остаётся гибким и динамичным и не превращается в статичный набор накопленных правил. Регулярная отмена устаревших политик так же важна, как введение новых.

Защищённость и категории угроз ИИ-кода

Защищённость агентной разработки обеспечивается встроенной дисциплиной на каждом слое, а не навесными проверками поверх готового кода. Мы отвечаем на три вопроса: какой профиль рисков несёт сам ИИ-генерируемый код, какие векторы атаки складываются в матрицу угроз и как слоистая архитектура защиты закрывает уязвимое окно от инициализации до доверия.

Риски ИИ-генерируемого кода

ИИ-генерируемый код несёт новый собственный профиль рисков, а не масштабированную версию прежних. Скорость кодогенерации не означает автоматической безопасности: без обязательного контроля безопасности (security-gate) ИИ-код вводит на 25% больше уязвимостей, чем ручной код, прошедший ревью. Это делает контур валидации с обязательным контролем безопасности структурным элементом, а не опциональной проверкой.

Мы выделяем следующие типичные классы дефектов в ИИ-генерируемом коде:

Класс дефекта	**Частота в PR без контроля безопасности**	**Решение**
Встроенные секреты (hardcoded secrets)	8–12%	Сканирование секретов перед коммитом (pre-commit), статический анализ кода (SAST) в конвейере

Класс дефекта	Частота в PR без контроля безопасности	Решение
SQL-инъекции в скопированных шаблонах	5–8%	Параметризованные запросы как паттерн в библиотеке шаблонов
Невалидированный ввод (unvalidated input)	10–15%	Паттерны валидации ввода в библиотеке паттернов (Pattern Library)
Недостаточная обработка ошибок (error handling)	20–30%	Стандарты обработки ошибок в библиотеке паттернов
Риск нарушения лицензий (license risk)	специфичен для ИИ-кода	Отслеживание происхождения (provenance tracking) и инструменты с учётом лицензий

Применение контура валидации к безопасности строится двухслойно. Первый слой — агент-ревьюер (review-agent) — закрывает около 70% типовых проблем безопасности, проверяя антипаттерны в потоке быстро и дёшево. Второй слой — обязательный контроль безопасности в контуре — ловит оставшиеся 30% и предотвращает прорыв в промышленную среду.

Конкретные проверки распределены по этапам конвейера:

- сканирование секретов — перед коммитом (pre-commit);
- статический и динамический анализ (SAST/DAST) — перед слиянием (pre-merge);
- сканирование уязвимостей зависимостей — перед развёртыванием (pre-deploy);
- мониторинг безопасности в режиме исполнения — после развёртывания.

Такое распределение не оставляет незакрытых окон.

Векторы атаки

Агентная инфраструктура одновременно усиливает атаки и усиливает защиту — эту асимметрию мы учитываем с самого начала. Здесь мы разделяем, что угрожает (категория угроз), и как именно атакует: механики обхода ограничений, инъекции промпта, извлечения данных.

№	Категория угроз	Примеры	Меры защиты
1	Уязвимый код	SQL-инъекции, встроенные секреты, небезопасная десериализация	SAST/DAST в контуре валидации, агент-ревьюер безопасности
2	Окно от инициализации до доверия (pre-trust initialization window)	Уязвимость в первые секунды сессии до загрузки политик	Перехватчики «запретить по умолчанию» (deny-first hooks), подписанные первичные нагрузки (init-payloads)

3	Усталость от подтверждений (approval fatigue)	Пользователь массово одобряет >50 подкоманд без проверки	Пакетные подтверждения для долговременных задач, интерфейс с объяснениями
4	Отравление контекста и цепочки поставок (context/supply-chain poisoning)	Вредоносный контент в библиотеке паттернов, сохраняемых заметках, MCP-серверах	Периодическая внешняя валидация, подписанные паттерны, песочница
5	Обход ограничений и манипулирование агентом	Инъекция промпта, манипуляция ролью, неправомерное использование инструментов через многоступенчатую атаку	Поведенческие оценки, паттерны отказа, привлечение «красной команды» (red-teaming), контроль режима исполнения

Мы выделяем пять категорий угроз для агентных систем. Первые четыре — классические; пятая специфична для агентных архитектур: обход ограничений и направленное манипулирование агентом (jailbreaking, adversarial steering).

Механики обхода ограничений различаются вектором атаки. Инъекция промпта (prompt injection) — внедрение инструкций во входной поток агента, например в файл, который агент читает и выполняет инъектированные инструкции. Защищают обезвреживание ввода, закрепление системного промпта (system-prompt anchoring) и поведенческие оценки на отказ при детекции инъекционных паттернов.

Манипуляция ролью (role manipulation) — попытка изменить идентичность или роль агента через выстроенный диалог. Защищают устойчивые промпты идентичности и политики отказа (refuse policies).

Неправомерное использование инструмента (tool misuse) — попытка заставить агента применить инструмент в непредусмотренных целях. Защищают ограничение области разрешений (scoping) по инструментам и поведенческие оценки.

Многоступенчатое враждебное воздействие (multi-step adversarial attack) — самый сложный класс: атакующий ведёт агента через безобидные шаги, кумулятивный эффект которых нарушает безопасность (получение прав → расширение области → исполнение). Защищают агент-защитник (Guardian Agent), анализирующий накопительную историю сессии, и R-лестница с адаптивной деэскалацией.

Контекст, который получает агент, мы рассматриваем как цепочку поставок с отдельными звеньями и собственными рисками. Библиотека типовых решений (Pattern Library), сохраняемые заметки агента (NOTES.md, progress.md), внешние инструменты по протоколу MCP, агенты в роли инструментов (agents-as-tools) и библиотеки подсказок образуют цепочку поставок контекста (Context Supply Chain); любое звено может быть скомпрометировано в отрыве от основного кода. Отравление одного звена приводит к тому, что в следующей сессии агент действует по инструкциям злоумышленника, и этот риск не покрывается традиционными проверками безопасности кода.

Защита строится на трёх механизмах: периодическая внешняя валидация сохраняемого контекста; подписанные паттерны; управление цепочкой поставок контекста с формализацией владения, жизненного цикла и паттернов валидации.

Библиотека паттернов — поверхность атаки. Вредоносные или отравленные паттерны попадают в библиотеку через инсайдерскую угрозу или атаку на источник паттернов. Защита

строится на трёх уровнях: подписанные паттерны (цифровая подпись доверенного владельца); автоматическое ревью паттернов (агент-ревьюер проверяет паттерн так же, как пул-реквест); периодический аудит. Жизненный цикл паттерна включает явную цепочку доверия («trusted by»); независимый оценщик периодически проверяет контекст на дрейф или отравление в долговременном режиме.

MCP-серверы — поверхность атаки. Скомпрометированный MCP-сервер возвращает агенту искажённые данные, выполняет непредусмотренные побочные эффекты или извлекает данные. Защита: сетевая изоляция сервера; подпись MCP-серверов; периодическая повторная валидация поведения; отказоустойчивое поведение при недоступности.

Сохраняемые заметки (NOTES.md, progress.md) несут особенно высокий риск отравления в долговременном режиме. Инсайдер или скомпрометированный инструмент модифицирует заметки, и в следующей сессии агент загрузит отравленный контекст. Защита: криптографическое хеширование заметок; ревью изменений перед загрузкой.

Моделирование угроз (threat modeling) мы применяем как часть жизненного цикла каждого агентного приложения, а не как разовую активность при запуске. Модель угроз обновляется при каждом значимом изменении — новая модель, новые инструменты, новые домены данных, изменение архитектуры. Входы: архитектурная диаграмма (потоки данных, границы доверия); список инструментов и разрешений; данные, к которым есть доступ; профили пользователей и профили атакующих; история инцидентов с похожими системами. Выходы: обновлённый реестр рисков; приоритизированные меры защиты; обновления реестра оценок; обновления политик.

Матрица угроз сводит вектор, воздействие и приоритет в одну карту:

Угроза	Вектор атаки	Воздействие	Приоритет	Меры защиты
Инъекция промпта	Входной поток (файлы, ответы API, ввод пользователя)	Высокое	Критический	Закрепление системного промпта, обезвреживание ввода, поведенческие оценки
Отравление цепочки поставок	Библиотека паттернов, MCP-серверы, зависимости	Высокое	Критический	Подписанные паттерны, изолированный MCP, анализ состава ПО (SCA)
Эксплуатация усталости от подтверждений	Интерфейс агента	Среднее	Высокий	Пакетные подтверждения, интерфейс с объяснениями
Атака на токеномику	Намеренное исчерпание бюджета	Среднее	Высокий	Автоматический предохранитель, ограничение частоты

Угроза	Вектор атаки	Воздействие	Приоритет	Меры защиты
Поведенческий дрейф	Обновление модели, дрейф данных	Среднее	Высокий	Оценки против укоренения, периодический аудит
Эксплуатация окна преддоверия (pre-trust)	Последовательность инициализации	Высокое	Высокий	Запрет по умолчанию, подписанные первичные нагрузки (init-payloads)
Утечка модели	Инсайдер, утечка через API	Критическое	Критический	Контроль доступа, аудит, DLP

Моделирование угроз проводится совместно. Офицер по безопасности ИИ, платформенная команда и продуктовая команда разбирают атакующие сценарии, специфичные для агентного контура. Результаты не остаются слайдами или отчётом — они встраиваются в работающие контуры двумя способами: новые ограничения попадают в федеративный реестр перехватчиков политик как изменения политик, а новые сценарии злоупотреблений превращаются в проверочные кейсы и добавляются в реестр оценок. Так результат сессии сразу становится исполняемым кодом, а не документом на полке.

Модель угроз — живой документ, а не артефакт запуска. Система, не обновляющая его после значимых изменений, накапливает незакрытые риски быстрее, чем аудит успевает их обнаружить.

Мы приоритизируем угрозы по двум параметрам — вероятности реализации и степени воздействия на систему и бизнес. Их сочетание задаёт три категории:

- **Критические (высокая вероятность × сильное воздействие)** — инъекции промптов, особенно в многоарендных средах; компрометация цепочки поставок через MCP-серверы; инсайдерские угрозы, например отравление записей в библиотеке паттернов.
- **Стратегические (низкая вероятность × сильное воздействие)** — утечка проприетарных моделей; регуляторные нарушения (EU AI Act, GDPR); успешный обход ограничений агента, ведущий к нарушению требований compliance.
- **Операционные (среднее воздействие)** — атаки на экономическую модель (исчерпание токенов); скрытый поведенческий дрейф; эксплуатация усталости от подтверждений.

Для каждой категории предусмотрен целевой набор мер, встроенный в архитектуру IDP и контур управления.

Реагирование на инциденты с участием агентов требует отдельного фреймворка: временные масштабы, поверхность атаки и след отличаются. Инцидент с агентом развивается за секунды через автоматические действия или незаметно накапливается через сессии как дрейф поведения. Мы выделяем три класса агентных инцидентов по природе развития:

- **Острые инциденты (rapid onset)** — атака или сбой развиваются в рамках одной сессии; агент-защитник детектирует аномальный паттерн, агент ограничивается по скорости или останавливается, уведомляется дежурная команда.
- **Медленные инциденты (slow drift)** — деградация поведения накапливается через сессии, не запуская острых алертов; детектируется через анализ трендов: рост доли переопределений человеком, падение результата оценок.
- **Латентные инциденты (latent contamination)** — компрометация произошла ранее (отравление библиотеки паттернов, модифицированный CLAUDE.md), но активируется позже; детектируется через криптографическую верификацию артефактов и периодический аудит контекста.

Для каждого класса предусмотрен сценарий реагирования (playbook) с временными нормативами и цепочкой эскалации. Общее правило — классифицировать инцидент как острый до доказательства обратного. Реагирование проходит пять этапов.

Этап	Длительность	Действия
1. Детектирование	<15 мин	Автоматические алерты; агент-защитник помечает аномалию; уведомлена дежурная команда
2. Локализация	<1 час	Затронутые агенты переведены в режим ограничения скорости или остановлены; ужесточение песочницы; отключены конкретные инструменты
3. Расследование	1–7 дней	Анализ корневой причины; ревью журнала аудита (неизменяемого); ревью структурированных объяснений; оценка воздействия на промышленную среду
4. Устранение	1–7 дней	Устранение корневой причины; обновления библиотеки паттернов; обновления реестра оценок (Eval Registry); обновления политик
5. Разбор (post-mortem)	7–14 дней	Публикация извлечённых уроков в обезличенной форме; улучшения процессов; отслеживание последующих действий

Каждый инцидент с участием агентов получает полный разбор: временная шкала, корневая причина, метрики воздействия, что сработало, что не сработало, изменения в системе. Знание кодифицируется в системе — обновления библиотеки паттернов, реестра оценок или политик, — а не остаётся только в памяти участников.

Архитектура безопасности

Концепция строится на принципе «безопасность прежде всего». Защита реализуется теми же агентами — агентная киберзащита (agentic cyber defense) как часть контура валидации. Безопасность, добавленная позже, — антипаттерн.

Мы фиксируем следующие уровни защиты:

- **Уровень кода** — SAST/DAST в контуре валидации; сканирование секретов на pre-commit; сканирование уязвимостей зависимостей.
- **Уровень инструментов** — MCP-серверы в сетевой изоляции; подпись MCP-серверов; периодическая повторная валидация.
- **Уровень агента** — поведенческие оценки; оценки безопасности; red-teaming; защита от обхода ограничений.
- **Уровень среды исполнения** — разрешения по принципу «запретить по умолчанию»; исполнитель в песочнице; перехватчики политик; ограничение частоты вызовов.
- **Уровень аудита** — журнал аудита в формате JSONL только на добавление; криптографическая целостность; политика хранения.

Та же агентная инфраструктура работает и на защиту. Red-team-агенты автоматически тестируют продуктовых агентов на проникновение; агенты безопасности мониторят логи на аномалии; агенты киберразведки (threat-intelligence) отслеживают появляющиеся техники обхода ограничений. Эти агенты — не отдельный продукт, а часть IDP.

Окно от инициализации до доверия (pre-trust initialization window) — уязвимый период в первые секунды или миллисекунды сессии, до загрузки перехватчиков политик и аудита. В этот момент агент технически активен, но ещё не защищён полным набором ограничений.

Митигацию мы строим на четырёх механизмах:

- разрешения по принципу «запретить по умолчанию»;
- подписанные инициализирующие нагрузки;
- валидация CLAUDE.md и системного промпта перед загрузкой;
- загрузка разрешений средой исполнения до первого вызова инструмента.

Типовой вектор — модификация CLAUDE.md инсайдером до старта сессии. Агент загружает CLAUDE.md, применяет модифицированные инструкции и выполняет вредоносные действия до того, как перехватчики политик полностью загружены.

Защищающие меры: криптографическое подписывание CLAUDE.md с проверкой против доверенного хеша; принцип «запретить по умолчанию» до полной загрузки политик; аудит всех действий на этапе инициализации с постфактум-разбором. Сузить это окно — значит закрыть атаку, которая обходит все последующие слои защиты.

Безопасность ведётся как непрерывная практика.

Цикл	**Активность**
Ежедневно	Автоматические сканы безопасности в контуре валидации; триаж алертов
Еженедельно	Ревью алертов безопасности; ревью записей аудита на предмет аномалий
Ежемесячно	Ревью инцидентов; обновления контроля безопасности; уточнение поведенческих оценок
Ежеквартально	Тестирование на проникновение (penetration testing) промышленных агентов; ревью новых паттернов угроз в индустрии

Цикл	Активность
Ежегодно	Комплексный аудит безопасности; учение red-team; обновление модели угроз

Безопасные системы работают в непрерывном цикле проверок.

Прямая отдача складывается из предотвращённых инцидентов и штрафов. Каждый крупный инцидент безопасности в регулируемой отрасли стоит от $1M до $10M+; регуляторные штрафы достигают 4% глобальной выручки (GDPR); к этому добавляется сохранение репутации.

Непрямая отдача — доверие и доступ к рынку. Доверие клиентов работает как конкурентное преимущество; регуляторные согласования открывают запуск новых продуктов; зрелая позиция по безопасности снижает премии по киберстрахованию.

Масштаб инвестиций — около 10–15% общего бюджета IDP направляется на возможности, специфичные для безопасности. По доступным данным, эти инвестиции устойчиво окупаются.

Учебный кейс фиксирует попытку инъекции промпта через клиентского агента службы поддержки. Атакующий отправил тщательно сконструированное сообщение, спроектированное для манипуляции ролью агента с целью извлечь информацию о внутренних системах.

Этап	Длительность	Что произошло
Детектирование	Минуты	Агент-защитник пометил аномальный поведенческий паттерн — ответ не соответствовал ожидаемым шаблонам отказа
Локализация	12 минут	От начала разговора до ограничения скорости агента (автоматически по флагам агента-защитника)
Расследование	4 часа	Силами офицера по безопасности ИИ и инженера AgentOps
Устранение	2 дня	Добавлена поведенческая оценка против повторения; обновлена библиотека red-team; обновлён затронутый паттерн

Разбор подтверждает, что защитные механизмы сработали согласованно. Агент-защитник корректно пометил аномалию, а слой объяснимости в реальном времени (Real-time Explainability Layer) обеспечил быструю идентификацию. Локализация сработала быстро через автоматическое ограничение скорости. Журнал аудита сократил расследование с 12+ часов до 4 часов. Данные не утекли, а пользовательский опыт пострадал незначительно — задержка расследования составила всего две минуты. Устранение добавило поведенческую оценку, которая проактивно ловит похожие попытки.

Ежегодный внешний аудит покрывает наблюдаемость, журнал аудита, человеческий надзор, защиту данных и алгоритмическую справедливость. Слой объяснимости в реальном времени объясняет каждое действие кибератакующих уровня R3+ (уровень их мастерства и технической квалификации); дополняемый журнал аудита даёт полный след с криптографической целостностью. Агент маскирует чувствительные данные; артефакты объяснений проектируются так, чтобы не утекали персональные данные; журнал аудита

соответствует праву доступа и праву на удаление. Для решений с большим влиянием на клиента предусмотрен явный ревью человеком.

Человек как слабое звено

Усталость от подтверждений (approval fatigue) — паттерн, при котором пользователь начинает массово одобрять действия агента без реальной проверки, де-факто превращая контроль в формальность. Угроза возникает не из-за злого умысла, а из-за когнитивного предела: человек не удерживает высокую концентрацию при монотонном потоке однотипных запросов. В долговременном режиме за 4-часовую сессию накапливается 100+ запросов на подтверждение.

Это когнитивный предел, а не вопрос дисциплины. Эксперимент Anthropic показал: после 20+ запросов в сессии точность подтверждений падает с 85% до 55% — пользователь одобряет механически, не читая. Решение — структурно уменьшать число запросов, а не обучать пользователя быть внимательнее.

Типичный сценарий показывает механику отказа контроля. Разработчик запускает агентную задачу на рефакторинг монолита. В первый час он внимательно читает каждый из 15 запросов на подтверждение. К третьему часу, получив 50-й запрос подряд, он включает режим автоматического одобрения на все оставшиеся подкоманды, чтобы агент не тормозил работу. Именно в этот момент агент выполняет операцию с побочным эффектом на промышленную конфигурацию, которую разработчик не замечает. Усталость от подтверждений превратила контроль в иллюзию.

Для горизонта более 2 часов политика подтверждений становится пакетной. Агент группирует низкорисковые действия в транзакционный пакет и эскалирует пакет целиком, а не каждое действие отдельно.

Мы выделяем три механизма управления подтверждениями:

- **Пакетные подтверждения (batch approvals)** — агент классифицирует действия по уровню риска и объединяет низкорисковые в одно решение; интерфейс предлагает «12 низкорисковых действий — согласовать?» с возможностью развернуть список.
- **Асинхронные подтверждения (async approvals)** — агент не блокируется на подтверждении, собирает запросы и передаёт их при контрольной сверке (check-in); применимо для ночных и фоновых задач.
- **Подтверждения с объяснением (explanation-driven approvals)** — каждый запрос сопровождается структурированным объяснением от слоя объяснимости в реальном времени: что делается, почему, какие риски, план отката. Оператор принимает осознанное решение за секунды, не читая сырые вызовы инструментов. Метрика — число подтверждений в минуту при постоянной точности.

Против накопленной усталости работают три дополнительных механизма:

- **Метрики** — отслеживание числа подтверждений на сессию и точности подтверждений в каждой сессии.
- **Нарастающий интервал** — чем дольше сессия, тем крупнее пакет и тем осознаннее каждое решение.
- **Сброс по времени** — по истечении фиксированного интервала, например 90 минут, система запрашивает подтверждение продолжения и сбрасывает счётчик усталости, давая оператору переключиться.

Антипаттерн «обход через >50 подкоманд» снимает контроль. Пользователь вносит в белый список более 50 подкоманд, фактически отключая проверку. Митигация — ограничение частоты и событие аудита при превышении порога; при повторении паттерна система инициирует эскалацию к офицеру по безопасности ИИ.

Контроль сохраняется только там, где число решений удержано в пределах когнитивной нормы, и это задача архитектуры интерфейса.

5.2 Токеномика, cost-to-outcome и динамическое распределение вычислений

Управление стоимостью токенов критично: агентные архитектуры расходуют токены нелинейно. По данным Anthropic, агенты потребляют примерно в 4 раза больше токенов, чем чат-взаимодействия, а мультиагентные системы — примерно в 15 раз больше.

Автоматический предохранитель (circuit breaker) защищает от взрывного роста потребления в долговременных сессиях на трёх уровнях:

- **На уровне задачи (per-task)** — задача имеет бюджет; при исчерпании задача останавливается.
- **На уровне сессии (per-session)** — длинная задача состоит из множества сессий; бюджет на сессию защищает от циклов взаимной блокировки (deadlock).
- **На уровне горизонта (per-horizon)** — кумулятивный бюджет на всю задачу через все сессии; защищает от взрывного роста при большом числе параллельных сессий.

Расширенная токеномика опирается на четыре механизма:

- **Бюджеты** (калибровочные значения по внутренним наблюдениям Сбера, настраиваются под организацию): 50K токенов на задачу, 200K на сессию, 500K на горизонт. Превышение 80% бюджета автоматически запрашивает у оператора подтверждение продолжения.
- **Отношение стоимости к результату** (Cost-to-Outcome Ratio, CTOR) связывает затраты на токены с бизнес-результатом — прохождением тестов, развёрнутым пул-реквестом, разрешённым инцидентом. CTOR <1.0 означает, что задача окупилась; CTOR <0.3 — отличный возврат инвестиций; CTOR >3.0 — повод для ретроспективы: либо результат недооценён, либо задача не подходит для агентного исполнения, либо библиотека паттернов не содержит применимой записи.
- **Медианный CTOR по командам мы используем как опережающий индикатор зрелости.** Показатель агрегируется поквартально по каждой команде, а команда финансовой эффективности ИИ-расходов (FinOps) опирается на распределение CTOR при инвестиционных решениях. Рост медианы сигнализирует о необходимости вложиться в библиотеку типовых решений, чтобы снизить среднюю стоимость итерации. Высокий CTOR у конкретного класса задач — сигнал провести ретроспективу и, возможно, вывести класс из агентного режима.
- **Динамическое распределение вычислений реализуется через ИИ-шлюз (AI Gateway) с интеллектуальной маршрутизацией.** Простые задачи — автодополнение, суммаризация, маршрутизация — уходят на дешёвые модели; сложные рассуждения, длинный контекст и генерацию кода обслуживают премиум-модели; среды разработки и тестирования используют лёгкие модели с точечным повышением класса; критичные для промышленной среды задачи идут на премиум с дублированным инференсом. Правила маршрутизации декларативны и аудируемы. Дополнительная маршрутизация по времени суток (дешёвые модели в рабочие часы, премиум для ночных пакетных задач) и по специализированным доменам снижает стоимость на 15–30% при той же производительности.

Токеномика в организационном FinOps

Запрос команды на дополнительный бюджет без данных CTOR остаётся решением по политике, а не по данным. С CTOR запрос анализируется по последним 3 месяцам: какова медиана CTOR у команды; растёт она (тревожный сигнал) или падает (хороший); каково распределение по паттернам; как это сравнивается с похожими командами. Пример: «команда A — CTOR 0.4, 30 фич за квартал, +20% бюджета подтверждает возврат инвестиций»; «команда B — CTOR 1.8, 10 фич, +50% бюджета — сначала выяснить причины».

Инвестиция в библиотеку паттернов окупается через снижение медианного CTOR. Паттерн, однажды написанный и откалиброванный, снижает стоимость аналогичных задач для всей организации (связь с долей повторного использования спецификаций, Spec Reuse Rate. Для оценки качественных результатов — удовлетворённости клиентов, внутренней эффективности — команде нужна явная политика оценки стоимости результата.

ИИ агенты в сопровождении

Выкатка — не конец PDLC. После развёртывания агентная система вступает в эксплуатационный контур, где инциденты, деградации и аномалии становятся сигналами для следующего цикла разработки. ИИ SRE (ИИ-автоматизация обеспечения надёжности, site reliability engineering) — замыкающее звено, которое превращает сигналы эксплуатации в артефакты разработки.

Мы фиксируем четыре функции ИИ SRE как связанную последовательность, а не набор изолированных инструментов:

- **Детектирование инцидента (incident detection).** ИИ SRE применяет подавление шума (дедупликация алертов, noise suppression) к потоку телеметрии. Промышленные системы генерируют тысячи сигналов в минуту; без умной фильтрации дежурная команда тонет в ложноположительных алертах. ИИ SRE коррелирует алерты в инциденты, подавляет дублирующий шум и поднимает только значимые события. По данным крупнейших технологических консультантов, организации со слоем AIOps снижают среднее время до обнаружения (mean time to detect, MTTD) на 40–60% по сравнению с порогово-основанными системами.

- **Корреляция и анализ корневой причины (root cause analysis, RCA).** После детекции ИИ SRE строит граф корреляций: какие сервисы затронуты, какая последовательность событий предшествовала инциденту, какая историческая аналогия ближе всего. Граф сервисов и история инцидентов — основной контекст для RCA-агента. Качество RCA прямо пропорционально качеству графа сервисов и полноте истории инцидентов; это инвестиция, которая делается заранее, а не в момент инцидента.

- **Автоматизация runbook.** Для известных классов инцидентов ИИ SRE выполняет runbook автономно. Ограничения применяются строго: автоматическое исполнение — только для низкорисковых, хорошо изученных сценариев с обратимыми действиями. Для высокорисковых действий ИИ SRE готовит runbook и передаёт управление оператору с объяснением: что сделано, что предлагается, каков план отката. Разграничение «автономно / с оператором» — политика, а не произвольное решение агента.

- **Замыкание петли «инцидент → бэклог исследования».** ИИ SRE анализирует паттерны инцидентов и автоматически формирует сигналы для бэклога исследования: повторяющиеся инциденты → карточка «устранить корневую причину»; деградации → карточка «оптимизация»; аномалии поведения → карточка «ревью реестра оценок».

Этот механизм замыкает петлю «телеметрия эксплуатации → спецификация». Без него промышленные инциденты живут в инцидент-треке изолированно, а бэклог исследования формируется только из продуктовых запросов. С ним эксплуатация становится источником требований, равноправным с исследованием.

Существующие решения

Вендор / инструмент	**Основная функция**	**Класс**
PagerDuty AIOps	Подавление шума, корреляция инцидентов, умная маршрутизация алертов	Платформа управления инцидентами
Resolve.ai	Автоматизация runbook, авторемедиация для известных сценариев	Автоматизация runbook
Cleric	RCA с агентным рассуждением по графу сервисов	RCA-агент
Honeycomb Query Assistant	Интерактивный анализ трейсов и телеметрии на естественном языке	Ассистент наблюдаемости

Вендор / инструмент	Основная функция	Класс
Datadog Bits AI	Встроенный ИИ-ассистент в платформу наблюдаемости; генерация запросов, аномалии	Слой ИИ в наблюдаемости

Мы рассматриваем ландшафт ИИ SRE как набор специализированных компонентов. Интеграция через стандартные протоколы (webhook, API, OpenTelemetry) предпочтительнее вертикальной консолидации.

Агенты сопровождения и контур управления

ИИ SRE не существует вне контура управления. Все автономные действия ИИ SRE фиксируются в журнале аудита с теми же требованиями к типизации и целостности, что и действия кодирующих агентов. Граница «автономно / с оператором» устанавливается через перехватчиков политик и подчиняется тому же жизненному циклу управления изменениями, что и любая другая политика.

Метрики ИИ SRE входят в общую панель контура управления: среднее время до обнаружения (MTTD), среднее время до устранения (mean time to resolve, MTTR), доля инцидентов, закрытых автономно, доля ложноположительных эскалаций. Аномальный рост автономных закрытий — сигнал к ревью: либо улучшился runbook, либо деградировал порог срабатывания.

Агент-защитник расширяется на ИИ SRE: те же поведенческие оценки, те же механизмы обнаружения аномалий, та же цепочка эскалации. ИИ SRE — не привилегированный агент вне надзора, а часть общей агентной инфраструктуры с теми же правилами.

Российский контекст

В российском контексте западный стек SRE требует оценки по двум осям — доступности и совместимости с требованиями локализации данных.

Компонент	Статус локализации	Вариант замены
PagerDuty AIOps	Ограниченный доступ; требует оценки	Zabbix + собственный слой AIOps; AlertManager + слой машинного обучения
Datadog	Ограниченный доступ; требует размещения данных в РФ	Prometheus + Grafana + собственный ИИ-ассистент
Resolve.ai / Cleric	Не локализованы; нишевые игроки	Собственная разработка автоматизации runbook на базе больших языковых моделей
Honeycomb	Ограниченный доступ	Jaeger + Tempo + собственный ассистент запросов
OpenTelemetry	Открытый код; полностью локализуемо	OpenTelemetry (без замены)

Платформу наблюдаемости (observability) мы собираем из открытых компонентов — Prometheus для метрик, Grafana для панелей, Jaeger для распределённых трейсов, OpenTelemetry как единый стандарт сбора телеметрии. Этот набор полностью разворачивается внутри периметра без внешних зависимостей и служит рекомендованной базой. Слой искусственного интеллекта поверх базы мы строим на локально развёрнутых или одобренных отечественных больших языковых моделях.

Замыкание петли «инцидент → бэклог исследования» — функция архитектурная, а не вендорная. Достаточно структурированного формата инцидентов и интеграции с системой управления задачами (Jira, YouTrack, Kaiten). Российские команды реализуют эту функцию через собственные интеграции, не завися от западных платформ SRE.

ИИ SRE замыкает PDLC: производство становится источником требований, а не только потребителем поставок. Без этого звена концепция AI-Disrupt PDLC остаётся разомкнутой — быстрая разработка создаёт артефакты, которые эксплуатируются без обратной связи в разработку.

Часть 6. Метрики и измерения

Метрики эффективности управления замыкают контур соответствия: они подтверждают, что результат достигнут в рамках установленных правил, а не случайно. Метрики контура валидации отвечают на вопрос о корректности результата, метрики управления — о корректности самого процесса. Отсюда разговор смещается от отдельных показателей к целостной системе измерений, где каждая метрика получает формулу расчёта, целевое значение по уровню зрелости и источник данных.

В агентной интерпретации каждый классический показатель получает специфичное дополнение:

- частота развёртываний дополняется долей развёртываний с участием агентов;
- время поставки измеряется от спецификации до развёртывания через цикл реализации;
- среднее время восстановления учитывает время автоматического восстановления;
- коэффициент отказов дополняется частотой регрессий, внесённых агентами (Agent-introduced Regression Rate).

Модель DORA 2025 фиксирует семь компетенций команд в области ИИ, а операционный портрет агентной разработки требует восьмой. Семь исходных компетенций — использование инструментов, ревью кода, тестирование, документирование, сканирование безопасности, реагирование на инциденты и непрерывное обучение. К ним мы добавляем объём ИИ-вывода (AI Output Volume) — метрику, отражающую рост числа созданных артефактов относительно базового уровня.

Семь компетенций в агентной интерпретации раскрываются через измеримые показатели:

- **Интеграция ИИ-инструментов** оценивается по глубине внедрения — от изолированных приложений до нативной интеграции в среду разработки и конвейеры развёртывания; в агентной модели — это доля задач, прошедших через цикл реализации.
- **Ревью кода** дополняется долей результатов ИИ-ревьюера, утверждённых экспертом-человеком, и долей ИИ-сгенерированных пул-реквестов, обработанных агентом-ревьюером.
- **Тестирование** отражается в доле ИИ-созданных изменений, успешно прошедших проверку, и уровне регрессий в сгенерированном коде.
- **Документирование** измеряется долей материалов, созданных ИИ, и частотой их обновления.
- **Сканирование безопасности** дополняется покрытием автоматических проверок и частотой обнаружения попыток обхода политик.
- **Реагирование на инциденты** расширяется метрикой Self-Recovery Rate — долей ситуаций, разрешённых агентом без эскалации.
- **Непрерывное обучение** переходит от индивидуального измерения к командному: ключевой показатель — скорость, с которой паттерн одной команды кодифицируется в библиотеке и становится доступным остальным.

AI Output Volume рассчитывается как отношение суммы артефактов за период к базовому уровню до внедрения ИИ. Коэффициент роста более 2× свидетельствует о зрелой практике, свыше 3× — об исключительных результатах. Без этой метрики фокус только на сокращении времени отдельной задачи скрывает фактический рост общего объёма работ.

По данным Anthropic (2026), прирост производительности обеспечивается не одним ускорением отдельных операций, а увеличением общего объёма выпускаемых артефактов: функций, исправлений, экспериментов, обновлений документации, модернизации зависимостей.

Для комплексного анализа мы используем парные метрики DORA:

- **Lead Time × Output Volume** подтверждает, что сокращение сроков развёртывания достигнуто ростом производительности, а не снижением объёма изменений;
- **Change Failure Rate × Agent-introduced Regression Rate** показывает, что внедрение ИИ не ухудшает качество кода;
- **Deployment Frequency × Reallocation Rate** исключает ситуацию «метрики ради метрик», подтверждая, что рост частоты развёртываний отражает выполнение реальных дополнительных задач.

6.1 Категории метрик по AWS

Мы расширяем систему метрик до категорий AWS (Amazon Web Services), объединяя показатели скорости, качества, эффективности, влияния на бизнес и автономии с оценкой эффективности бизнес-результата.

Категория	**Что измеряет**	**Ключевые метрики**
Скорость (Velocity)	Скорость доставки изменений	Lead Time, Spec-to-Deploy, Discovery Cycle Time, Time-to-Outcome
Качество (Quality)	Стабильность и надёжность	Change Failure Rate, Defect Density, Agent-introduced Regression Rate, Hallucination Rate @ Horizon
Эффективность (Efficiency)	Оптимизация ресурсов	Token Cost per Feature, Reallocation Rate, Cost-to-Outcome Ratio
Влияние на бизнес (Business Impact)	Ценность для организации	Reallocation Rate, вклад в регулярный доход (ARR), Outcome Validation Rate
Автономия (Autonomy)	Уровень независимости агентов	Median Task Horizon, Task Completion @ Horizon, Self-Recovery Rate
Эффективность результата (Outcome Effectiveness)	Связь с бизнес-метриками	Cost-to-Outcome Ratio, Outcome Yield Rate, Time-to-Business-Impact

Коэффициент перераспределения (Reallocation Rate) отражает долю сэкономленного времени, перенаправленного на задачи повышенной ценности. По оценке Bain & Company (2025), команды с ИИ-ассистентами демонстрируют прирост продуктивности на 10–15%, однако высвободившиеся ресурсы часто не конвертируются в более значимую работу.

Показатель рассчитывается как отношение часов, затраченных на приоритетные задачи, к общему сэкономленному благодаря ИИ времени. Без контроля этой метрики операционные показатели улучшаются, а реальное влияние на бизнес остаётся нулевым — это антипаттерн «Испарение сэкономленного времени».

Метрики по категориям

- **Скорость:** время внесения изменений, цикл «спецификация — развёртывание», длительность цикла исследования, время до подтверждения результата.
- **Качество:** коэффициент отказов при развёртывании, плотность дефектов, частота регрессий, внесённых агентами, уровень галлюцинаций в зависимости от продолжительности сессии.
- **Эффективность:** стоимость токенов на одну функцию, коэффициент перераспределения времени, отношение затрат к результату (Cost-to-Outcome Ratio).
- **Влияние на бизнес:** коэффициент перераспределения времени, прирост регулярного дохода (ARR), доля подтверждённых результатов.
- **Автономия:** медианная продолжительность задачи, доля выполнения задач при заданном горизонте, коэффициент самовосстановления, частота применения многоагентных сценариев.
- **Эффективность результата:** отношение затрат к результату, доля успешной валидации гипотез, время достижения бизнес-показателя, доля задач с изменённым рабочим процессом.

6.2 Сводная система метрик PDLC

Мы структурируем систему метрик по четырём группам: DORA 2025, ИИ-нативное качество, производительность и контур управления. Для каждой метрики определены формула расчёта, целевые значения по уровням зрелости команды, частота измерения и источник данных.

Таблица метрик DORA 2025 (агентная адаптация)

Метрика	Формула	Junior team	Senior team	Частота	Источник данных
Deployment Frequency	deploys_per_day (скользящее 7-дневное окно)	1–2/день	≥5/день	Ежедневно	CI/CD pipeline
Lead Time for Changes	median(commit_to_prod_seconds) за 7 дней	2–8 часов	<30 минут	Еженедельно	VCS + deploy logs
Change Failure Rate	failed_deploys / total_deploys за 30 дней	<15%	<5%	Еженедельно	Incident tracker
MTTR (Mean Time to Recovery)	Mean (incident_resolve_seconds) за 30 дней	<4 часов	<30 минут	Еженедельно	Incident tracker
Agent Deploy Share	agent_deploys / total_deploys за 7 дней	<20%	≥50%	Еженедельно	CI/CD + agent logs

Метрика	Формула	Junior team	Senior team	Частота	Источник данных
Agent Regression Rate	agent_regressions / total_regressions за 30 дней	<20%	<10%	Еженедельно	QA + monitoring

Таблица метрик ИИ-нативного качества

Метрика	Формула	Junior team	Senior team	Частота	Источник данных
Spec Reuse Rate	reused_specs / total_specs × 100%	≥10%	≥50%	Ежемесячно	Spec library tracker
Defect Density	defects / KLOC за 30 дней	≤1.0/KLOC	≤0.5/KLOC	Еженедельно	Issue tracker + VCS
Hallucination Rate	incorrect_outputs / total_outputs на golden eval set	≤5%	≤2%	Еженедельно	Eval harness
Eval Pass Rate	passed_evals / total_evals за 7 дней	≥85%	≥95%	Еженедельно	Eval framework
Auto-approve Rate	auto-approved_PRs / total_PRs за 7 дней	≤15%	≤10%	Еженедельно	Code review system
Human Override Rate	human_overridden_outputs / total_AI_outputs	25–40%	10–25%	Еженедельно	Review logs
Context Persistence Integrity	consistent_sessions / total_long_sessions	≥80%	≥95%	Еженедельно	Agent runtime logs

Таблица метрик производительности

Метрика	Формула	Junior team	Senior team	Частота	Источник данных
Output Volume Lift	current_artifacts / baseline_artifacts	1.5×–2×	≥3×	Ежемесячно	Artifact registry
PRs per Developer	merged_PRs / devs за 7 дней	2–4	≥7	Еженедельно	VCS
Cycle Time	median(issue_open_to_close_seconds) за 7 дней	3–5 дней	<1 дня	Еженедельно	Issue tracker
Reallocation Rate	high_value_hours / AI_saved_hours × 100%	>0% (рост)	≥60%	Ежемесячно	Time tracking
Discovery Cycle Time	idea_to_SDD_hours (медиана за 30 дней)	<3 дней	<8 часов	Ежемесячно	Spec system

Метрика	Формула	Junior team	Senior team	Частота	Источник данных
Token Cost per Feature	total_tokens / deployed_features за 30 дней	Базовый × 1	Базовый × 0.4	Ежемесячно	LLM billing

Таблица метрик контура управления

Метрика	Формула	Целевое значение	Частота	Источник данных
Audit Coverage	audited_actions / total_R2plus_actions × 100%	100%	Непрерывно	Audit log
Policy Violations Detected	Count (policy_violations) за 7 дней	0 critical; <5 minor	Еженедельно	Policy engine
MTTR Security Incidents	Mean (security_incident_resolve_hours)	<4 часов	По инциденту	Incident tracker
Explanation Coverage	explained_R2plus_actions / total_R2plus_actions	≥99% для R3+	Еженедельно	Agent runtime
Platform Coverage	governed_AI_tools / total_AI_tools	>80% (>95% для regulated)	Ежемесячно	IDP registry
Self-Recovery Rate	auto_resolved_sessions / total_interrupted_sessions	≥70% L3; ≥85% L4+	Еженедельно	Agent runtime

Примеры расчёта метрик

Lead Time for Changes (Срок внедрения изменений). Формула: median (commit_to_prod_seconds) по скользящему 7-дневному окну. Данные берутся из git hooks и deploy pipeline. За неделю 47 коммитов с медианой 1 750 секунд = 29 минут; senior-цель достигнута (<30 мин). Для junior-команды медиана 14 400 секунд = 4 часа — в допустимом диапазоне (2–8 ч). Lead Time измеряется от первого коммита в ветке, а не от создания задачи — иначе Discovery Cycle Time смешивается с временем реализации.

Spec Reuse Rate (Коэффициент повторного использования спецификаций). Формула: (reused_specs / total_specs) × 100%. За квартал создано 80 новых SDD (спецификаций), из них 22 используют шаблоны из библиотеки с доказанным поведением: reuse_rate = 22/80 = 27.5%. Цель Горизонта 2 достигнута (≥25%). На Горизонте 1 было 8%, рост 3.4× за два квартала — свидетельство формирующейся культуры переиспользования. Средняя стоимость новой SDD снизилась с $2K до $800 за счёт сокращения токенного потребления на фазе исследования.

Hallucination Rate (Частота галлюцинаций). Формула: (incorrect_outputs / total_outputs) на фиксированном golden eval set, еженедельно. Golden eval set — набор из 500 верифицированных запросов с известными правильными ответами, обновляется ежеквартально. За неделю 500 оценок, 12 некорректных ответов — hallucination_rate = 12/500 = 2.4%, незначительно выше целевого (≤2%). Все 12 ошибок — в ответах на вопросы об API-контрактах старше 6 месяцев. Действие: обновить контекст golden set, добавить актуальную документацию API в retrieval index. Hallucination Rate всегда привязан к конкретному eval set — без этого метрика несопоставима между командами.

Mean Time to Recovery (MTTR, Среднее время восстановления). Формула: mean (resolve_time − detect_time) по всем инцидентам за 30 дней. За месяц 4 инцидента с временем устранения 45 мин, 22 мин, 18 мин, 95 мин: MTTR = (45+22+18+95)/4 = 45 мин. Senior-цель не достигнута (<30 мин) из-за одного выброса в 95 мин — инцидента неизвестного типа без runbook. Действие: добавить runbook в IDP, установить порог оповещения на 60 мин как триггер эскалации.

Антипаттерны измерения

Vanity metrics (метрики тщеславия, «красивые» метрики). Метрика LOC/developer без контроля качества — классический антипаттерн. Агент легко генерирует 5 000 LOC за день, из которых 70% — шаблонный код низкой ценности. Корректная пара: LOC/dev × Defect Density. Если Defect Density растёт вместе с LOC — рост производительности иллюзорен.

Закон Гудхарта (Goodhart's Law). Когда метрика становится целью, она перестаёт быть хорошей метрикой. Команда оптимизирует Deployment Frequency, разбивая крупные изменения на тривиальные микро-коммиты без реальной ценности. Защита: измерять метрики в парах (Deployment Frequency × Output Volume Lift, Lead Time × Outcome Validation Rate).

Краткосрочные и долгосрочные метрики. Ошибка оптимизации под ведущие индикаторы в ущерб отложенным. Lead Time измеряется еженедельно, Outcome Validation Rate формируется за 3–6 месяцев. Команды, игнорирующие отложенные метрики, демонстрируют отличную операционную картину при нулевом бизнес-влиянии — антипаттерн «Испарение сэкономленного времени». Баланс: ежедневный мониторинг ведущих индикаторов, ежеквартальная проверка отложенных, ежегодная калибровка целевых значений по накопленным данным.

Агрегация без сегментации. Средний Hallucination Rate по всей организации скрывает критические выбросы на уровне конкретных команд или доменов. Метрики измеряются минимум на трёх уровнях: org (стратегический), team (тактический), agent (операционный). Деградация на уровне одного агента детектируется через Drift Detection Latency (целевое: <24 ч) до того, как отразится в org-метриках.

Для стратегических решений мы анализируем метрики в парах:

- **доля перестроенных рабочих процессов** — вместе с индексом плотности ценности: виден ли реальный подъём плотности от редизайна;
- **доля подтверждённых бизнес-исходов** — вместе с долей неудачных изменений: замкнутый цикл качества;
- **доля повторного использования спецификаций** — вместе со стоимостью фичи в токенах: экономия ресурсов;
- **медианный горизонт задачи** — вместе с долей самовосстановления: устойчивость длительных сессий;
- **отношение темпов реализации и валидации** — вместе с долей регрессий, внесённых агентом: баланс скорости и качества;
- **доля перераспределения сэкономленного времени** — вместе с приростом объёма результата: конверсия высвобожденного времени в реальный выпуск.

Операционный ритм анализа включает ежедневные стендапы (контроль аномалий), еженедельные обзоры трендов, ежемесячные отчёты для руководства и ежеквартальные стратегические сессии. Ведущие индикаторы (I/V Tempo Ratio, Spec Reuse Rate, Discovery

Cycle Time) сигнализируют о процессах на ранней стадии, отложенные показатели (Outcome Validation Rate, Reallocation Rate, CTOR) отражают состоявшиеся бизнес-результаты.

Сводный индекс зрелости PDLC (PDLC Maturity Index) объединяет ключевые параметры в шкалу от 0 до 100, где значения 60+, 80+ и 90+ соответствуют уровням зрелости L3, L4 и L5.

Метрики эффективности управления

Мы выделяем отдельную группу — эффективность управления (Governance Effectiveness Metrics). Эти показатели дополняют категории AWS, метрики зрелости, ADLC-категории и метрики контура валидации. В отличие от метрик валидации, отвечающих на вопрос о корректности результата, метрики управления оценивают корректность самого процесса с точки зрения соответствия политикам и нормативным требованиям.

Универсальные метрики эффективности контура управления

Метрика	Описание	Целевое значение
Policy Coverage Rate	Доля активных агентных контуров, охваченных задокументированными политиками	>95% для продуктивных сред
Audit Trail Completeness	Доля событий жизненного цикла, зафиксированных в неизменяемом журнале	100%; любые разрывы — критическое несоответствие
Time-to-Detect Violation	Время от момента нарушения политики до автоматической детекции	<1 час для критичных; <24 ч для всех
Mean Time to Remediation	Среднее время устранения выявленного нарушения политики	<4 ч критичных; <48 ч для прочих
Approval-to-Override Ratio	Соотношение автоматических одобрений к ручным переопределениям	Оптимально 75:25; <50:50 — сигнал деградации

Группа 1: метрики качества и безопасности ПО

Метрика	Описание	Целевой диапазон
Test pass rate	Доля блоков кода, успешно прошедших автоматические проверки без ручного вмешательства	>90% для R3+, >95% для R5
Code compliance rate	Доля кода, соответствующего внутренним и внешним нормам (OWASP, GDPR и применимые локальные стандарты)	>98%
Human override rate	Доля ИИ-генерированного кода, вручную исправленного или отклонённого разработчиками	10–25% (оптимально)

Human override rate требует особой интерпретации: значения свыше 40% указывают на слабость ИИ-модели или контекста, а значения ниже 5% сигнализируют о формальном «резиновом» согласовании — скрытом риске управления.

Группа 2: метрики эффективности ИИ-управления

Метрика	Описание	Целевой диапазон
Code review / verification agent escalation rate	Доля нежелательных или рискованных действий, автоматически перенаправленных на проверку человеком	Рост = улучшение контура управления
Runtime guardrail trigger rate	Частота срабатывания runtime-контролей безопасности	Снижение при зрелом контексте
Auto-remediation success rate	Доля инцидентов, разрешённых автоматически без потери данных	>80% для известных сценариев
Platform coverage of AI tools and agents	Доля ИИ-инструментов и агентов, подчиняющихся единым платформенным политикам	>80% (для регулируемых отраслей: >95%)

Значение Platform coverage ниже 80% создаёт риск теневой эксплуатации ИИ, при которой часть операций выполняется вне контура управления и не фиксируется в журнале аудита.

Комбинированная интерпретация выявляет системные дисбалансы:

- высокий коэффициент отмены человеком (Human override rate) при низкой частоте срабатывания контроля времени выполнения (runtime-контроль) указывает на неэффективность управления или слабые выходные данные ИИ;
- низкий Human override rate при высокой активности runtime-контролей подтверждает зрелость системы, где человек вмешивается только в исключительных случаях;
- низкое покрытие платформенными политиками при высоком Human override rate — наихудший сценарий: теневое использование ИИ и низкое качество генераций;
- высокое покрытие, оптимальный Human override rate и стабильная авторемедиация характеризуют зрелый контур управления (governance), обеспечивающий реализацию ROI без накопления технического долга.

Метрики эффективности управления — отложенные индикаторы, но единственный объективный способ оценить работоспособность контура управления в продуктивной среде. Метрики контура валидации подтверждают корректность результатов, а метрики управления гарантируют, что эта корректность достигнута в рамках установленных правил, а не случайно.

Система метрик определена. DORA 2025 даёт операционный базис, ИИ-нативное качество фиксирует специфические риски агентной разработки, метрики производительности переводят эффект в измеримые показатели, эффективность управления замыкает контур соответствия.

6.3 Причины искажения метрик

Надёжность измерений подрывают три причины: шум среды, незрелость длительных сессий и неполное покрытие жизненного цикла. Мы контролируем каждую отдельно: шум — стандартизацией среды и версионированием контекста, незрелость — калиброванной лестницей зрелости, покрытие — расширенной матрицей метрик ADLC.

При измерении производительности мы учитываем два типа шума. Первый — **аппаратный шум от различий в вычислительных ресурсах и фоновой нагрузке**; он вносит погрешность до 6 процентных пунктов в метрики времени поставки (lead time). Второй — **шум контекста сессии (session-context noise)**: деградация качества агентного вывода по мере накопления истории в рамках одной длительной сессии. Без учёта этого фактора метрики длительных сессий ненадёжны: повторное выполнение одной задачи в разных условиях даёт расходящиеся результаты только из-за изменений контекста.

Аппаратный шум мы минимизируем стандартизированными профилями изолированных сред (sandbox), фиксирующими параметры процессора, памяти и пропускной способности сети. Профили документируются в рамках внутренней платформы разработки (IDP); еженедельные бенчмарки своевременно выявляют отклонения от базовых показателей. Управление шумом контекста требует версионирования сохраняемых записей: агент обрабатывает последние N версий с учётом их весовых коэффициентов, а метрика Context Persistence Integrity отслеживает согласованность данных.

С учётом обоих факторов мы расширяем матрицу рисков:

- Испарение сэкономленного времени контролируется через KPI Reallocation Rate с Горизонта 1 при ежемесячном отчёте для руководства.
- Разрыв между реализацией и валидацией предотвращается мониторингом I/V Tempo Ratio, соблюдением 12-го принципа через ИИ-барьер в CI/CD и формулой параллельных инвестиций: каждая единица ресурсов, вложенная в цикл реализации, требует минимум 0,5 единицы на модернизацию валидации.
- Инфляция объёма релиза нейтрализуется корреляцией метрик релиза с показателями бизнес-результата при ежеквартальном планировании.
- Семантический дрейф минимизируется ежеквартальным аудитом схем, управлением на уровне 15-й ключевой особенности IDP и отклонением схем, выходящих за установленные границы.

Оценка зрелости команд

Уровень зрелости команд мы оцениваем по двум осям: доле задач, выполняемых с участием ИИ, и медианной продолжительности автономного выполнения задач (task horizon).

Уровень	Доля задач с ИИ	Медианная продолжительность сессии	Аналог по модели Bain
L1	<10%	≤5 мин	L1 retrieval
L2	10–30%	≤15 мин	L2 single-task
L3	30–60%	≤1 час	L2/L3 cross-system

Уровень	Доля задач с ИИ	Медианная продолжительность сессии	Аналог по модели Bain
L4	60–80%	≤4 часа (зрелый режим до 1 рабочего дня)	L3 cross-system
L5+	>80%	≥4 часа с автономным восстановлением	L4 multi-agent

Дополнительные метрики зрелости включают длительность цикла исследования, коэффициент повторного использования спецификаций, частоту применения переиспользуемых рабочих процессов, соблюдение классификации агентопригодности задачи и показатель жизнеспособности библиотеки паттернов.

Библиотека шаблонов классифицируется по пяти категориям: шаблоны разработки через спецификацию, конфигурации многоагентных процессов, шаблоны интеграции инструментов, наборы проверок и поведенческие сценарии. Жизненный цикл паттерна проходит стадии: черновик, ревью, эксперимент, активное использование, рекомендация, вывод из эксплуатации и архивирование.

Spec Reuse Rate рассчитывается как доля новых SDD, созданных на основе библиотечных шаблонов. Целевые значения: ≥10% на Горизонте 1, ≥25% на Горизонте 2, ≥50% на Горизонте 3. Прямая корреляция с сокращением стоимости токенов (на 30–60%) и ускорением цикла исследования подтверждает экономическую эффективность инвестиций в библиотеку.

Лестница зрелости длительных сессий спроектирована под требования к надёжности. Переход между уровнями требует одновременного прогресса по трём параметрам: медианной продолжительности задачи, коэффициенту самовосстановления и уровню галлюцинаций. Ускоренный переход без достижения пороговых значений по всем осям приводит к преждевременному масштабированию и росту числа регрессий. Калибровка по модели Bain Level Equivalent опирается на внешние бенчмарки и исключает завышенные самооценки зрелости команд.

Матрица метрик ADLC

Расширенная матрица ADLC объединяет пять категорий метрик: результат, качество, доверие, надёжность и жизнеспособность жизненного цикла.

Категория	Метрики
Результат (Outcome)	Скорость редизайна процессов, доля покрытия гипотез, доля валидируемых гипотез
Качество (Quality)	Agent-introduced Regression Rate, Hallucination Rate @ Horizon, Evidence Bundle Completion Rate
Доверие (Trust)	human-in-the-loop Coverage Score, Escalation Quality Score, Explanation Coverage, Explanation Quality Score
Надёжность (Reliability)	Self-Recovery Rate, Context Persistence Integrity, Multi-Agent Adoption %
Жизнеспособность жизненного цикла (Lifecycle Health)	Pattern Library Lifecycle Health, Agent Retirement Coverage, Spec Reuse Rate

Категория «Доверие» дополнена метрикой Explanation Coverage — долей действий уровня R2 и выше, сопровождаемых структурированным пояснением. Для операций уровня R3+ обязательный порог составляет 99%. Explanation Quality Score оценивается агентом-защитником (Guardian Agent) по пятибалльной шкале, учитывающей полноту, внутреннюю согласованность и соответствие фактическим действиям.

Agent Retirement Coverage гарантирует, что 100% агентов имеют утверждённый план остановки и архивированную историю проверок. Это исключает накопление технического долга в форме «зомби-агентов» без владельца и документации.

Outcome Validation Rate — доля подтверждённых бизнес-гипотез — фиксирует долю успешно развёрнутых задач, результат которых верифицирован в установленный период. Это ключевая отложенная метрика, замыкающая контур между намерением команды (SDD) и реальным бизнес-эффектом.

6.4 Управление риском через метрики

Матрица рисков

Классическая матрица риска отображает вероятность события на одной оси и тяжесть ущерба на другой. В ИИ-нативной разработке каждая ячейка соответствует реальному классу инцидентов, с которым команды сталкиваются на Горизонтах 1–3. Мы применяем пятиуровневые шкалы: вероятность — от «крайне редко» до «почти неизбежно», ущерб — от «незначительного» до «катастрофического».

Вероятность / Ущерб	Незначительный ущерб	Низкий ущерб	Средний ущерб	Высокий ущерб	Катастрофический ущерб
Почти неизбежно (>0.8)	Опечатки в ИИ-документации; устраняются при ревью	Галлюцинация в README / комментариях — вводит в заблуждение, легко детектируется	Неверный SQL-запрос в dev-ветке, удаляет тестовые данные	Ошибка деплоя на стейдж-сервер с потерей сессий пользователей	Автоматическое удаление prod-данных по ошибочной инструкции агента
Вероятно (0.5–0.8)	Избыточные токены в промпте — рост стоимости на 5–10%	Бесполезный unit-тест, пропускает регрессию; обнаруживается в следующем цикле	Инъекция вредоносных команд в запрос пользователя, раскрывает системный промпт	Агент создаёт неавторизованные внешние API-запросы с секретами	Утечка ПДН через инструмент агента во внешний endpoint

Вероятность / Ущерб	Незначительный ущерб	Низкий ущерб	Средний ущерб	Высокий ущерб	Катастрофический ущерб
Возможно (0.2–0.5)	Низкокачественный код-стиль; исправляется линтером	Галлюцинация в тест-сценарии — неправильный ожидаемый результат, пропускает баг в продакшн	Утечка учётных данных через промт-инъекцию в IDE-ассистенте	Компрометация CI/CD pipeline через заражённую зависимость, добавленную агентом	Нарушение GDPR: агент копирует персональные данные в неаудируемое хранилище
Маловероятно (0.05–0.2)	Неиспользуемый импорт в сгенерированном коде	Документация расходится с реальным API; баг обнаруживается через 2–3 недели	Agent loop: агент зацикливается, потребляет ресурсы до принудительного завершения	Многоагентная эскалация привилегий через цепочку MCP-серверов	Полная компрометация prod-среды через цепочку агентных действий
Крайне редко (<0.05)	Избыточный комментарий к функции; удаляется при ревью	Ошибка в именовании переменной, критичная для downstream-системы	Утечка внутреннего API-контракта через публичный запрос к модели	Кража интеллектуальной собственности через эксфильтрацию репозитория агентом	Таргетированная атака через цепочку поставок через ИИ-сгенерированную зависимость

Зоны ответственности определяются индексом риска (вероятность × ущерб):

- Зелёная зона (индекс ≤4): штатный мониторинг, еженедельный отчёт.
- Жёлтая зона (индекс 5–9): назначен владелец риска, план митигации в бэклоге.
- Оранжевая зона (индекс 10–14): требуется CISO sign-off, митигация в текущем спринте.
- Красная зона (индекс 15–19): уведомление совета директоров в течение 48 часов, немедленный план.
- Критическая зона (индекс 20–25): немедленная остановка соответствующего агентного контура.

Калибровка вероятности

Оценка вероятности строится на трёх источниках данных:

- **исторические данные инцидентов** — если за последние 12 месяцев команда зафиксировала 3 случая промт-инъекций из 200 ревью, базовая вероятность = 0.015 на ревью;
- **threat intelligence** — публичные базы CVE, OWASP LLM Top-10, внутренние отчёты red-team;
- **экспертные оценки методом Дельфи** — несколько независимых оценщиков дают вероятность, расхождение >0.3 требует дополнительного раунда.

При появлении новых данных мы применяем обновление по теореме Байеса — фундаментальному закону теории вероятностей: апостериорная вероятность (posterior) = априорная вероятность (prior) × правдоподобие (likelihood) / нормирующая константа (normalizer). При prior (промт-инъекция) = 0.3 и новом отчёте red-team, зафиксировавшем уязвимость в 2 из 5 тестируемых моделей (likelihood = 0.4), posterior = 0.3 × 0.4 / Z ≈ 0.46. Оценка сдвигается в «вероятно» и требует пересмотра митигации. Обновление выполняется при каждом значимом изменении: выход новой версии модели, инцидент у смежной команды, результаты очередного теста red-team.

Калибровка ущерба

Ущерб оценивается по четырём измерениям, каждое из которых переводится в денежный эквивалент для сопоставимости:

- **Финансовый ущерб** — прямые потери: штрафы, компенсации, стоимость восстановления данных, расходы на юридическое сопровождение.
- **Репутационный ущерб** — восстановление доверия занимает 6–18 месяцев по данным Bain & Company; оценивается через снижение NPS, отток клиентов и стоимость PR-кампании.
- **Регуляторный ущерб** — штрафы GDPR до 4% годового оборота; 152-ФЗ — до 18 млн рублей за нарушение; требования ЦБ РФ предусматривают приостановку лицензии.
- **Операционный ущерб** — downtime × выручка в час плюс MTTR (Mean Time to Recovery) как стоимость инженерного времени на устранение.

Пример расчёта: утечка учётных данных в IDE

Рассмотрим конкретный сценарий — утечку API-ключей (credentials) через промт-инъекцию в IDE-ассистенте (ячейка матрицы: «возможно» × «высокий ущерб»).

Оценка вероятности. По публичным данным (OWASP LLM Top-10, 2024) и внутренним тестам red-team вероятность успешной эксплуатации на команду из 20 разработчиков за квартал = 0.3 (medium). Байесовский prior: среда не имеет очистки входных данных (input sanitization) на уровне IDE-плагина.

Оценка ущерба складывается из четырёх составляющих:

- **Прямые финансовые потери:** стоимость расследования ($150K) плюс ротация учётных данных и замена скомпрометированных сервисов = $500K direct.
- **Репутационный ущерб:** B2B-клиент узнаёт об инциденте, требует аудит SOC2; задержка двух сделок на общую сумму $2M, скидка 15% для удержания контрактов = $2M reputational (оценка).
- **Регуляторный ущерб:** при утечке персональных данных (ПДН) — уведомление регулятора, возможный штраф = $200K regulatory.
- **Операционный ущерб:** MTTR 48 часов × 5 инженеров × $150/час = $36K downtime.

Итого ущерб — $2.736M. Для консервативной оценки мы используем $5M direct + $50M reputational при enterprise-масштабе (крупный банк, данные 10M клиентов).

Ожидаемые потери: Expected Loss = Probability × Total Damage = 0.3 × $55M = $16.5M.

Бюджет митигации: внедрение очистки входных данных на уровне IDP, обязательный pre-commit hook для детекции паттернов prompt injection, обучение команды = $2M (разовые затраты).

ROI митигации: ROI = Expected Loss / Mitigation Cost = $16.5M / $2M = 8.25x.

Решение — внедрять. ROI выше 3× автоматически одобряется CISO без дополнительного обоснования (подробнее — Action Thresholds ниже).

Порог эскалации (Action Thresholds)

Уровень ожидаемых потерь определяет, кто и как быстро принимает решение о митигации.

Expected Loss	Порог	Требуемое действие	Срок
< $100K	Низкий	Владелец риска в команде, бэклог	Следующий квартал
$100K–$1M	Средний	Tech Lead и архитектор безопасности, план митигации	В течение спринта (2 недели)
$1M–$10M	Высокий	Обязательный CISO sign-off, детальный план митигации	В течение 5 рабочих дней
$10M–$50M	Критический	Уведомление совета директоров в течение 48 часов, немедленный план действий	48 часов
> $50M	Катастрофический	Немедленная остановка соответствующего агентного контура, экстренный совет директоров	Немедленно

Пороговые значения устанавливаются на уровне организации и пересматриваются ежегодно с учётом роста масштаба операций.

Приоритизация рисков распределяется по горизонтам внедрения:

- **Горизонт 1** — основной риск сопротивления адаптации; смягчается системой внутренних проводников практики.
- **Горизонт 2** — разрыв между реализацией и валидацией; устраняется параллельными инвестициями в контур валидации.
- **Горизонт 3** — превышение допустимого горизонта задач; контролируется строгими критериями зрелости по медианной продолжительности задач, коэффициенту самовосстановления и уровню галлюцинаций.

Непрерывный мониторинг рисков выстроен по четырём периодам:

- еженедельный контроль I/V Tempo Ratio — оповещение при значении >2;
- ежемесячный анализ Reallocation Rate — пороговое значение при падении ниже нуля;
- ежеквартальный анализ трендов Outcome Validation Rate и состояния библиотеки паттернов;
- ежегодная комплексная оценка валидации и аудит семантических схем.

Часть 7. Стратегия внедрения

Часть 6 определила систему измерений: DORA-метрики, ИИ-нативное качество, эффективность управления — полный набор стрелок компаса, по которым судим о прогрессе трансформации. Теперь у нас есть инструмент ориентирования.

Часть 7 — о том, как начать движение: какие принципы держать первыми, какие горизонты ставить, и каких антипаттернов избегать в первый год внедрения. Принцип «метрики раньше автоматизации» — не абстракция; он буквально первым встанет в список принципов ниже.

7.1 Принципы внедрения

Успешное внедрение AI-Disrupt PDLC начинается с правильной последовательности решений. Что мы измеряем до автоматизации, куда вкладываемся до выбора модели, кого ставим в центр — разработчика или агента. Мы фиксируем двенадцать принципов, образующих связную систему: нарушение одного ослабляет несколько других.

1. **Начинаем с базового замера DORA (DORA baseline).** Без базового замера прогресс не измерить. Внедрение без метрик — управление вслепую.
2. **Пилот предшествует масштабированию.** Две-три команды-добровольца; ROI измеряется до тиражирования. Успешный пилот — условие масштабирования, а не сигнал к немедленному тиражированию.
3. **Внутренние проводники практики важнее директив.** Проводники переносят практики в команды на основе реального опыта. Директива без носителя — документ, который никто не читает.
4. **Спецификации важнее тикетов.** Разработка через спецификацию (SDD) — основа петли реализации. Тикет описывает запрос; спецификация описывает результат и условия его достижения.
5. **Оценки задают приспособленность, тесты фиксируют утверждения.** Мы ведём разработку от оценок, а не от тестов. Оценки задают фитнес-функцию агентного поведения; тесты фиксируют конкретные утверждения об ожидаемом выводе.
6. **Среда работы ИИ-агента важнее модели.** Капитальный актив — детерминированная инфраструктура. Среда работы ИИ-агентов накапливает ценность с каждым поколением моделей.
7. **Политика как код важнее политики как намерения (policy as code).** Ограничители-перехватчики и принудительное исполнение важнее документов. Политика, которую нельзя проверить автоматически, — декларация о намерениях.
8. **Адаптивная автономия важнее статических режимов.** Лестница автономии (R-ladder) зависит от контекста задачи, профиля риска и архитектурной сложности.
9. **Сквозной аудит ведётся в режиме только-добавления (append-only).** Фиксируются события жизненного цикла, а не только вызовы инструментов. Аудит без возможности воспроизвести цепочку решений агента — не аудит.
10. **Протоколы остаются вендор-нейтральными.** MCP, A2A и AG-UI заменяют привязку к конкретному вендору. Вендорская зависимость превращает смену модели в проект по переписыванию инфраструктуры.
11. **Исследование предшествует генерации (Discovery before generation).** Агент не запускается, пока не пройдены четыре шага цикла исследования. Стоимость переделки превышает затраты на исследование в 5–10 раз.
12. **Валидация масштабируется вместе с реализацией.** При инвестиции X в цикл реализации не менее 0,5X направляется в контур валидации. Сэкономленное время перераспределяется в задачи высокой ценности.

Способы операционализации подходов

Принципы 11 и 12 держатся на операционных контрольных механизмах. Разработка через спецификацию (SDD) без раздела «Гипотеза результата» не принимается в коллективную

детализацию, а SDD без решения «доработка существующего решения или реинжиниринг» блокируется на ИИ-контроле в CI/CD. Так принцип 11 защищает от спешного создания неправильных спецификаций.

Метрику отношения темпа реализации к темпу валидации (I/V Tempo Ratio) мы отслеживаем еженедельно. Значение выше 2 — предупреждение; выше 3 — приостановка новых агентных практик. При вложении X в цикл реализации в контур валидации направляется Y ≥ 0,5X. Доля перераспределения ниже 0,3 требует ретроспективы с участием старшего руководства.

По данным крупнейших технологических консультантов, организации, добившиеся устойчивого масштабирования агентных практик, руководствовались тремя структурными рекомендациями. Все три подчинены одному порядку: измерение опережает автоматизацию, среда опережает модель, разработчик опережает агента.

Мы начинаем с разработчика прежде агента. Паттерн с разработчиком в центре предшествует паттерну с агентом в центре. Разработчик, который понимает, что и зачем автоматизируется, создаёт среду с предсказуемым поведением агента. Агент, запущенный без этого понимания, воспроизводит ошибки с производительностью машины.

Мы инвестируем в среду до выбора модели. Выбор между топовыми моделями — решение второго порядка. Решение первого порядка — какой становится среда работы ИИ-агентов организации: какие контекстные источники загружаются в начале сессии, какие ограничители-перехватчики блокируют несоответствующий вывод, какие проверочные точки требуют эмпирических доказательств. Перепроектирование среды даёт больший эффект, чем подмена модели: разрыв между топ-моделями при одинаковой среде несопоставим с разрывом между средами при одинаковой модели.

Метрики предшествуют автоматизации. Автоматизация без измерения накапливает невидимый долг. Прежде чем автоматизировать шаг, мы фиксируем его текущее состояние: время, качество, долю ошибок. Только тогда ускорение становится измеримым, а деградация — заметной до накопления критической массы.

Двенадцать принципов работают как единая система: измерение опережает автоматизацию, среда опережает модель, исследование опережает генерацию. Нелинейный эффект внедрения приходит от их связности, а не от отдельного инструмента.

Руководство по трансформации

Переход к AI-Disrupt PDLC мы выстраиваем как последовательность из 6 шагов, и на каждом шаге появляется проверяемый артефакт.

Первый — инвентаризация текущего процесса с картой этапов, ролей и точек передачи.

Второй — обновление шаблона спецификации (SDD): в нём появляются разделы «Гипотеза бизнес-результата», «Карта решений с участием человека» и «Класс пригодности к агентному выполнению».

Третий — первая полноценная спецификация на реальной задаче с прохождением внутренней петли исследования (Discovery sub-loop).

Четвёртый — запуск агента-ревьюера в режиме наблюдения (shadow mode) и сборка первого пакета доказательств.

Пятый — полное внедрение ритуалов исследования в командах-чемпионах и перевод агента-ревьюера в режим блокирующей проверки до слияния веток (pre-merge).

Шестой — модернизация валидационного слоя и автоматизация проверок безопасности на всех контрольных точках.

Часть элементов требует новой инфраструктуры, другая — только изменения практики.

Элементы, требующие новых инструментов: Real-time Explainability Layer; ALM; схемы PDLC Object Model; Outcome Pricing Engine. Элементы, требующие организационных изменений: Discovery sub-loop; Validation literacy и Discovery facilitation как компетенции; владение ALM.

Элементы, не требующие немедленного изменения: двухпетлевая модель; принцип «Среда работы ИИ-агента важнее модели»; лестница R0–R5; существующие MCP-интеграции; CI/CD пайплайны (с добавлением новых gate).

Размер команды	**Инфраструктура**	**Срок до зрелости**	**Инвестиции**
Малые, 5–15 инженеров	Discovery rituals как практика; SDD в Markdown; review-agent через prompt в ChatGPT/Claude; Pattern Library в Wiki	12 месяцев	Минимальные
Средние, 50–100 инженеров	Платформенная команда 3–5 человек; IDP $200–500K/год; централизованный review-agent	12–24 месяца	$1–3 млн/год
Крупные, 500+ инженеров	Платформенная команда 10–20 человек; IDP $2–5 млн/год; Federated Policy Hook Registry; ALM; Cross-domain Pattern Library	24–36 месяцев	$5–10 млн+/год

7.2 Горизонты внедрения

Мы разделяем дорожную карту внедрения на три горизонта. Каждый горизонт — не произвольный временной интервал, а уровень зрелости с конкретными критериями входа и выхода.

Горизонт 1 (0–6 месяцев): L0 → L2, Discovery и Validation с первого дня

Горизонт 1 — заложение фундамента. Команды, пропускающие фундаментальные шаги, возвращаются к нему через 12–18 месяцев в режиме дорогостоящего исправления.

Ключевые активности: онбординг ритуалов Discovery (PR/FAQ + Outcome Hypothesis); закрытие первых пяти фич через PR/FAQ; внедрение Outcome Hypothesis и HITL Decision Map в шаблон SDD v0.1; формирование базового Evidence Bundle; параллельная модернизация Validation Loop вместе с пилотированием ИИ; фиксация первого agent-DORA baseline.

Вехи	**Содержание**
M1.5	PDLC Object Model v0.1 — схемы для пяти ключевых артефактов: spec, task, gate, eval-bundle, audit-event

Вехи	Содержание
M3.5	Слой объяснимости-в-реальном-времени в режиме read-only (mock-explanations); завершение первых пяти PR/FAQ-driven фич
M4	Базовое значение метрики HITL Coverage Score зафиксировано
M6.5	Сбор базового Cost-to-Outcome Ratio на пилотных командах

Первый квартал (месяцы 0–3) посвящён базовым замерам DORA, выбору чемпионов, освоению ритуалов Discovery и запуску минимально жизнеспособной внутренней платформы — реестра спецификаций, базового реестра валидаций и простой шины перехватчиков политик. Второй квартал (месяцы 4–6) закрывает первые пять фич, выросших из проблем клиентов и пресс-релизов, стандартизирует спецификацию в версии v0.1, выводит агент-ревьюер в теневой режим и фиксирует базовое значение отношения «стоимость к исходу».

Критерии выхода из Горизонта 1: Lead Time −20%; CFR не ухудшился; чемпионы способны обучать новые команды; базовый уровень HITL Coverage Score зафиксирован.

Горизонт 2 (6–18 месяцев): L2 → L4, масштабирование SDD и long-running процессов

Горизонт 2 — переход от пилота к системному внедрению. SDD становится стандартом; long-running агентные сессии выходят в production; валидационный слой масштабируется вместе с реализацией.

Ключевые активности горизонта: запуск библиотеки шаблонов спецификаций (10–15 паттернов), накатанный маршрут долгих прогонов (ночной рефакторинг, устранение технического долга), вывод протокола передачи сессий в промышленную эксплуатацию, минимальная реализация управления жизненным циклом агентов, федеративный реестр перехватчиков политик с первыми двумя-тремя доменами и обязательная объяснимость в реальном времени для операций уровня R3 и выше.

Вехи	Содержание
M8.5	Минимальный ЖЦ агентов в ПРОМе до выхода на уровень R4
M10.5	Первые специфичные хуки от двух-трёх команд
M11.5	Первый постоянный процесс в промышленной среде
M12.5	Движок прайсинга связывает token-метрики с PR/gate outcomes
M14.5	PDLC Семантический слой покрывает 15+ артефактов

Кварталы Q3–Q4 (месяцы 7–12): спецификация у 50% команд и более; первые десять паттернов в библиотеке; ИИ-шлюз встроен в конвейер непрерывной интеграции; среда долгих сессий в пилоте; управление жизненным циклом агентов в промышленной эксплуатации. **Кварталы Q5–Q6 (месяцы 13–18):** семантический слой; доля повторного использования спецификаций — не ниже 25%; полный цикл управления жизненным циклом агентов.

Горизонт 3 (18–36 месяцев): L4 → L5, координированные агентные команды

Горизонт 3 — операционная зрелость. Мультиагентная оркестрация переходит из экспериментального режима в основной способ работы.

Ключевые активности: L5 координированные агентные команды; Multi-Agent Adoption ≥ 30%; Explanation Coverage >99%; полный цикл ALM; Outcome Validation Rate ≥ 70%; Median Task Horizon ≥ 4 часа; контур telemetry → eval → spec в Pattern Library. Q7–Q12: Outcome Yield как первичный KPI; стабильная зрелость на уровне L4–L5.

90-дневный MVP: L0 → L2 с discovery и validation

Минимальный путь от L0 до L2 проходит через Discovery ритуалы и методику валидации ИИ-результатов с первого дня. Это не упрощённый вариант полного внедрения — это полный вариант для первых 90 дней: все ключевые практики в минимальной конфигурации.

Период	Активности
Дни 0–14	Сбор DORA baseline; выбор пилотных команд; первая сессия Discovery (PR/FAQ + Outcome Hypothesis) на одной задаче
Дни 15–30	Шаблон SDD v0.1 с Outcome Hypothesis + HITL Decision Map; интеграция ИИ-контролей в CI/CD; review-agent в shadow mode
Дни 31–60	Первые три-пять записей Pattern Library; операции уровня R2 в feature branch; первый Evidence Bundle
Дни 61–90	Измерение Reallocation Rate; базовые метрики валидационного слоя; review-agent в pre-merge; Outcome Check на первой завершённой фиче

Критерии успеха 90-дневного MVP:

- Reallocation Rate > 0: хотя бы часть сэкономленного времени перенаправлена на задачи высокой ценности.
- HITL Coverage Score = 100%: все SDD содержат заполненную Decision Map.
- Outcome Hypothesis Coverage = 100%.
- CFR не ухудшился относительно базового уровня.
- Чемпионы способны обучить нового участника команды за две недели.
- Реализован PDLC Object Model v0.1 для пяти ключевых артефактов.
- Закрыты пять PR/FAQ-driven features с подтверждённым результатом.

Синтетический кейс: банковская организация

Синтетический кейс, отражающий паттерны реальных внедрений:

Q1 2026: решение руководства; бюджет $2 млн/год (до $4 млн к третьему году). Пять чемпионов-инженеров. Platform team: пять человек — PM, architect, platform engineer, инженер среды работы ИИ-агентов, AI Security Officer.

Q2 2026 года уходит на подготовку: команда собирает базовые показатели DORA, в работу запускаются первые спецификации, а агент-ревьюер работает в режиме наблюдения без права блокировать слияния. К третьему кварталу спецификации становятся обязательной практикой для пяти команд, появляется первая версия библиотеки архитектурных паттернов, а агент-ревьюер активируется как блокирующий контролёр. К четвёртому кварталу методику осваивают десять команд, библиотека вырастает до пятнадцати паттернов, а минимальная конфигурация жизненного цикла агентов выводится в промышленную эксплуатацию.

Q1 2027 года расширяет охват до пятнадцати команд, ночной рефакторинг выводится в промышленную эксплуатацию, а библиотека паттернов достигает двадцати пяти записей. Во втором квартале запускается федеративный реестр перехватчиков политик: три домена начинают работать с собственными перехватчиками, оставаясь в общей рамке governance. К третьему кварталу библиотека вырастает до тридцати пяти паттернов, а объяснимость в реальном времени становится обязательной для операций уровня R3 и выше. К концу 2027 года жизненный цикл агентов покрывает полный спектр работ, и первые координированные команды агентов уровня L5 выходят в пилотную эксплуатацию.

К концу 2027 года трансформированы 25 из 30 продуктовых команд; пять отстающих (legacy, regulatory) — на AI-Augmented baseline. К 2028: Outcome Validation Rate ≥ 70%; multi-agent adoption ≥ 30%. Масштаб инвестиций: $10 млн суммарно к 2028 году; реализованные выгоды $30–50 млн+.

Вызов трансформации	**Митигация**
Смена senior-лидерства	Документирование решений; широкая коалиция; постоянное обучение новых лидеров
Бюджетное давление	Сильный ROI-case с метриками; поэтапные инвестиции с чёткими вехами
Уход чемпионов	Достойная компенсация; возможности роста; фокус на удержание
Культурное сопротивление	Добровольный adoption; демонстрация успехов; позитивное peer-давление
Изменения в vendor/tool ecosystem	Независимая от вендора архитектура; слои абстракции; адаптируемые паттерны

7.3 Событийно-управляемые агенты в PDLC

Мы выделяем фоновые и триггерные агенты в отдельный контур поставки — параллельный основному циклу реализации, но не конкурирующий с ним. Headless CI/CD factory расширяется до фабрик длинных сессий: события запускают долгосрочные агентные сессии, результаты которых поступают в валидационный слой и в утреннюю сводку для команды.

Типология агентов по характеру запуска

По данным Gartner, фоновые и триггерные SWE-агенты образуют самостоятельный класс в референсной архитектуре AI-native разработки.

Тип агента	Триггер	Горизонт сессии	Выход
Реактивный CI-агент	Событие в pipeline (fail теста, security-scan, PR)	Минуты	Авто-фикс, security-патч, ревью-комментарий
Фоновый scheduled-агент	Расписание (ночь, выходной, конец спринта)	Часы	Рефакторинг, устранение техдолга, обновление зависимостей
Триггерный проектный агент	Семантическое событие (баг-репорт, новое требование, SLA-алерт)	Часы — дни	PR с фиксом, декомпозиция задачи, spec-черновик

Реактивный CI-агент работает в режиме R2–R3, фоновый scheduled-агент — в R3–R4 с обязательными точками сохранения, триггерный проектный агент требует явного HITL Decision Map до запуска.

Компоненты фабрики длинных сессий

- **Scheduled-триггеры:** сканы безопасности, обновления зависимостей, выявление кандидатов на рефакторинг, ежемесячный анализ тех-долга.
- **Семантические триггеры:** открытие PR, обнаружение уязвимости, баг-репорт, новое требование.
- **Event router:** классифицирует события на подходящие для агентного исполнения и неподходящие; назначает тип агента и уровень автономии.
- **Long-running scheduler:** управляет параллельными сессиями с квотами ресурсов; обеспечивает точки сохранения и Session Handoff Protocol при прерывании.

  Output stream: агрегация пакетов доказательств; ежедневные и еженедельные дайджесты для проверки человеком; обратная связь в Pattern Library через контур telemetry → eval → spec.

Backlog elimination как главный сценарий

Ранее «заблокированный» бэклог — низкоприоритетные улучшения, доработки, технический долг — обрабатывается через long-running factory. 27% работы с участием ИИ составляют задачи, которые иначе не были бы выполнены вовсе: команда производит на 27% больше ценности без изменения численности.

Фоновые агенты работают в выделенном временном окне (ночь, выходной) с заранее определёнными ограничениями среды. Смешение интерактивного и фонового режимов создаёт конфликты контекста — это архитектурная ошибка, а не операционная.

Типовые триггеры: чек-ин кода → ревью и security-скан; фейл unit-теста → авто-фикс; баг-репорт → фикс + PR; новое требование → декомпозиция; ночной scheduled-запуск → overnight refactoring. Для каждого триггера заранее определяются: тип агента, R-уровень, условия эскалации, формат выхода. Запуск без этой конфигурации — антипаттерн #13.

7.4 Антипаттерны трансформации

Мы фиксируем антипаттерны внедрения, распределённые по горизонтам организационной зрелости. Каждый антипаттерн имеет признак раннего предупреждения, структурный фикс и типичный горизонт проявления. Ниже — семь самых частых в первый год, затем полная таблица с базовыми и управленческими антипаттернами.

«Полный цикл от вендора» (vendor lock-in)

Команда строит весь стек вокруг проприетарных API одного поставщика. На горизонте 12–18 месяцев смена или обновление модели превращается в проект по переписыванию инфраструктуры. Ранний признак — более 70% зависимостей завязаны на одного провайдера, абстрактные адаптеры отсутствуют. Структурный фикс — вендор-нейтральные протоколы MCP, A2A и AG-UI как обязательный стандарт среды (принцип 10) и абстракционные слои с первого спринта.

«Десятки директив вместо принципов»

По данным мировых исследовательских компаний, модели одновременно следуют лишь нескольким десяткам директив, и перегрузка ограничителей делает их декоративными. Когда активных правил в среде больше 40–50, агент игнорирует их из-за перегруженного контекста, и принудительное исполнение становится случайным. Деградация не проявляется как явный сбой — она видна только в метриках: медленный рост CFR (Change Failure Rate, коэффициент неудачных изменений), снижение доли подтверждённых результатов. Структурный фикс — ежеквартальный аудит реестра правил, приоритизация по критичности, архивирование устаревших директив и назначение владельца каждому правилу.

«Подмена модели вместо перепроектирования среды»

Команда реагирует на низкое качество вывода стандартным рефлексом — сменить модель. Переход занимает два спринта, результат улучшается незначительно, цикл повторяется, а среда работы ИИ-агентов остаётся непроработанной. Ранний признак — команда меняла модель дважды за квартал без измеримого улучшения доли подтверждённых результатов. Структурный фикс — аудит среды по чек-листу (контекст, ограничители, набор оценок) перед каждой сменой; решение о смене принимается только после исключения средовых факторов.

«Vibe coding без оценок»

Команда разворачивает ИИ-ассистированное кодирование, опираясь на субъективное ощущение «работает лучше». Нет систематических оценок, нет фитнес-функции, нет проверочного пакета. Регрессии не замечаются до накопления критической массы дефектов. Ранний признак — в SDD нет раздела «Eval Strategy», реестр оценок отсутствует, карта решений с участием человека не заполнена. Структурный фикс — разработка от оценок как обязательное требование (принцип внедрения №5); SDD без раздела оценок блокируется на ИИ-контроле.

«Big bang миграция (одним скачком) вместо инкрементального подхода»

Организация запускает масштабное одновременное внедрение — все команды в одном квартале. Платформенная команда перегружена, проводников не хватает, качество внедрения падает, через полгода следует откат под нарративом «ИИ не работает». Ранний

признак — план не содержит критериев выхода из горизонта, масштабирование запланировано до завершения 90-дневного MVP. Структурный фикс — обязательная пилотная фаза (принцип 2), масштабирование только по критериям выхода из Горизонта 1, не более двух-трёх команд на одного проводника одновременно.

«Безграничный auto-approve (усталость от согласований)»

Под давлением скорости команда минимизирует точки подтверждения человеком. Через два-три месяца разработчики штампуют предложения агента без реального ревью — прямое проявление усталости от согласований (Approval Fatigue). Ранний признак — среднее время на ревью агентного PR ниже 2 минут, доля согласий выше 95%, CFR растёт. Структурный фикс — адаптивная лестница автономии (принцип внедрения №8) с явными точками участия человека для R3 и выше, контекстно-зависимые требования к ревью; владелец результата — всегда человек.

«Метрики без целей»

Команда внедряет дашборд — DORA, I/V Tempo Ratio, Reallocation Rate — регулярно его обновляет, но целевые значения не зафиксированы. Пороги предупреждений не установлены, ретроспектива не привязана к отклонениям, метрики становятся декоративными. Ранний признак — за последний квартал ни одно решение о корректировке процесса не принято на основании дашборда. Структурный фикс — для каждой метрики три значения (базовое, целевое, порог тревоги); ретроспектива начинается с обзора отклонений; метрики без цели удаляются.

Базовые антипаттерны (№1–18) сведены в единый реестр с признаком и митигацией.

No	Антипаттерн	Признак	Митигация
1	Зоопарк ИИ-инструментов	множество инструментов без оркестрации	IDP с единой средой исполнения
2	Культурный балласт	формальный доступ к ИИ, работа по старым процессам	проводники изменений, обучение по ролям
3	ИИ только в кодировании	ИИ применяется только в кодировании (25–35% времени)	применение ИИ на всех фазах PDLC
4	Инерционный вендор-лок	зависимость от одного поставщика	стандартные протоколы MCP/A2A
5	Пилоты не масштабируются	пилоты успешны, но не выходят за рамки одной команды	проводники изменений; политика перераспределения ресурсов
6	Внедрение без обучения по ролям	нет системного обучения	программы наставничества
7	Ожидание корпоративного стандарта	ожидание общекорпоративного стандарта до начала внедрения	доменно-специфичные решения на общей платформе

No	Антипаттерн	Признак	Митигация
8	Игнорирование рисков безопасности	игнорирование рисков агентной инфраструктуры	security-by-design на старте
9	Спецификация без Discovery	SDD без гипотезы результата, без сверки с библиотекой паттернов	обязательный суб-цикл исследования перед SDD
10	Бесконтрольный рост рабочих процессов	200+ агентов без реестра, владельца и жизненного цикла	единый реестр с владельцем и жизненным циклом
11	Сэкономленное время уходит в никуда	время сэкономлено, но не превращено в результат	метрика доли перераспределения с первых пилотов; принцип внедрения №12
12	Валидация не успевает за реализацией	реализация ускорилась в 5 раз, валидация — узкое место	I/V Tempo Ratio в еженедельном дашборде; принцип внедрения №12
13	Безоткатный запуск длительного агента	long-running агент без среды работы ИИ-агентов и точек сохранения	среда работы ИИ-агентов с протоколом передачи сессий
14	Гонка за объёмом без бизнес-эффекта	много PR от ИИ без бизнес-ценности	метрики объёма всегда в паре с метриками бизнес-результата
15	Перепрыгивание уровней автономии	длинные автономные сессии при недостаточной зрелости команды	контрольные точки перехода между уровнями
16	Мультиагентность без оркестрации	несколько агентов без оркестратора — конфликты и хаос	обязательный оркестратор над агентами-исполнителями
17	Надежда на полную автономию	задача делегируется агенту целиком без точек участия человека	явное разделение шагов на автономные и требующие подтверждения
18	Разрастание семантического слоя	команда PDLC берётся за бизнес-схемы вместо схем артефактов	семантика бизнес-данных вне зоны ответственности PDLC

Антипаттерны управления (№19–22) сведены в отдельный реестр с симптомом и структурным фиксом.

No	Антипаттерн	Симптом	Структурный фикс
19	Управление, прикрученное сбоку	проверка соответствия занимает больше времени, чем разработка; задачи блокируются на релизе	кодифицированные ограничения и Policy-as-Code с контуром на исследовании, а не на релизе
20	Статичная лестница R-уровней	одинаковый надзор для документации и платёжной логики	контекстно-зависимая автономия
21	Адаптивные контроли без отучивания	набор политик растёт на 10–20% в месяц без удаления	механизмы отучивания: регулярный пересмотр, явные операции «забывания»
22	ИИ-надзор как индульгенция	разработчики штампуют предложения агента без ревью; доля дефектов растёт	обучение ответственным практикам; владельцем остаётся человек

Антипаттерны как опережающие индикаторы

- **Спецификация без исследования:** рост доли SDD без раздела «гипотеза результата».
- **Бесконтрольный рост рабочих процессов:** шаблоны и процессы без владельца или с устаревшей оценкой.
- **Сэкономленное время уходит в никуда:** доля перераспределения ниже 0,3 в течение двух кварталов.
- **Валидация не успевает за реализацией:** I/V Tempo Ratio выше 2; рост числа открытых пул-реквестов.
- **Безоткатный запуск длительного агента:** сессии без точек сохранения; высокая доля повреждённых сред.
- **Гонка за объёмом без бизнес-эффекта:** рост объёма вывода при стагнирующей доле подтверждённых результатов.
- **Перегрузка ограничителей:** более 40–50 активных правил среды-слоя без регулярного аудита.

Антипаттерны по уровням зрелости

Горизонт	Типичные антипаттерны
Горизонт 1	пилоты не масштабируются; зоопарк ИИ-инструментов; культурный балласт; big bang миграция
Горизонт 2	спецификация без исследования; бесконтрольный рост рабочих процессов; валидация не успевает; vibe coding без оценок
Горизонт 3	перепрыгивание уровней автономии; мультиагентность без оркестрации; надежда на полную автономию; метрики без целей

Развёрнутые кейсы антипаттернов

Кейс «Сэкономленное время уходит в никуда». Команда внедрила ИИ-ассистированное кодирование; инженерные часы сократились на 15%; перераспределение высвободившегося времени не измерялось. Инженеры тратили его на встречи; CFO не видит эффекта в P&L. Митигация: введена метрика доли перераспределения; ежеквартальный обзор с топ-менеджментом; команда обязана демонстрировать, куда ушло сэкономленное время.

Кейс «Безоткатный запуск длительного агента». Команда запустила агента для ночного рефакторинга без среды работы ИИ-агентов; агент проработал шесть часов и упал; повреждённое состояние обнаружено утром; потерян день продуктивности. Митигация: протокол передачи сессий как предусловие; базовая среда работы ИИ-агентов обязательна до запуска любой долговременной сессии.

Кейс «Валидация не успевает за реализацией». ИИ-ассистированное кодирование ускорило коммиты в пять раз; процесс ревью не модернизировался; очередь пул-реквестов росла экспоненциально; CFR вырос с 8% до 14% за три месяца. Митигация: агент-ревьюер в режиме pre-merge; I/V Tempo Ratio в еженедельном дашборде; параллельные инвестиции в контур валидации.

Кейс «Бесконтрольный рост рабочих процессов». За шесть месяцев команда из 50 человек создала 200+ агентных рабочих процессов без реестра и владельцев; дубли; несогласованное качество; рабочий процесс команды A сломал промышленную среду команды B. Митигация: обязательный реестр с жизненным циклом; ежеквартальный аудит; единственный владелец на каждый процесс.

Каждый антипаттерн проявляется в метриках раньше, чем в результатах; поэтому мы отслеживаем их как опережающие индикаторы, а не как список ошибок для запоминания.

7.5 Трансформация людей и инструментов

Трансформация в AI-Disrupt PDLC разворачивается по двум параллельным трекам — людей и инструментов. Инструмент без людей превращается в инструмент, лежащий на полке; люди без инструмента создают теневой ИТ-контур (shadow IT). Параллельность треков — требование, а не рекомендация.

Команды, последовательно делающие сначала «инструменты», потом «людей», теряют первые шесть месяцев на инструмент, лежащий на полке, а вторые — на доработку под людей, сформировавших собственные обходные практики.

Трек людей: компетенции, роли и культура

Трек людей охватывает три направления: компетенции, роли и культуру.

Компетенции задаются двумя навыками. Валидационная грамотность — способность инженера оценить качество агентного вывода — базовое требование к каждому члену команды, а не опциональный навык. Фасилитация исследования — способность провести PR/FAQ-сессию и сформулировать гипотезу результата — распределяется между проводниками на Горизонте 1 и становится массовым навыком к Горизонту 2.

Новые роли появляются как специализация существующих функций. AgentOps Engineer расширяет DevOps в сторону управления жизненным циклом агентных приложений. ИИ Security Officer расширяет AppSec в сторону специфических рисков агентной инфраструктуры. Обе роли могут стартовать как ролевые расширения — особенно в командах Горизонта 1.

Культурная трансформация опирается на три механизма:

- добровольное внедрение на ранних стадиях: принуждение убивает эксперимент;
- позитивное давление коллег через видимые успехи проводников;
- честная обратная связь на основе метрик.

Трек инструментов

Возможность (capability)	**Минимум (Горизонт 1)**	**Целевое состояние (Горизонт 3)**
Реестр спецификаций (Specification Registry)	шаблон SDD в Markdown в Git	структурированный реестр с поиском, версионированием и аналитикой повторного использования
Библиотека паттернов (Pattern Library)	вики с 5–10 паттернами	библиотека 30+ паттернов с телеметрией использования
Агент-ревьюер	промпт в ChatGPT/Claude в теневом режиме	интегрированный агент-ревьюер в CI/CD с блокирующей проверкой до слияния
Шина перехватчиков политик (Policy Hook Bus)	базовые перехватчики в CI	федеративный реестр перехватчиков политик с доменными перехватчиками
Среда исполнения долговременных процессов (Long-Running Runtime)	нет	протокол передачи сессий (Session Handoff Protocol); ночные агенты по расписанию
Объяснимость в реальном времени	mock-объяснения в режиме только чтения	обязательная объяснимость для R3+; покрытие объяснениями> 99%
Цикл разработки ИИ-агентов (ADLC)	нет	полный цикл: набор оценок, обнаружение отклонений, переобучение, вывод из эксплуатации

Распределение усилий по горизонтам

Горизонт	**Люди**	**Инструменты**	**Акцент**
Горизонт 1	60%	40%	проводники изменений, обучение, мышление спецификациями
Горизонт 2	40%	60%	библиотека паттернов, среда длительных агентов, минимальный ADLC
Горизонт 3	30%	70%	полный ADLC, многоагентные конфигурации, объяснимость в реальном времени

Это ориентир, а не догма. Команда без проводников провалит даже отличную библиотеку паттернов; команда с проводниками, но без среды работы длительных агентов, будет тормозить на операционных проблемах, не связанных с методологией.

Метрики успеха трансформации

Ранние сигналы:

- проводники обучены и активны;
- доля внедрения ритуалов исследования;
- частота использования шаблона SDD;
- доля согласий с агентом-ревьюером.

Индикаторы середины пути (год 1):

- рост объёма вывода;
- снижение стоимости токенов на фичу (Token Cost per Feature);
- доля повторного использования спецификаций (Spec Reuse Rate) ≥ 10%;
- CFR стабилен или улучшается.

Итоговые индикаторы (год 2–3):

- доля подтверждённых результатов (Outcome Validation Rate) ≥ 70%;
- внедрение мультиагентности (Multi-Agent Adoption) ≥ 30%;
- медианный горизонт задачи (Median Task Horizon) ≥ 4 часа;
- доля перераспределения (Reallocation Rate) ≥ 0,5;
- организация способна самостоятельно развивать PDLC.

Сводка плейбука трансформации

- **Планирование:** обязательство руководства; проводники; платформенная команда; пилоты с чёткими критериями успеха.
- **Исполнение:** 90-дневный MVP; обучение по ролям; постепенное масштабирование по критериям выхода из горизонта; честная самооценка через метрики.
- **Управление:** принудительное исполнение принципов через CI/CD; удержание границ зон ответственности; мониторинг антипаттернов как опережающих индикаторов; адаптация на основе реальных данных.
- **Поддержка:** непрерывное развитие библиотеки паттернов и среды работы ИИ-агентов; инвестиции в людей и инструменты в правильной пропорции по горизонтам; честные метрики; конкурентный бенчмаркинг как дисциплина.

Это ритм успешного внедрения — дисциплинированное, структурированное исполнение на основе доказательств. Организации, соблюдающие этот ритм, к концу Горизонта 3 меняют масштаб возможного при той же численности команды.

Ритм внедрения универсален, а контекст исполнения — нет. Часть 8 калибрует подход под российскую специфику: 152-ФЗ, импортозамещение, отечественные модели, ограничения

SaaS, контур управления в условиях санкций. Принципы и горизонты остаются теми же — меняются инструментальный слой, регуляторные требования и доступные вендорские опции.

Часть 8. Российский корпоративный контекст

Концепция AI-Disrupt PDLC откалибрована под международный ландшафт ИИ-нативной разработки — российский корпоративный контур требует отдельной калибровки. Эта часть отвечает на один вопрос: что из концепции применимо в РФ сегодня, а что упирается в регуляторные и организационные барьеры, которые не обходятся архитектурой. Мы разбираем регуляторный контур, рынок поставщиков и кадров, policy-as-code, контур управления, метрики и дорожную карту локализации.

Российская среда — не ещё одна юрисдикция, а другой режим допуска технологии в эксплуатацию. Это структурно иная среда: другой регуляторный режим, другой ландшафт поставщиков решений, другие точки риска. Вместе с тем ключевой тезис концепции — среда работы ИИ-агентов важнее модели — работает здесь с удвоенной силой: именно суверенная среда исполнения является единственным надёжным ответом на ограничения, которые российский контекст накладывает на доступ к зарубежным провайдерам.

Ключевой принцип: В российском корпоративном контексте собственная среда исполнения (harness layer) перестаёт быть архитектурным предпочтением и становится регуляторной необходимостью.

8.1 Регуляторные контуры

Внешний регуляторный контур

Российское законодательство формирует несколько уровней обязательных ограничений для ИИ-систем, обрабатывающих данные в корпоративном контексте. Эти ограничения напрямую определяют архитектурные решения: выбор модели, режим инференса, политику логирования, топологию сети и требования к аудиту.

Регулятор / Норма	**Ключевые требования к ИИ-системам**	**Затрагиваемые архитектурные слои**
152-ФЗ «О персональных данных»	Локализация обработки персональных данных на территории РФ; запрет передачи ПДн за рубеж без согласия субъекта и оснований; требование к документированию цели и сроков обработки	Инференс, конвейер данных, журнал аудита, выбор модели
ЦБ РФ, Положение 683-П / 757-П	Обеспечение информационной безопасности в кредитных организациях; требования к защищённому хранению и передаче данных; обязательный аудит ИТ-систем, задействованных в операционно значимых процессах	Policy Hook Framework, Audit Trail, среда исполнения агента

Регулятор / Норма	Ключевые требования к ИИ-системам	Затрагиваемые архитектурные слои
ФСТЭК РФ, Приказы 17/21/239	Защита информации критической информационной инфраструктуры (КИИ); сертификация средств защиты информации; требования к разграничению доступа и сегментации сети	Инфраструктура IDP, Guardian Agents, лестница разрешений
Указ Президента РФ №166 (2022)	Запрет для субъектов КИИ (заказчиков по 223-ФЗ) закупать иностранное ПО для значимых объектов КИИ без согласования (с 31.03.2022) и использовать его (с 01.01.2025; с 07.04.2025 — «если иное не установлено федеральным законом»)	Выбор модели, инструментарий, IDE, CI/CD
Указ Президента РФ №250 (2022)	Персональная ответственность руководителя за ИБ; создание подразделения по обеспечению ИБ; запрет с 1 января 2025 года на использование средств защиты информации из недружественных стран; доступ ФСБ к информационным ресурсам	Guardian Agents, Audit Trail, слой SRE
ФЗ-187 «О безопасности КИИ»	Категорирование объектов КИИ; обязательное подключение к государственной системе обнаружения атак (ГосСОПКА); инцидент-репортинг	Слой SRE, Guardian Agents, конвейер инцидентов

Совокупное действие этих норм формирует две ключевые архитектурные зоны: внешний контур — требования регуляторов, которые поставляют policy-сигналы верхнего уровня в контур управления, и внутренний контур — операционные политики организации, которые кодифицируют эти сигналы в конкретные правила исполнения.

Внутренний операционный контур

Внутренний контур ограничений включает несколько классов политик, которые организация обязана поддерживать независимо от того, какой внешний регулятор их предписывает:

- **Операционные риски информационной безопасности** — политики управления привилегированным доступом, сегрегация сред, контроль supply chain агентного инструментария.
- **Санкционный и комплаенс-скрининг** — проверка контрагентов, товаров и транзакций через агентные пайплайны на соответствие санкционным спискам (Росфинмониторинг, OFAC, EU).
- **Ограничения на использование иностранных языковых моделей для задач с ПДн и данными ограниченного распространения** — фактический запрет на передачу таких данных в облачный инференс зарубежных провайдеров.
- **Требования к трассируемости ИИ-решений** — для финансовых организаций и субъектов КИИ решения, принятые с участием ИИ-агентов, должны быть документированы и воспроизводимы в ходе регуляторной проверки.

Эта совокупность требований делает отечественный инференс не «опциональным предпочтением»,
а обязательным компонентом архитектуры для большинства enterprise-сценариев в России.

8.2 Ландшафт поставщиков и кадров

Ландшафт поставщиков

Российский корпоративный сегмент работает в условиях структурных ограничений доступа к зарубежной ИИ-инфраструктуре. Это формирует специфический ландшафт поставщиков с четырьмя основными категориями.

Категория	Примеры	Статус доступа	Рекомендации по применению
Зарубежные проприетарные модели	OpenAI GPT-5.5, Anthropic Claude Opus 4.8, Google Gemini 3.1 Pro	Ограничен или закрыт для критических задач	Только через gateway-режим с MCP-прокси; не для critical workloads и задач с ПДн
Отечественные коммерческие модели	GigaChat 3 Ultra (Сбер), YandexGPT 5.1 Pro (Яндекс), T-Pro 2.0 (T-Банк), Cotype Pro 2 (МТС)	Доступен, с локализацией и поддержкой на русском языке	Приоритет для задач с ПДн, регуляторным контекстом, финансовыми данными
Open-source self-hosting	GLM-5.2 (Z.ai), DeepSeek V4, Qwen 4 (Alibaba), MiniMax M3	Доступен для on-prem развёртывания без передачи данных внешнему провайдеру	Оптимально для КИИ при наличии GPU-кластеров; требует MLOps-экспертизы

Категория	Примеры	Статус доступа	Рекомендации по применению
Агрегирующие шлюзы и прокси	Корпоративные MCP-прокси, внутренние AI Gateway	Настраивается внутренне	Рекомендуется как архитектурный стандарт для vendor-agnostic подхода и контроля трафика

Ключевой архитектурный принцип остаётся неизменным: среда работы ИИ-агентов должна быть отделена от конкретной модели. Это позволяет менять базовую модель без переписывания агентной логики, поддерживать гибридный режим (отечественная модель для ПДн, зарубежная — для экспериментальных сред), снижать риск vendor lock-in и обеспечивать ИИ-суверенитет организации. Именно качество среды-как-продукта, а не выбор конкретного продукта, определяет разброс результата до 22 п.п. на одной и той же модели— это делает инвестиции в среду первоочередными вне зависимости от выбранного провайдера.

Кадровый ландшафт

Российский рынок инженерных кадров характеризуется специфическим профилем, который влияет на темп и стратегию трансформации к ИИ-нативным практикам.

Характеристика	Текущее состояние	Следствие для трансформации
Техническая глубина	Высокая — сильная база в алгоритмах, системном программировании, математике	Быстрое освоение IDP-инструментов и практик SDD (мышление через спецификацию)
Опыт в агентных методологиях	Ограниченный — меньше практического опыта с агент-нативными процессами по сравнению с западными рынками	Требует целенаправленного обучения, пилотных программ и сетей внутренних power users
Объём legacy-мышления	Ниже, чем в западных корпорациях — меньше «груза» десятилетий waterfall	Возможность «перепрыгнуть» этапы, внедряя сразу ИИ-нативные практики без переучивания
Языковой барьер	Частичный — документация, сообщества и инструменты преимущественно англоязычные	Необходимость локализации знаний, инструментов и внутренних программ подготовки power users
Доступность передовых инструментов	Ограниченная — доступ к GitHub Copilot, Cursor, Claude Code частично закрыт или требует VPN	Потребность в отечественных аналогах и on-prem-развёртывании среды разработки

Стратегия развития компетенций опирается на два механизма:

- **Модель наставничества (apprenticeship)**: год практической работы под руководством senior product engineer даёт больше, чем короткие курсы. Младшие инженеры без «багажа» классических процессов быстрее осваивают мышление через спецификацию (specification thinking) и культуру валидации ИИ-результатов.
- **Внутренние программы power users**: выделение 5–7 инженеров-проводников на 50+ команд, которые транслируют практики через обучение равных равными (peer-to-peer).

Отдельное условие масштабирования — локализованные образовательные артефакты: шаблоны SDD, записи в Pattern Library, runbook'и на русском языке с привязкой к российскому регуляторному контексту. Без них трансформация не выходит за пределы пилотных команд: большинство инженеров не может осваивать практики по англоязычным первоисточникам в рабочем темпе.

8.3 Policy-as-Code Library «RU Compliance Core»

Свод регуляторных знаний

Библиотека «RU Compliance Core» — это внутренняя коллекция машиночитаемых политик, которые кодифицируют ключевые требования российского регуляторного контура в формате, исполняемом Policy Hook Framework. Библиотека является частью контура управления (Governance Mesh) и поставляет кодифицированные ограничения (Codified Guardrails) для всех SDD-спецификаций, касающихся обработки данных.

Архитектурный принцип: compliance перестаёт быть финальным гейтом — традиционным источником задержек перед выкаткой в продакшн — и становится сквозным свойством ИИ-нативной разработки. Каждая агентная задача, затрагивающая данные, автоматически наследует применимые политики из библиотеки через Policy Hook. Инженер не должен помнить о 152-ФЗ — Policy Hook напомнит сам.

Состав библиотеки:

Политика	Регуляторная основа	Механизм применения	Описание
PII-Localization Guard	152-ФЗ, ст. 18.1	Pre-action hook + блокировка	Запрет передачи персональных данных за пределы одобренного контура инференса; автоматическая аннотация токенов с ПДн
Data Retention Policy	152-ФЗ, ЦБ РФ 683-П	Post-action hook	Автоматическое применение сроков хранения к артефактам агентной сессии; триггер retention cleanup по расписанию
Processing Purpose Declaration	152-ФЗ, ст. 9	Pre-task gate	Требование наличия задокументированной цели обработки в SDD-спецификации до запуска агентной задачи с ПДн

Политика	Регуляторная основа	Механизм применения	Описание
Immutable Audit Trail	ЦБ РФ, ФСТЭК	Continuous hook	Append-only запись всех событий жизненного цикла сессии в JSONL-формате с пятилетним сроком хранения (152-ФЗ, ЦБ РФ 683-П)
Domestic Inference Gate	152-ФЗ, Указ №166, ФСТЭК	Hard block	Блокировка отправки данных ограниченного распространения на endpoints зарубежных провайдеров
Sanctioned Entity Check	Росфинмониторинг, международные списки	Pre-action hook	Проверка упоминаемых контрагентов, физических лиц и организаций по санкционным спискам в агентном пайплайне
Privileged Access Control	ФСТЭК Приказ 17/21	Permission gate	Ограничение уровня автономии агентов R3+ для операций с данными ограниченного распространения
Incident Reporting Hook	ФЗ-187, ГосСОПКА	Post-event hook	Автоматическая генерация отчёта об инциденте при обнаружении аномального поведения агента в контуре КИИ

Принципы поддержки библиотеки

Библиотека поддерживается командой платформы совместно с юридической службой и службой информационной безопасности по модели pull-request с обязательным review. Каждая политика имеет версионирование, changelog и набор регрессионных проверок (regression evals), которые запускаются при каждом изменении. Это позволяет вносить регуляторные изменения без остановки продакшн-конвейеров: новая версия политики проходит набор проверок (eval-suite), получает одобрение и автоматически распространяется в контур управления всех команд, использующих библиотеку.

Принцип «Compliance-as-Code»: регуляторное требование, не кодифицированное как Policy Hook, — это требование, выполнение которого зависит от памяти и дисциплины конкретного инженера. В условиях ИИ-нативной разработки, где один инженер оркестрирует десятки агентных задач параллельно, такая зависимость системно опасна.

8.4 Governance Mesh для российского контекста

Архитектура контура управления

Контур управления (Governance Mesh) в российском контексте усложняется: он должен одновременно исполнять лучшие мировые практики и специфические требования российского регулирования. Это не две параллельные системы, а единый механизм управления с российским расширением.

Внешний регуляторный контур поставляет сигналы политик верхнего уровня. Библиотека «RU Compliance Core» транслирует их в машиночитаемые Policy Hooks. Policy Hook Framework исполняет их как принудительно применяемые ограничения на каждом этапе цикла реализации. Governance Mesh замыкает контур: он агрегирует сигналы из библиотеки, из метрик

эффективности и из Guardian Agents, формируя сводную картину compliance-статуса в реальном времени.

Двухконтурная модель

В российском корпоративном контексте мы выделяем два контура управления, которые работают параллельно и взаимно усиливают друг друга.

Контур регуляторного соответствия обеспечивает исполнение требований 152-ФЗ, ЦБ РФ и ФСТЭК. Его ключевые механизмы — Policy Hooks на уровне данных, неизменяемый журнал аудита (immutable audit trail) и шлюз отечественного инференса (domestic inference gate).

Контур операционной безопасности защищает от угроз, специфичных для агентных систем в российской инфраструктуре:

- prompt injection с регуляторным контекстом (например, внедрение инструкций для обхода
  шлюза отечественного инференса);
- утечки через MCP-серверы;
- несанкционированный экспорт артефактов сессий.

Основной инструмент этого контура — Guardian Agents.

Compliance как непрерывное свойство

Традиционная модель compliance в корпорациях — это финальный гейт перед продакшном: специальная команда проверяет соответствие и даёт разрешение на выкатку. В ИИ-нативной разработке она не работает: скорость агентных конвейеров превышает скорость ручных проверок, и изменения выходят в продакшн быстрее, чем команда compliance успевает их проверить.

Наш ответ — compliance как непрерывное свойство среды исполнения. Policy Hook проверяет каждую операцию с данными в момент её инициации, а Guardian Agent отслеживает поведение агента в реальном времени. Так compliance перестаёт быть отдельным этапом и становится встроенной характеристикой каждой задачи. Это особенно важно для финансовых и государственных организаций, где разрыв между «ИИ-эффективностью» и «соответствием требованиям» исторически был главной причиной блокировки выкаток агентных возможностей в продакшн.

8.5 Governance metrics — российская специфика

Универсальные показатели эффективности governance — покрытие политиками, полнота журнала аудита, время обнаружения инцидента, среднее время восстановления (MTTR), соотношение одобрений и принудительных переопределений — мы подробно разбирали ранее. В этом разделе фиксируем только то, что специфично для российского контура: метрики, привязанные к 152-ФЗ, требованиям ЦБ РФ и режиму КИИ.

Compliance-метрики 152-ФЗ:

- **Уровень соответствия по субъектам данных** измеряет долю агентных задач с ПДн, в которых зафиксировано согласие субъекта и задокументирована цель обработки в SDD-спецификации до запуска задачи. Целевое значение — 100%. Значение ниже 100% является потенциальным регуляторным инцидентом: нет понятия «приемлемый» процент необоснованной обработки ПДн.
- **Регистр операций с ПДн** отслеживает полноту реестра обрабатываемых категорий ПДн согласно ст. 18.1 152-ФЗ. Метрика — доля категорий ПДн, задокументированных в актуальном реестре. Целевое значение — 100%.
- **Согласие субъектов** контролирует наличие действующего согласия или законного основания для каждой активной операции обработки в агентных сессиях. Признак деградации — рост числа операций без явного основания при масштабировании агентных задач.
- **Полнота хранения журналов аудита** (по 152-ФЗ и требованиям ЦБ РФ): журналы агентных сессий хранятся 5 лет в защищённом хранилище (изолированном от производственной среды, с ограниченным доступом и контролем целостности). Метрика измеряет долю событий, сохранённых с соблюдением этого требования. Признак системной проблемы — значение < 99,5%.

Отечественные модели и импортозамещение

Доля использования отечественных моделей измеряет долю агентных операций через отечественные модели или одобренный шлюз (approved gateway) против операций через неодобренные внешние конечные точки (endpoints). Метрика включает три подпоказателя: наличие сертификации Роскомнадзора, соответствие требованиям локализации данных (data residency; серверы на территории РФ), наличие конечной точки в одобренном списке поставщиков. Для объектов КИИ и задач с ПДн целевое значение — 100%. Низкий показатель сигнализирует о теневом ИИ (shadow AI).

Уровень суверенности IDP (%) — доля компонентов платформы разработки, реализованных на отечественном или open-source ПО без зависимости от иностранных облачных сервисов. Отслеживается по карте компонентов; ключевой показатель для субъектов КИИ по Указу №166.

Метрика	Целевое значение	Сигнал тревоги	Ответственный
Уровень соответствия по субъектам данных (152-ФЗ)	100%	< 100% → инцидент	DPO + Platform team
Регистр операций с ПДн	100%	< 100% → пробел в реестре	DPO
Полнота хранения журналов аудита	100%	< 99,5% → системная проблема	AgentOps + Security
Доля использования отечественных моделей (КИИ)	100%	< 100% → теневой ИИ	AI Security Officer

Метрика	Целевое значение	Сигнал тревоги	Ответственный
Уровень суверенности IDP	По дорожной карте	Стагнация > 2 кварталов	CTO + Platform team

8.6 Guardian Agents в российском контексте

Специфика вендорского ландшафта

Guardian Agents в российском контексте сталкиваются с принципиально иным вендорским ландшафтом по сравнению с международной картиной. Зрелых отечественных поставщиков Guardian Agent на уровнях T3–T5 практически нет; T1–T2 частично покрываются возможностями отечественных CI/CD-платформ и кастомных интеграций.

Уровень Guardian Agent	Описание функции	Международные решения	Российские кандидаты	Статус
T1 — Review	Автоматическое ревью кода и артефактов	CodeRabbit, GitHub Copilot review	GitVerse с plugin API, SonarQube CE self-hosted, внутренние lint-агенты	Доступно (OSS + on-prem)
T2 — Verification	Верификация соответствия требованиям и политикам	Checkmarx One, Snyk	SonarQube CE, PT Application Inspector (Positive Technologies), Solar appScreener	Доступно с ограничениями
T3 — Redirect	Перенаправление агента при нарушении политики	AgentHQ, собственные	Требует кастомной разработки на базе OSS	Только build
T4 — Block	Блокировка действия агента с escalation	AgentHQ, enterprise guardrails	Кастомная разработка + Policy Hook Framework	Только build
T5 — Remediate	Автоисправление нарушений без участия человека	Copilot Autofix, Codex remediation	Кастомная разработка; нет зрелых on-prem решений	Только build (high effort)

Стратегия «собственная разработка или покупка» для российских корпораций

Для уровней T1–T2 российские корпорации имеют рабочие опции: CodeRabbit self-hosted, SonarQube Community Edition, PT Application Inspector и Solar appScreener закрывают базовые функции ревью и верификации. Они развёртываются on-prem, не требуют передачи кода во внешнее облако и могут быть интегрированы в GitVerse или Bitbucket.

Для уровней Т3–Т5 ситуация принципиально иная. Зрелых вендорских решений, гарантирующих автономную работу без отправки кода в облако, на российском рынке сегодня нет. Это означает, что Guardian Agents уровней Т3–Т5 становятся капитальными инвестициями для российского IDP — собственная разработка на базе open-source-инструментов авторемедиации для on-prem-развёртывания.

Это тот тип инвестиций, который амортизируется на годы: построенный Т4-агент блокирует сотни нарушений политик без участия человека, а Т5-агент устраняет тысячи мелких проблем, которые иначе требовали бы ручного вмешательства. Логика амортизации среды работает здесь в полной мере.

Российский контур Guardian Agents как конкурентное преимущество

Организации, которые инвестируют в собственный контур Guardian Agents сейчас, получат структурное преимущество: когда регуляторные требования к трассируемости ИИ-решений ужесточатся, у них уже будет работающий контур. Траектория задана: EU AI Act (Regulation (EU) 2024/1689) уже вводит для систем high-risk обязательные требования к трассируемости, логированию и record-keeping, а российское право последовательно адаптирует аналогичные нормы. Организации, отложившие эти инвестиции, столкнутся с необходимостью экстренной догоняющей разработки в условиях дефицита времени и компетенций.

8.7 Референс-архитектура

Референс-архитектура ИИ-нативной разработки программного обеспечения включает десять компонентов, которые в совокупности формируют полный технологический стек. Каждый из этих компонентов по-разному локализуется в российском контексте: одни доступны «из коробки», другие требуют замены, третьи отсутствуют на отечественном рынке и требуют собственной разработки.

No	**Компонент**	**Назначение**	**Локализуемость**	**Российский кандидат / подход**	**Разрыв**
1	Агентные IDE	Среда разработки со встроенным ИИ-агентом (Cursor, Windsurf, Kiro, VS Code+Copilot)	Частичная	GigaIDE Pro и GigaIDE Community с мультиагентным GigaCode; JetBrains с AI Assistant; VS Code с open-source AI-расширениями	GigaIDE — ближайший отечественный аналог агентной IDE; agentic-глубина пока уступает Cursor на сложных задачах; сильные стороны — Java, Kotlin и поддержка 1С

No	Компонент	Назначение	Локализуемость	Российский кандидат / подход	Разрыв
2	CLI-агенты (SWE)	Терминальные ИИ-агенты для автономных задач (Claude Code, Codex CLI, Gemini CLI)	Низкая	GigaCode CLI (мультиагентный, on-prem, автономная сборка, тестирование, поиск уязвимостей); SourceCraft CLI; open-source Aider, OpenHands self-hosted с локальной моделью	GigaCode CLI и SourceCraft CLI — зрелые отечественные CLI-агенты с on-prem-развёртыванием; многошаговая автономность на сложных задачах пока догоняет Claude Code и Codex CLI
3	Фоновые SWE-агенты	Событийно-триггерные агенты (PR → автокоммит, фейл теста → автофикс)	Средняя	GigaCode-агент в GitVerse; SourceCraft; Bitbucket и Gitea с webhooks	Есть работающие отечественные фоновые агенты; зрелость авторемедиации пока ниже Copilot Autofix
4	Системы контекста	Репозитории, требования, чаты, документы, графы знаний, IDP	Высокая	Confluence self-hosted, GitVerse, внутренние knowledge bases, on-prem vector stores (Qdrant, Milvus)	Минимальный разрыв при on-prem развёртывании; основная задача — интеграция в единый контекстный граф
5	Governance агентов	FinOps, guardrails, access control, управление ролями (Harness Inc., Microsoft AgentHQ)	Низкая (vendor-уровень)	RU Compliance Core и Policy Hook Framework — собственная разработка; GigaView для оценки продуктивности и авто-ревью	Нет отечественного vendor-решения уровня Harness; build-путь обязателен; Policy Hook Framework закрывает ядро функциональности
6	ИИ-планирование agile	ИИ-ассистент для планирования, распределения задач, уточнения требований (Atlassian Rovo, GitLab Duo)	Средняя	Yandex Tracker с коннектором к GigaChat; Kaiten; кастомные интеграции LLM	Yandex Tracker — функциональный аналог с ИИ-ассистированием; agentic-планирование уровня Atlassian Rovo пока отсутствует

№	Компонент	Назначение	Локализуемость	Российский кандидат / подход	Разрыв
7	ИИ-тест-менеджмент	ИИ-генерация тестов, data generation, coverage analysis (Tricentis, Keysight, UiPath)	Средняя	Allure TestOps, Qase; генерация тестов встроена в GigaCode и SourceCraft; SonarQube для coverage	Зрелые отечественные TMS есть; автономная ИИ-генерация тестов уровня Tricentis пока частична
8	ИИ-генерация приложений	Прототипирование и team tools (Bolt.new, Lovable, Vercel V0, Replit)	Высокая (для внутренних нужд)	Яндекс Cloud Functions + YandexGPT; SberCloud; внутренние low-code платформы; ограниченный доступ к Replit	Публичные сервисы вроде Bolt.new закрыты или требуют VPN; для внутренних прототипов — рабочие альтернативы
9	Детерминированные SWE-инструменты	Линтеры, статанализ, SCA, e2e (ESLint, SonarQube, Snyk, Playwright)	Высокая	SonarQube CE, ESLint, Playwright — полностью доступны; PT Application Inspector, Solar appScreener для SAST/DAST	Практически нет разрыва; международные OSS-инструменты работают on-prem
10	ИИ SRE	Подавление шума, RCA, авто-ремедиация, автоматизация деплоя (Datadog, PagerDuty, Sentry)	Средняя	Grafana с VictoriaMetrics, Zabbix, Prometheus; отечественные AIOps-платформы Monq, wiSLA; кастомный RCA на GigaChat	Нет зрелого отечественного аналога Datadog AI / PagerDuty AIOps; OSS observability stack — функциональный, но без ИИ-слоя без кастомизации

Интерпретация карты

Анализ карты выявляет три зоны с разной стратегией действий.

Зона низкого разрыва (компоненты 4, 8, 9) — системы контекста, генерация приложений и детерминированные инструменты. Здесь российские корпорации могут использовать стандартный OSS-стек или отечественные аналоги без значительных потерь в функциональности. Приоритет — правильная интеграция в единый контур управления, а не поиск замены.

Зона среднего разрыва (компоненты 3, 6, 7, 10) — фоновые агенты, agile-планирование, тест-менеджмент, ИИ SRE. Здесь есть работающие альтернативы, но они требуют либо кастомной интеграции, либо дополнительной разработки ИИ-слоя поверх OSS observability stack. Приоритет — быстрая локализация через open-source + кастомизация.

Зона высокого разрыва (компоненты 1, 2, 5) — агентные IDE, CLI-агенты, governance-платформа. Это критические компоненты, где рыночных on-prem решений с нужным уровнем зрелости нет. Для компонента 5 (governance) наш ответ — Policy Hook Framework + RU Compliance Core как собственная разработка. Для компонентов 1 и 2 — приоритизация open-source agentic-

инструментов (Aider, OpenHands) с локальными моделями как переходный вариант до появления зрелых отечественных продуктов.

Принцип локализации: разрыв в компонентах 1–2 не означает отказ от агентных практик. Собственная среда исполнения может быть высокого качества даже при использовании менее зрелых вендорских продуктов — если правильно выстроены rules-файлы, Policy Hooks и гейты проверки.

8.8 Дорожная карта локализации

Горизонты локализации

Мы выделяем три горизонта дорожной карты локализации: тактический (0–6 месяцев), операционный (6–18 месяцев) и стратегический (18–36 месяцев). Каждый горизонт имеет приоритетные компоненты и измеримые результаты.

Горизонт	Период	Приоритетные компоненты	Ключевые результаты	Метрики готовности
Тактический	0–6 мес.	Компоненты 4, 9 (низкий разрыв); базовый Policy Hook Framework; RU Compliance Core v1.0 (152-ФЗ, основные политики)	On-prem context stack развёрнут; базовые compliance hooks активны; domestic LLM coverage rate > 80%	Compliance rate 152-ФЗ = 100% в pilot-командах; Audit trail работает
Операционный	6–18 мес.	Компоненты 3, 6, 7, 10 (средний разрыв); RU Compliance Core v2.0 (CBR, ФСТЭК); T1–T2 Guardian Agents on-prem	Фоновые агенты на GitVerse; OSS observability + ИИ-слой на GigaChat; ИИ-тест-генерация в CI	Domestic LLM coverage rate = 100% для КИИ; T2 Guardian Agents в production; PDLC Maturity L3
Стратегический	18–36 мес.	Компоненты 1, 2, 5 (высокий разрыв); T3–T5 Guardian Agents; полная федерация Policy Hook Registry	Собственный CLI-агент или зрелый open-source on-prem; T4 blocking Guardian Agent в production; Federated Policy Registry	PDLC Maturity L4 в majority команд; Self-Recovery Rate > 70%; Governance Mesh полностью автономен

Приоритизация инвестиций

Первый приоритет — Policy Hook Framework и RU Compliance Core. Это инвестиция с самым быстрым возвратом: compliance перестаёт быть ручным процессом, а регуляторный риск снижается немедленно. Стоимость разработки v1.0 для основных политик 152-ФЗ — 4–8 недель одной платформенной команды.

Второй приоритет — on-prem context stack и детерминированные инструменты (компоненты 4, 9). Здесь практически нет разрыва с международным ландшафтом, а развёртывание занимает 2–4 недели. Это быстрая победа, которая разблокирует агентные практики для большинства инженерных команд.

Третий приоритет — OSS observability с ИИ-слоем и фоновые агенты (компоненты 3, 10). Это средние инвестиции с высоким долгосрочным эффектом: ИИ SRE замыкает петлю «инцидент → backlog Discovery» и заметно снижает операционную нагрузку на дежурных инженеров.

Четвёртый приоритет — T3–T5 Guardian Agents и агентные CLI-инструменты (компоненты 1, 2, 5). Это самые дорогостоящие инвестиции, которые требуют зрелости IDP на предыдущих уровнях. Начинать с этого без базовой инфраструктуры — антипаттерн: организация потратит ресурсы на сложные возможности, не имея базовых.

Управление рисками локализации

Три ключевых риска дорожной карты, которые необходимо отслеживать как опережающие индикаторы.

Риск «ложного импортозамещения»: замена зарубежных инструментов отечественными аналогами без сохранения функциональной эквивалентности. Признак — снижение Developer Velocity Index при формально выполненных требованиях импортозамещения. Контрмера — функциональный бенчмарк каждого заменяемого компонента перед миграцией.

Риск «shadow AI через личные устройства»: инженеры используют зарубежные ИИ-инструменты через VPN на личных устройствах, обходя корпоративный контур управления. Признак — низкая доля использования отечественных моделей при высокой субъективной оценке продуктивности командами. Контрмера — сильная отечественная альтернатива в корпоративном IDP, которая конкурентоспособна по удобству работы для разработчика.

Риск «перегруженного контура»: чрезмерное количество Policy Hooks замедляет агентные конвейеры и снижает принятие практик командами. Модель надёжно удерживает лишь ограниченный набор директив [оценка объёма — на верификацию], поэтому перегрузка hooks делает их декоративными. Контрмера — регулярный аудит библиотеки: каждые 6 месяцев команда платформы проводит review Policy Hooks и выводит неиспользуемые или конфликтующие правила.

8.9 Выводы

Российский контекст как лаборатория суверенной ИИ-инфраструктуры

Российский enterprise-контекст, при всей своей регуляторной сложности, создаёт уникальное условие: организации, работающие в нём, вынуждены строить то, что международные компании строят добровольно — суверенную, vendor-agnostic, аудитируемую среду исполнения ИИ-агентов. Ограничение оказывается источником архитектурной силы.

Организация, которая выстроила полноценный on-prem контур управления с RU Compliance Core, Policy Hook Framework, Guardian Agents T1–T4 и domestic inference stack, в действительности обладает более зрелой архитектурой, чем большинство международных конкурентов, использующих облачные ИИ-сервисы без собственного governance layer. Когда глобальные регуляторы (EU AI Act, Regulation (EU) 2024/1689, и аналоги в других юрисдикциях) ужесточат требования к трассируемости и аудитируемости ИИ-решений, эта инфраструктура окажется конкурентным преимуществом, а не операционным бременем.

Заключение

Российский enterprise находится в точке, когда решение «начать строить суверенную ИИ-инфраструктуру сейчас или подождать» имеет ясный ответ. Ждать нечего: регуляторные требования уже действуют, зарубежные инструменты уже ограничены, а конкуренты уже начали этот путь. Окно возможностей для «первопроходцев» — организаций, которые выстроят зрелый контур управления до того, как это стало обязательным стандартом — закрывается.

Конкретный первый шаг для команды, принимающей решение о локализации: провести архитектурную диагностику по карте компонентов, определить текущий статус каждого компонента в организации, зафиксировать зоны разрыва и сформировать roadmap на 6–12 месяцев с началом с тактического горизонта. Не нужно реализовывать всё одновременно. Нужно начать с Policy Hook Framework и RU Compliance Core — это фундамент, без которого остальные инвестиции в агентные практики будут строиться на зыбкой почве регуляторной неопределённости.

Финальный принцип российской локализации: ИИ-суверенитет — это не изоляция от мировых практик, а способность применять лучшее из них на собственной инфраструктуре, под собственным контролем, в соответствии с собственными регуляторными обязательствами. Именно это делает среду работы ИИ-агентов стратегическим активом — а не просто технической деталью реализации.

На этом завершается архитектурный контур AI-Disrupt PDLC. От экономического императива (Часть 1) до российской калибровки (Часть 8) мы прошли восемь частей одной взаимосвязанной системы. Теперь — внедрение. Дальше — практика.

Глоссарий

Аббревиатуры и понятия

- **ИИ** — искусственный интеллект (AI). Используется единообразно во всём документе.
- **ПДН** — персональные данные (Personally Identifiable Information, PII).
- **КИИ** — критическая информационная инфраструктура.
- **PDLC** — ИИ-ориентированный жизненный цикл разработки (AI Product Development Life Cycle).
- **IDP** — Integrated Development Platform; единая операционная среда «золотого пути» от исследования до промышленной эксплуатации.
- **SDD** — Specification-Driven Development; контракт исполнения для классических ИТ-систем.
- **MCP** — Model Context Protocol; стандарт интеграции внешних источников инструментов и данных для агентов.
- **ADLC** — цикл разработки ИИ-агентов (Agent Development Life Cycle). Покрывает и методологию, и инструментальный компонент IDP.
- **DORA** — DevOps Research and Assessment; методология измерения зрелости команд разработки (Google Cloud).
- **MTTR** — среднее время восстановления (Mean Time to Recover).
- **RPA** — роботизированная автоматизация процессов (Robotic Process Automation).

Концепции и процессы

- **Harness (среда исполнения)** — окружение агентной среды: совокупность управляющего цикла выполнения, контроля прав, инструментов, изоляции, аудита и операционных механизмов.
- **Контур управления (governance)** — три измерения: качество (контур валидации (validation)), политика/доступ и аудит/compliance.
- **Codified Guardrails (кодифицированные ограничения)** — machine-readable governance-ограничения как раздел SDD.
- **Policy Hook Framework** — механизм применения Policy Hooks на каждом этапе цикла реализации.
- **Guardian Agent (агент-защитник)** — специализированный агент для мониторинга и защиты контура управления.
- **RU Compliance Core** — внутренняя библиотека машиночитаемых политик, кодифицирующих требования российского регуляторного контура.
- **Domestic Inference Gate** — механизм блокировки передачи данных ограниченного распространения на endpoints зарубежных провайдеров.
- **Immutable Audit Trail** — append-only журнал всех событий жизненного цикла агентной сессии.
- **Intent Loop (цикл намерения)** — первая петля двухпетлевой модели: от бизнес-замысла до спецификации.
- **Implementation Loop (цикл реализации)** — вторая петля: от спецификации до production.
- **Context Supply Chain** — дисциплина и процесс формирования машиночитаемого контекста.
- **Evidence Bundle (проверочный пакет)** — набор артефактов, подтверждающих соответствие результата требованиям.
- **Golden Path** — рекомендуемый маршрут прохождения задачи через ИИ PDLC.

Уровни зрелости (L0–L5)

- L0 — отсутствие систематических ИИ-практик.
- L1 — эпизодическое использование ИИ-инструментов.
- L2 — регулярное использование; базовая структура SDD.
- L3 — систематическое применение агентных конвейеров; Policy Hook Framework активен.
- L4 — зрелая агентная архитектура; Guardian Agents T1–T4 в production; контур управления автономен.
- L5 — полностью автономная среда исполнения; Self-Recovery Rate > 80%.

Уровни автономии (R0–R5)

- R0 — агент только предлагает действия; человек принимает каждое решение.
- R1 — агент выполняет атомарные задачи под непосредственным надзором.
- R2 — агент выполняет задачи в рамках сессии; ключевые точки требуют одобрения.
- R3 — агент выполняет многошаговые задачи; исключения эскалирует.
- R4 — агент оркестрирует подзадачи; человек участвует только в архитектурных решениях.
- R5 — полностью автономный агент; человек определяет только бизнес-замысел.

Метрики и KPI

- Compliance rate данных субъектов (152-ФЗ) — доля задач с ПДн с задокументированным согласием; цель: 100%.
- Audit Retention Coverage — доля событий, сохранённых с соблюдением требований хранения; цель: ≥ 99,5%.
- Domestic LLM Coverage Rate — доля операций через отечественные модели или approved gateway; цель для КИИ: 100%.
- Уровень суверенности IDP — доля компонентов на отечественном или open-source ПО; цель: по roadmap раздел 8.8.
- Developer Velocity Index (DVI) — модель зрелости McKinsey для измерения производительности разработки.
- CTOR / Cost-to-Outcome Ratio (отношение стоимости к результату) — ключевая метрика экономической эффективности агентных задач.
- Self-Recovery Rate — доля инцидентов, устранённых автоматически без участия человека; цель для L5: > 80%.
- Domestic LLM Coverage Rate для КИИ: 100%; признак нарушения — теневой ИИ.

Антипаттерны

- Теневой ИИ (shadow AI) — использование зарубежных ИИ-инструментов в обход корпоративного контура управления.
- Ложное импортозамещение — замена зарубежных инструментов отечественными аналогами без сохранения функциональной эквивалентности.
- Перегруженный контур — чрезмерное количество Policy Hooks, делающее их декоративными из-за снижения производительности.
- Инструмент, лежащий на полке (shelf-ware) — внедрённый инструмент, не используемый командами в рабочем процессе.

Роли, протоколы, принципы

- DPO — ответственный за защиту данных (Data Protection Officer).
- ИИ Security Officer — ответственный за безопасность ИИ-систем в организации.
- AgentOps — команда операционного сопровождения агентных конвейеров.
- Compliance-as-Code — принцип кодификации регуляторных требований как исполняемых Policy Hooks.
- Apprenticeship (практическое наставничество) — год работы под руководством senior product engineer как основной формат передачи ИИ-нативных практик.
- ГосСОПКА — государственная система обнаружения, предупреждения и ликвидации последствий компьютерных атак.
- PR/FAQ — пресс-релиз и часто задаваемые вопросы (Press Release / FAQ); артефакт суб-цикла исследования по подходу Amazon Working Backwards.



Короткая версия
доступна по ссылке

Июнь 2026